



University of Applied Sciences
Fakultät für Informatik und Wirtschaftsinformatik

Maschinelles Lernen

Zusammenfassung zur Klausur

Teil I: Unüberwachtes Lernen (Preisach)

Teil II: Überwachtes Lernen & Neuronale Netze (Janka)

Sommersemester 2026

Referenten: Prof. Dr. Christine Preisach
Prof. Dr. Dennis Janka

Ersteller: Devin Uyan

Inhaltsverzeichnis

Über dieses Dokument	4
I Grundlagen & Unüberwachtes Lernen	
(Vorlesung Prof. Preisach)	5
1 Einführung: Was ist Maschinelles Lernen ?	5
1.1 Definition und Grundidee	5
1.2 Die vier Schritte jedes ML-Projekts	6
1.3 Taxonomie: Die Lernparadigmen	6
1.4 Daten: Objekte, Merkmale, Dimension	7
1.5 Fluch der Dimensionalität (Curse of Dimensionality)	8
1.6 Dünn besetzte Datenbestände (Sparsity)	8
1.7 Skalierung von Merkmalen	9
2 Distanz- und Ähnlichkeitsmaße	9
2.1 Motivation: Lernen über Ähnlichkeit	9
2.2 Die Minkowski-Familie	10
2.3 Metrik: Die drei Axiome	11
2.4 Varianz, Kovarianz und Korrelation	12
2.5 Mahalanobis-Distanz	13
2.6 Dynamic Time Warping (DTW)	15
2.7 Ähnlichkeitsmaße	16
2.8 Entscheidungshilfe: Welches Maß für welche Daten ?	17
3 Clustering	17
3.1 Aufgabenstellung und Charakterisierung	17
3.2 k-Means	18
3.3 Hierarchisches Clustering	21
3.4 DBSCAN: Dichtebasiertes Clustering	22
4 Transformationen: PCA und Kernel-PCA	24
4.1 Motivation: Dimensionsreduktion	24
4.2 Mathematische Grundlagen: Eigenwerte und Eigenvektoren	24
4.3 Der PCA-Algorithmus	25
4.4 Kernel-Trick und Kernel-PCA	28

5	Entscheidungsbäume & Ensemble Learning	29
5.1	Klassifikation, Wiederholung	29
5.2	Entscheidungsbäume	29
5.3	Ensemble Learning	31
6	Zeitreihen & Forecasting	34
6.1	Zeitreihen und ihre Eigenschaften	34
6.2	Datenaufbereitung	36
6.3	Forecasting als überwachtes Lernen	36
6.4	Naive Vorhersagemethoden	37
6.5	Methoden im Überblick	37
6.6	Evaluation von Zeitreihenmodellen	38
 II Überwachtes Lernen & Neuronale Netze		
	(Vorlesung Prof. Janka)	39
7	Überwachtes Lernen: Das Framework	39
7.1	Lernen als Funktionssuche	39
7.2	Die Loss-Funktion	40
7.3	Das eigentliche Ziel: Generalisierungsfehler	40
8	Lineare Regression	41
8.1	Einfache lineare Regression	41
8.2	Multiple lineare Regression	42
8.3	Nichtlineare Features	43
8.4	Bias-Variance-Tradeoff	43
9	Optimierung	44
9.1	Grundbegriffe	44
9.2	Optimalitätsbedingungen	44
9.3	Analytische Lösung der linearen Regression: Normalengleichung	44
9.4	Gradientenabstieg	45
10	Klassifizierung und logistische Regression	46
10.1	Klassifizierung vs. Regression	46
10.2	Das logistische Regressionsmodell	46
10.3	Evaluation: Konfusionsmatrix und Metriken	48
10.4	Multi-Klassen-Klassifizierung: Softmax	49
10.5	Nichtlineare Entscheidungsgrenzen und Regularisierung	49

11 Generalisierung, Overfitting und Regularisierung	50
11.1 Drei Fehlerbegriffe	50
11.2 Overfitting auf den Trainingsdaten	50
11.3 Overfitting auf den Testdaten (die subtilere Falle)	51
11.4 Kreuzvalidierung	51
11.5 Regularisierung: Ridge und Lasso	52
11.6 Data Leakage	53
12 Neuronale Netze: Grundlagen	54
12.1 Motivation: Warum reichen lineare Modelle nicht?	54
12.2 Das künstliche Neuron	54
12.3 Aktivierungsfunktionen	55
12.4 Netze mit einer verdeckten Schicht	55
12.5 Tiefe Netze (Deep Learning)	56
13 Training von Neuronalen Netzen	57
13.1 Ausgangslage	57
13.2 Vom Gradientenabstieg zum SGD	57
13.3 Backpropagation: Gradienten automatisch berechnen	59
13.4 Probleme beim Training und Lösungsansätze	61
14 Faltungsnetze (CNNs)	63
14.1 Motivation	63
14.2 Die Faltungsoperation	63
14.3 Faltungsschichten im neuronalen Netz	64
14.4 Pooling	65
14.5 Typische Architekturen	65
15 Autoencoder	66
15.1 Idee und Aufbau	66
15.2 Lineare Autoencoder und PCA	67
15.3 Anwendungen	68
16 Sequenzielle Entscheidungsprobleme & Reinforcement Learning	70
16.1 Grundbegriffe: Zeitdiskrete dynamische Systeme	70
16.2 Policy und Problemformulierung	71
16.3 Lösungsstrategien	71
III Anhang	73
17 Formelsammlung (Schnellreferenz)	73

Über dieses Dokument

Diese Zusammenfassung deckt **beide Vorlesungsteile** der Kombiklausur *Maschinelles Lernen* ab (120 Minuten, je 60 Minuten pro Teil):

- **Teil I (Prof. Preisach):** Modellbasiertes & unüberwachtes Lernen, Distanz- und Ähnlichkeitsmaße, Clustering, Transformationen (PCA, Kernel-PCA), Entscheidungsbäume & Ensemble Learning (Voting, Bagging, Random Forest, Boosting/XGBoost).
- **Teil II (Prof. Janka):** Überwachtes Lernen, lineare/logistische Regression, Optimierung, Generalisierung, neuronale Netze, Training (SGD, Backpropagation), Faltungsnetze, Autoencoder.

Wie dieses Dokument aufgebaut ist

Jedes Kapitel folgt demselben Muster: Erst die *Intuition* (warum gibt es das Konzept überhaupt?), dann die *präzise Definition mit Formeln*, dann *durchgerechnete Beispiele* und zuletzt der *Klausur-Fokus*. Die farbigen Boxen haben feste Bedeutungen:

Intuition: einfach erklärt

So sehen **Intuitionsboxen** aus: Sie erklären ein Konzept ohne Formeln, „wie für einen Fünfjährigen“. Wer ein Kapitel nicht versteht, liest zuerst nur diese Boxen.

Merke

Merkboxen enthalten die Kernaussage eines Abschnitts, das Minimum, das man auswendig können muss.

Beispiel

Beispielboxen enthalten vollständig durchgerechnete Zahlenbeispiele, exakt in dem Stil, wie sie in der Klausur abgefragt werden.

Klausur-Fokus

Klausur-Fokus-Boxen sagen, was von diesem Thema in den Altklausuren (SS22, WS22/23, SS23, WS23/24, WS25/26) tatsächlich gefragt wurde und in welcher Form.

Das zugehörige Dokument **Übungen.pdf** enthält alle Altklausuraufgaben beider Teile, neu formuliert, ausführlicher gelöst als in den offiziellen Lösungen sowie zusätzliche selbst erstellte Aufgaben pro Themengebiet.

Teil I

Grundlagen & Unüberwachtes Lernen

(Vorlesung Prof. Preisach)

1 Einführung: Was ist Maschinelles Lernen ?

1.1 Definition und Grundidee

Intuition: einfach erklärt

Klassische Programmierung: Ein Mensch schreibt *Regeln* (Code), der Computer wendet sie auf *Daten* an und produziert *Antworten*.

Maschinelles Lernen dreht das um: Der Computer bekommt *Daten* und *Antworten* (Beispiele) und findet die *Regeln* selbst. Statt einem Kind zu erklären, was eine Katze ausmacht („spitze Ohren, Schnurrhaare, ...“), zeigt man ihm einfach 1 000 Katzenbilder, es lernt das Muster von allein.

Merke

„[Machine Learning is the] field of study that gives computers the ability to learn without being explicitly programmed.“ (Arthur Samuel, 1959).

ML-Prozesse bilden **Daten auf Wissen** ab:

Es werden **Modelle** (= mathematische Funktionen) identifiziert, die

- (1) empirische Daten akkurat beschreiben und
- (2) **Vorhersagen** für neue, ähnliche Situationen treffen können (**Generalisierbarkeit**).

1.1.1 Historisches Beispiel: Gauß und Ceres (1801)

Das Grundprinzip ist über 200 Jahre alt: Der Zwergplanet Ceres wurde 1801 von Piazzi entdeckt, 41 Tage beobachtet und ging dann „verloren“. Carl Friedrich Gauß bestimmte aus den Beobachtungsdaten die Bahnparameter (**Modell**: Ellipsengleichung + Keplersche Gesetze; **Lernen**: Finden der plausibelsten Parameter mit seiner neu entwickelten **Methode der kleinsten Quadrate**) und sagte die Position vorher. Von Zach fand Ceres am 31.12.1801 exakt an der vorhergesagten Stelle wieder, Training, Vorhersage und Validierung in Reinform.



Abbildung 1: Universelle ML-Pipeline: von Gauß' Bahnberechnung bis zu modernen neuronalen Netzen identisch.

1.2 Die vier Schritte jedes ML-Projekts

Am Beispiel „Katten von Hunden unterscheiden“ (Janka, Kapitel 1):

1. Datensammlung:

Trainingsdatensatz mit bekannten *Labels* aufbauen (Katze/Hund pro Bild).
Labeling ist oft der teuerste Schritt.

2. Feature Design:

Rohdaten in aussagekräftige Merkmale übersetzen, z. B. (Ohrenform, Nasengröße).
Jedes Bild wird dadurch ein *Punkt im Merkmalsraum* $\mathbb{R}^2$.
Features können auch automatisch gelernt werden (Deep Learning).

3. Modelltraining:

Geometrisches Problem. finde eine Linie/Kurve, die die Klassen im Feature-Raum trennt.
Training = Finden/Optimieren der Parameter.

4. Modellvalidierung:

Auf *ungesehenen* Daten (Testdatensatz) prüfen, ob das Modell generalisiert.
Schlechte Ergebnisse $\Rightarrow$ Diversere Daten, andere Features / komplexeres Modell.

Merke

Mathematisches Framework:

Daten = Vektoren,
Modelle = Funktionen zwischen Vektorräumen,
Lernen = Finden der besten Funktionsparameter.
Benötigte Mathematik: Lineare Algebra + Analysis + Optimierung (+ Statistik).

1.3 Taxonomie: Die Lernparadigmen

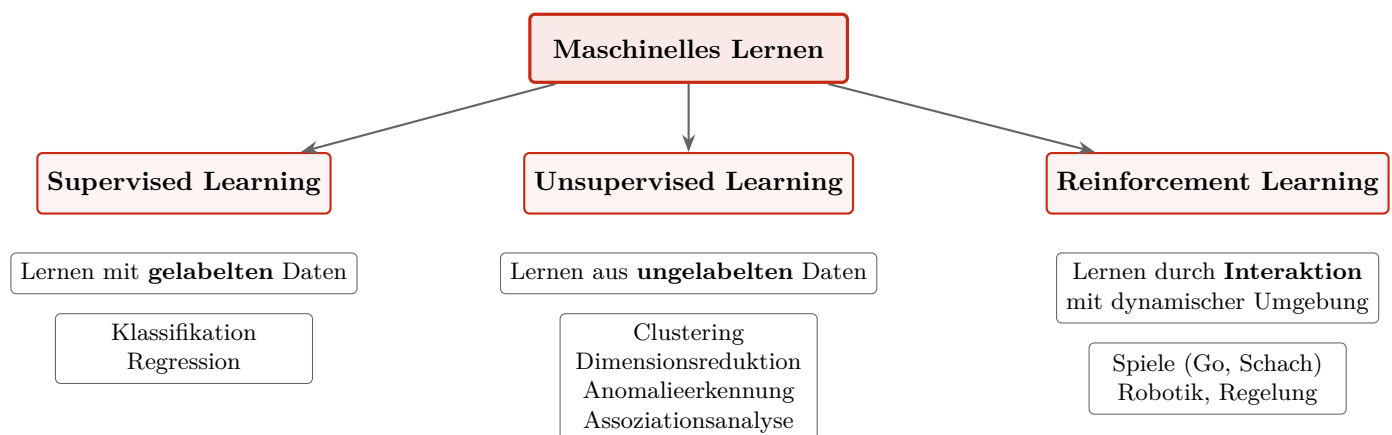


Abbildung 2: Die drei Hauptparadigmen des ML. Dazwischen liegen Mischformen:
Semi-Supervised Learning (wenige Labels + viele ungelabelte Daten) und
Active Learning (Labels werden während des Lernens angefragt).

Aufgabe	Beschreibung	Beispiel
Klassifikation (überwacht)	Neue Objekte werden einer vorgegebenen Klasse zugewiesen	Spamfilter, Bilderkennung, Betrugserkennung
Regression (überwacht)	Vorhersage einer kontinuierlichen Größe , oft über die Zeit	Aktienkurs, Temperatur, Energieverbrauch
Clustering (unüberwacht)	Aufteilung der Daten in Gruppen ähnlicher Objekte	Dokumenten-Clustering, Marktsegmentierung
Dimensionsreduktion (unüberwacht)	Hochdimensionale Daten mit wenigen Merkmalen möglichst gut beschreiben	PCA, Autoencoder, Word Embeddings
Anomalieerkennung (beides)	Objekte finden, die sich signifikant unterscheiden	Kreditkartenbetrug, Predictive Maintenance, Waldschäden
Assoziationsanalyse (unüberwacht)	Stark abhängige Merkmale in den Daten entdecken	Warenkorbanalyse („Bier $\Rightarrow$ Windeln“), Recommender

Tabelle 1: ML-Problemstellungen dieser Vorlesung im Überblick.

Merke

Deep Learning ist *kein* viertes Paradigma, sondern eine Modellfamilie (tiefe neuronale Netze, ≥ 2 verdeckte Schichten), die in *allen* drei Kategorien eingesetzt werden kann (Deep Supervised / Deep Unsupervised / Deep Reinforcement Learning).

1.4 Daten: Objekte, Merkmale, Dimension

Ein **Datenbestand** ist eine Menge von N **Objekten** $O_1, \dots, O_N$ (Zeilen). Jedes Objekt wird durch d **Merkmale** (Features, Attribute, Spalten) beschrieben.

- **Umfang** des Datenbestands = Anzahl der Objekte N (Zeilen)
- **Dimension** des Datenbestands = Anzahl der Merkmale d (Spalten)

Wir setzen i. d. R. voraus, dass die Daten **vektorisert** sind: jedes Objekt ist ein Vektor $\vec{g}_i \in \mathbb{R}^d$, $i = 1, \dots, N$. Merkmale werden als Stichproben von Zufallsvariablen $X_1, \dots, X_d$ aufgefasst; geschätzt werden z. B. Erwartungswert μ (durch den Mittelwert) und Varianz σ^2 (durch die korrigierte empirische Varianz).

1.4.1 Merkmalstypen

Typ	Beispiel	Eigenschaft
qualitativ (kategorisch)	<code>lane</code> $\in \{\text{BOTTOM, MID, TOP}\}$, <code>championId</code>	keine sinnvolle Ordnung/Arithmetik
quantitativ (numerisch)	<code>goldEarned</code> = 54239, Temperatur	Arithmetik sinnvoll (Differenzen, Mittelwerte)
binär	Hochschulabschluss $\in \{0, 1\}$	Spezialfall; Basis für SMC/Jaccard (Kap. 2.7)

Tabelle 2: Merkmalstypen.

Reale Datensätze sind *heterogen*: quantitativ und qualitativ bunt gemischt.

1.4.2 Anwendungsdomänen und ihre Datenstrukturen

Die Struktur der Daten ist stark anwendungsabhängig, alle müssen am Ende auf *Vektoren* abgebildet werden:

- **Computer Vision:** Bild = Vektor der Pixelwerte. Graubild $H \times B \rightarrow \mathbb{R}^{H \cdot B}$; Farbbild (RGB) $\rightarrow \mathbb{R}^{3 \cdot H \cdot B}$, Wertebereich $0 \dots 255$.
- **NLP / Text Mining:** *Bag of Words* (BOW): jedes Dokument = Vektor der Worthäufigkeiten; *Set of Words*: nur binär vorhanden/nicht vorhanden. Mit n -Grammen entstehen Merkmalsräume mit $> 100\,000$ Dimensionen $\Rightarrow$ Reduktion durch LSA, PCA, word2vec (Embeddings, vgl. Autoencoder Kap. 15).
- **Warenkorbanalyse:** Transaktionsdaten (TID + Item-Menge), extrem hochdimensional und dünn besetzt.
- **Social Network Analysis:** Graphdaten, dargestellt als *Adjazenzmatrix*.
- **Gesten-/Aktivitätserkennung:** Sequenzen von 2D/3D-Koordinaten; *unterschiedlich lang!* $\Rightarrow$ braucht spezielle Distanzmaße (DTW, Kap. 2.6).
- **Zeitreihen:** Umweltmonitoring, Satellitenbilder (Sentinel-2: 12 Kanäle, Zeitreihe pro Pixel für Waldschadenserkenkung).

1.5 Fluch der Dimensionalität (Curse of Dimensionality)

Intuition: einfach erklärt

Stell dir vor, du verteilst 100 Bonbons auf einer Linie von 1 m, die liegen dicht. Auf einer Fläche von 1×1 m wird es schon leer. In einem Würfel $1 \times 1 \times 1$ m findest du kaum noch eins. Mit jeder zusätzlichen Dimension wächst das Volumen *exponentiell*, dieselbe Datenmenge wird immer „dünnere“ verteilt. Genau das passiert mit Datenpunkten im Merkmalsraum.

Begriff von Richard **Bellman**. Das Volumen eines Merkmalsraums wächst exponentiell mit der Dimension; eine Datenmenge fester Größe besetzt ihn zunehmend dünner. **Folgen:**

- Benötigte Datenmenge wächst exponentiell (Jede Kombination von Merkmalsausprägungen sollte $\geq 5 \times$ vorkommen).
- **Distanzen verlieren Aussagekraft:** in hohen Dimensionen sind alle Punktabstände ähnlich groß, fatal für k-NN & Co.
- **Overfitting:** Modelle passen sich an das Training an und generalisieren schlecht.
- **Rechenaufwand explodiert.**

Gegenmittel: Dimensionsreduktion in der Vorverarbeitung (PCA, Kap. 4).

1.6 Dünn besetzte Datenbestände (Sparsity)

Viele Merkmale weichen nur selten von einem Standardwert (meist 0) ab.

Beispiel: Studierenden-Datenbank mit allen Vorlesungen der HKA als binäre Merkmale; die Mehrheit der Einträge ist 0.

Konsequenzen:

- Effizienter, nur die Nicht-Null-Werte zu betrachten.
- Ähnlichkeitsmaße müssen das berücksichtigen: gemeinsame Nullen sagen nichts aus $\Rightarrow$ Jaccard statt SMC (Kap. 2.7), Kosinus für Dokumente.

1.7 Skalierung von Merkmalen

Merkmale haben oft völlig verschiedene Wertebereiche (Alter 10 bis 100 vs. Einkommen 0 bis 500 000). Viele Verfahren (lineare/logistische Regression, alle distanzbasierten Methoden, PCA, Gradientenverfahren) brauchen **gleich skalierte** Variablen, sonst dominiert das großskalige Merkmal.

Methode	Formel	Eigenschaften
Min-Max-Normalisierung	$x' = \frac{x - \min(x)}{\max(x) - \min(x)}$	skaliert auf $[0, 1]$; nicht robust gegen Ausreißer
Z-Score-Standardisierung	$x' = \frac{x - \bar{x}}{\sigma}$	Mittelwert 0, Varianz 1; Standard im ML; Annahme Normalverteilung

Tabelle 3: Die beiden Standardskalierungen. Wichtig (Kap. 11): Parameter $\bar{x}, \sigma$ bzw. $\min, \max$ werden *nur auf den Trainingsdaten* bestimmt und dann auf Trainings- und Testdaten angewendet (sonst Data Leakage).

Beispiel : Warum Skalierung Distanzen verändert

Alter X [Jahre] und Körpergröße Y . Person A = (14, 1,60 m), Person B = (15, 1,75 m).
 Messung in **Metern**: $d_2(A, B) = \sqrt{(15 - 14)^2 + (1,75 - 1,60)^2} = \sqrt{1 + 0,0225} \approx 1,01$
 Messung in **Zentimetern**: $d_2(A', B') = \sqrt{(15 - 14)^2 + (175 - 160)^2} = \sqrt{1 + 225} \approx 15,0$
 Gleiche Personen, gleiche Geometrie, aber die Distanz ändert sich um den Faktor 15, nur weil die Einheit wechselt. Die Minkowski-Metriken sind **nicht skaleninvariant**.

Klausur-Fokus

Einführungsthemen werden selten direkt abgefragt, bilden aber die Begriffsbasis:

- Lernparadigmen sicher zuordnen können („Ist Clustering überwacht oder unüberwacht“)
- Fluch der Dimensionalität: Definition + 2 bis 3 Folgen nennen
- Skalierungsformeln (Min-Max, Z-Score) hinschreiben und anwenden
- Umfang vs. Dimension eines Datenbestands unterscheiden

2 Distanz- und Ähnlichkeitsmaße

2.1 Motivation: Lernen über Ähnlichkeit

Intuition: einfach erklärt

Menschen gruppieren intuitiv nach Nähe: Liegen zwei Punkte im Diagramm beieinander, „gehören sie zusammen“. Maschinelles Lernen formalisiert genau das, aber dafür braucht man eine präzise Antwort auf die Frage: *Wie weit sind zwei Objekte voneinander entfernt?* Distanzmaße sind das Lineal des ML. Fast alles in Teil I baut darauf auf: k-NN klassifiziert nach dem nächsten Nachbarn, k-Means gruppiert nach Distanz zum Zentrum, Anomalieerkennung sucht Punkte, die „weit weg von allem“ sind.

2.1.1 Einschub: k -Nearest-Neighbors (k -NN)

Der einfachste distanzbasierte Klassifikator

(Beispiel: Bälle nach Umfang und Gewicht der Sportart zuordnen):

- **Training:** nur Speichern der Trainingsdaten ($X_{\text{train}}, y_{\text{train}}$) (*lazy learning*).
- **Vorhersage** für $\mathbf{x}$: Berechne alle Distanzen von $\mathbf{x}$ zu X_{train} , bestimme die k nächsten Nachbarn, sage die **häufigste Klasse** unter ihnen vorher (bei Regression: den Mittelwert).

Vorteile	Nachteile
Einfach und intuitiv	Naive Version langsam bei großen Datensätzen ($O(N)$ pro Anfrage)
Für Klassifikation <i>und</i> Regression	Anpassung nötig bei unausgewogenen Klassen (gewichtetes k -NN)
Robuste Baseline, oft besser als komplexe Verfahren	Schwäche: verrauschte/irrelevante Features
Sehr ausdrucksstark (lernt komplexe Hypothesen)	Empfindlich bzgl. hoher Dimensionen (Fluch der Dimensionalität)

Tabelle 4: k -NN: Vor- und Nachteile. Offene Designfragen:
Wahl von k und Wahl des Distanzmaßes.

2.2 Die Minkowski-Familie

Für quantitative Vektoren $\vec{x}, \vec{y} \in \mathbb{R}^d$ ist die **Minkowski-Distanz** die allgemeine Form:

$$d_r(\vec{x}, \vec{y}) = \left(\sum_{k=1}^d |x_k - y_k|^r \right)^{\frac{1}{r}}$$

r	Name	Formel	Anschauung
1	Cityblock / Manhattan	$\sum_k x_k - y_k $	Weg im Straßengitter nur horizontal/vertikal;
2	Euklidisch	$\sqrt{\sum_k (x_k - y_k)^2}$	Luftlinie (Satz des Pythagoras)
∞	Chebyshev / Maximums-Norm	$\max_k x_k - y_k $	größte Koordinatendifferenz; z.B. Schach

Tabelle 5: Die drei wichtigen Spezialfälle der Minkowski-Distanz.

Beispiel : alle drei Distanzen für dieselben Punkte

$\vec{x} = (0, 0)^T, \vec{y} = (3, 3)^T$:

$$d_2(\vec{x}, \vec{y}) = \sqrt{(0-3)^2 + (0-3)^2} = \sqrt{18} \approx 4,24$$

$$d_1(\vec{x}, \vec{y}) = |0-3| + |0-3| = 6$$

$$d_\infty(\vec{x}, \vec{y}) = \max(|0-3|, |0-3|) = 3$$

Es gilt stets $d_\infty \leq d_2 \leq d_1$.

2.2.1 Skalarprodukt $\rightarrow$ Norm $\rightarrow$ Metrik

Von jedem Skalarprodukt lässt sich eine Norm ableiten, von jeder Norm eine Metrik:

$$\langle \vec{x}, \vec{y} \rangle := \vec{x}^T \vec{y} \Rightarrow \|\vec{x}\|_2 := \sqrt{\langle \vec{x}, \vec{x} \rangle} = \left(\sum_{k=1}^d x_k^2 \right)^{1/2} \Rightarrow d_2(\vec{x}, \vec{y}) := \|\vec{x} - \vec{y}\|_2$$

Eine **Norm** misst die „Größe“ (Länge) *eines* Vektors,
eine **Distanz** den Abstand *zweier* Vektoren.

2.2.2 Einheitskreise: Die Geometrie der Normen

Der **Einheitskreis** einer Norm ist die Menge aller Punkte mit Abstand genau 1 vom Ursprung: $\{\mathbf{x} \in \mathbb{R}^2 : \|\mathbf{x}\| = 1\}$. Die Form hängt von der Norm ab:

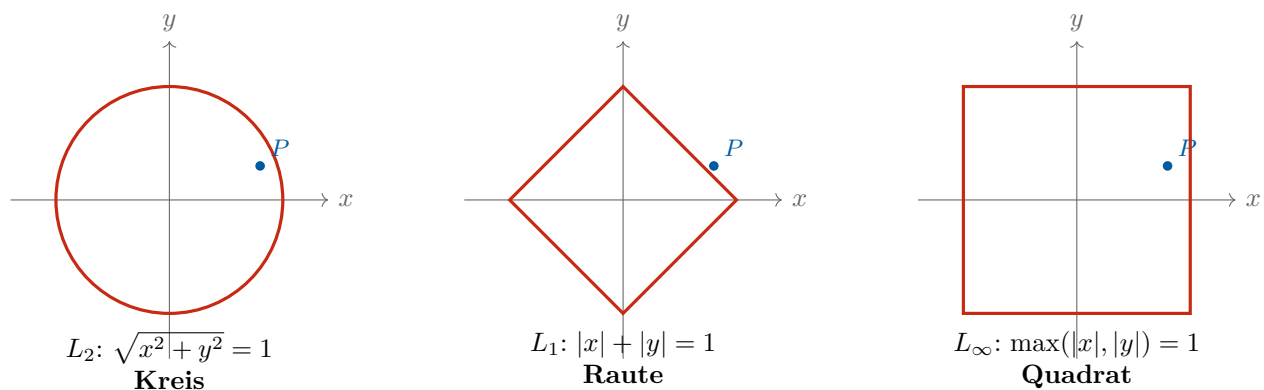


Abbildung 3: Einheits„kreise“ der drei Normen mit dem Punkt $P = (0,8; 0,3)$.

L_2 -Norm: $\sqrt{0,64 + 0,09} \approx 0,86$ (innerhalb);

L_1 : 1,10 (außerhalb!); L_∞ : 0,80 (innerhalb).

Punkt ist je nach Norm „nah“ oder „fern“; Nachbarschaften hängen von der Norm ab.

2.3 Metrik: Die drei Axiome

Merke

Eine Distanzfunktion d heißt **Metrik**, wenn für alle $\vec{x}, \vec{y}, \vec{z}$ gilt:

1. **Positive Definitheit:** $d(\vec{x}, \vec{y}) \geq 0$ und $d(\vec{x}, \vec{y}) = 0 \Leftrightarrow \vec{x} = \vec{y}$
2. **Symmetrie:** $d(\vec{x}, \vec{y}) = d(\vec{y}, \vec{x})$
3. **Dreiecksungleichung:** $d(\vec{x}, \vec{z}) \leq d(\vec{x}, \vec{y}) + d(\vec{y}, \vec{z})$
(Umweg nie kürzer als der direkte Weg)

Alle Minkowski-Distanzen sind Metriken.

Die Mahalanobis-Distanz ist eine Metrik. DTW ist *keine* Metrik.

Gegenbeispiele (Distanzmaße, die keine Metrik sind):

- **Mengendifferenz** $|A - B|$: nicht symmetrisch ($A = \{1, 2, 3, 4\}, B = \{1, 2, 3\}$:
 $A - B = \{4\}$, aber $B - A = \emptyset$). Wohl aber ist $d(A, B) = |A - B| + |B - A|$ eine Metrik.
- **Zeitdifferenz auf der Uhr** (immer vorwärts): $d(13:00, 14:00) = 1$ h,
aber $d(14:00, 13:00) = 23$ h, nicht symmetrisch.

2.4 Varianz, Kovarianz und Korrelation

Diese Begriffe braucht man für die Mahalanobis-Distanz *und* für die PCA (Kap. 4).

2.4.1 Empirische Varianz

Die Varianz beschreibt die **Streuung** eines Merkmals um seinen Mittelwert:

$$\bar{X} = \frac{1}{N} \sum_{k=1}^N x_k \quad \text{Var}(X) = \frac{1}{N-1} \sum_{k=1}^N (x_k - \bar{X})^2$$

Achtung Nenner $N - 1$ (korrigierte/empirische Varianz, Bessel-Korrektur).

Nachteil: nicht in der Originaleneinheit interpretierbar (quadriert)

⇒ Standardabweichung $\sigma = \sqrt{\text{Var}(X)}$.

2.4.2 Empirische Kovarianz

Die Kovarianz zweier Merkmale beschreibt das Maß ihrer **linearen Abhängigkeit**:

$$\text{Cov}(X, Y) = \frac{1}{N-1} \sum_{k=1}^N (x_k - \bar{X}) (y_k - \bar{Y})$$

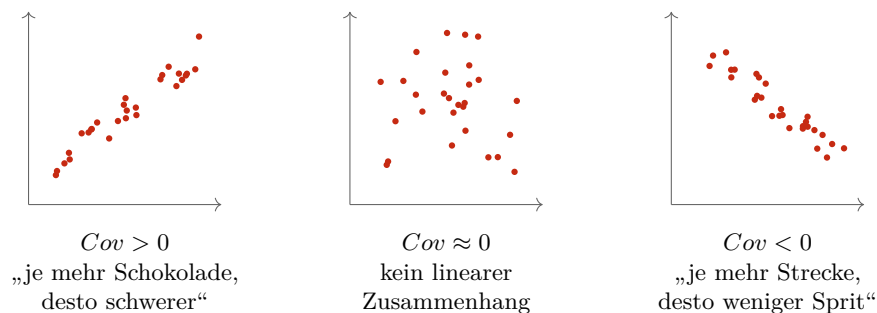


Abbildung 4: Vorzeichen der Kovarianz: positiver, kein und negativer linearer Zusammenhang.

Merke

Eigenschaften der Kovarianz(matrix):

- $\text{Cov}(X, X) = \text{Var}(X)$, auf der Diagonale stehen die Varianzen (nichtnegativ)
- Symmetrie: $\text{Cov}(X, Y) = \text{Cov}(Y, X)$
- **Skalenabhängig:** Einheit ändert den Wert
⇒ Vorzeichen gibt die Richtung des Zusammenhangs, *nicht* dessen Stärke!

Für eine Stärke-Aussage:

Pearson-Korrelation (standardisierte Kovarianz, Werte in $[-1, +1]$):

$$r_{x,y} = \frac{\text{Cov}(X, Y)}{\sqrt{\text{Var}(X) \cdot \text{Var}(Y)}}$$

2.4.3 Kovarianzmatrix über die zentrierte Datenmatrix

Für d Merkmale ist $\mathbf{C} = (c_{ij}) = (\text{Cov}(X_i, X_j)) \in \mathbb{R}^{d \times d}$.

Praktischer Rechenweg über die **zentrierte Datenmatrix** (so auch bei der PCA):

1. **Zentrierung:** $\vec{g}_{mean} = \frac{1}{N} \sum_n \vec{g}_n$, dann $\Delta \vec{g}_n = \vec{g}_n - \vec{g}_{mean}$
(Verschiebung des Koordinatenursprungs in den Schwerpunkt).
2. **Datenmatrix** (zentrierte Vektoren spaltenweise): $G = [\Delta \vec{g}_1 | \dots | \Delta \vec{g}_N] \in \mathbb{R}^{d \times N}$
3. **Kovarianzmatrix:** $\mathbf{C} = \frac{1}{N-1} G G^T \in \mathbb{R}^{d \times d}$

Beispiel : Kovarianzmatrix (Standardbeispiel aus der Vorlesung)

$N = 5$ Punkte: $\begin{pmatrix} 0 \\ 2 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 0 \\ -2 \end{pmatrix}, \begin{pmatrix} -1 \\ 0 \end{pmatrix}, \begin{pmatrix} 0 \\ 0 \end{pmatrix}$.

Mittelwerte: $\bar{X} = \frac{1}{5}(0+1+0-1+0) = 0$, $\bar{Y} = \frac{1}{5}(2+0-2+0+0) = 0$ (schon zentriert).

Varianzen/Kovarianz:

$$\begin{aligned} Var(X) &= \frac{1}{4} (0 + 1 + 0 + 1 + 0) = \frac{1}{2} \\ Var(Y) &= \frac{1}{4} (4 + 0 + 4 + 0 + 0) = 2 \\ Cov(X, Y) &= \frac{1}{4} (0 \cdot 2 + 1 \cdot 0 + 0 \cdot (-2) + (-1) \cdot 0 + 0) = 0 \end{aligned}$$

$$\Rightarrow \mathbf{C} = \begin{pmatrix} \frac{1}{2} & 0 \\ 0 & 2 \end{pmatrix}$$

Interpretation: Y streut viermal stärker als X ; kein linearer Zusammenhang.

2.5 Mahalanobis-Distanz

Intuition: einfach erklärt

Die euklidische Distanz fragt nur: „Wie weit weg?“ Die Mahalanobis-Distanz fragt: „Wie ungewöhnlich relativ zur Form der Daten?“ Liegt eine Punktwolke wie eine gestreckte Ellipse im Raum, dann ist ein Punkt am Ende der langen Achse „normal“ (dort streuen die Daten eben stark), während ein gleich weit entfernter Punkt quer zur Wolke ein Ausreißer ist. Mahalanobis rechnet die Streuung heraus, wie ein Lineal, das sich der Datenform anpasst.

Merke

$$d_C(\vec{x}, \vec{y}) = \sqrt{(\vec{x} - \vec{y})^T \mathbf{C}^{-1} (\vec{x} - \vec{y})}$$

mit $\mathbf{C}^{-1}$ = invertierte (empirische) Kovarianzmatrix des Datenbestands.

Spezialfall: Mit $\mathbf{C} = I$ (unkorrelierte Merkmale, Varianz je 1)
wird sie zur euklidischen Distanz $d(\vec{x}, \vec{y}) = \sqrt{(\vec{x} - \vec{y})^T I (\vec{x} - \vec{y})}$.

Geometrisch führt die Mahalanobis-Distanz drei Operationen durch:

- (1) Verschiebung um μ ,
- (2) **Drehung** (Eigenvektoren der Kovarianzmatrix, vgl. PCA),
- (3) **Skalierung** durch die inverse Kovarianz.

Punkte gleicher Mahalanobis-Distanz liegen auf **Ellipsen** (statt Kreisen):

Richtungen mit hoher Varianz zählen weniger, Richtungen mit geringer Varianz mehr.

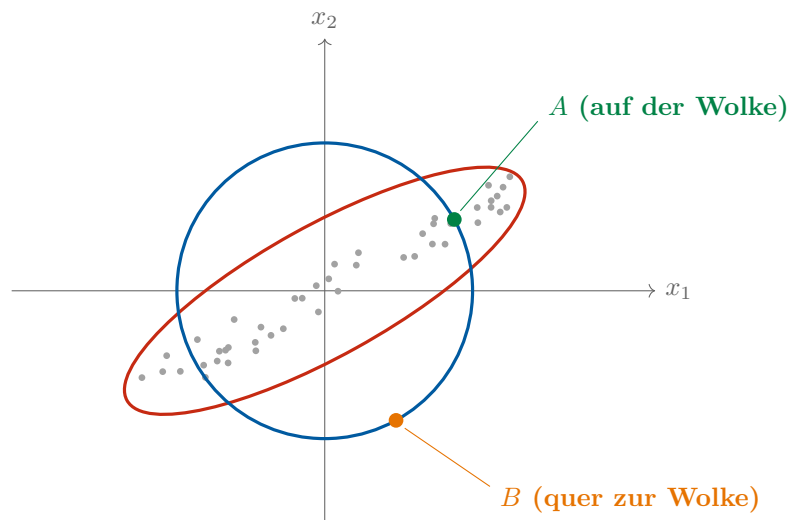


Abbildung 5: A und B haben die **gleiche euklidische Distanz** zum Mittelwert, aber völlig verschiedene **Mahalanobis-Distanzen** (rote Ellipsen gleicher Distanz):

A liegt entlang der natürlichen Streuung (normal), B quer dazu (Ausreißer).

Vorlesungszahlen: $A = (3, 3)$, $B = (3, -3)$: euklidisch je 4,24; Mahalanobis 1,58 vs. 4,74.

Beispiel : Mahalanobis-Distanz berechnen (Vorlesungsaufgabe)

Datenbestand wie oben mit $\mathbf{C} = \begin{pmatrix} 1/2 & 0 \\ 0 & 2 \end{pmatrix}$.

Inverse (Diagonalmatrix $\Rightarrow$ Kehrwerte): $\mathbf{C}^{-1} = \begin{pmatrix} 2 & 0 \\ 0 & 1/2 \end{pmatrix}$.

Abstand von $\vec{x} = \begin{pmatrix} 0 \\ 2 \end{pmatrix}$ zu $\vec{o} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$:

$$d_C(\vec{x}, \vec{o}) = \sqrt{\begin{pmatrix} 0 & 2 \end{pmatrix} \begin{pmatrix} 2 & 0 \\ 0 & 1/2 \end{pmatrix} \begin{pmatrix} 0 \\ 2 \end{pmatrix}} = \sqrt{0 \cdot 0 + 2 \cdot \frac{1}{2} \cdot 2} = \sqrt{2} \approx 1,41$$

Abstand von $\vec{y} = \begin{pmatrix} 1/2 \\ 0 \end{pmatrix}$ zu $\vec{o}$:

$$d_C(\vec{y}, \vec{o}) = \sqrt{\frac{1}{2} \cdot 2 \cdot \frac{1}{2}} = \sqrt{\frac{1}{2}} \approx 0,71$$

Euklidisch wäre $\vec{x}$ viermal so weit weg wie $\vec{y}$ (2 vs. 0,5); Mahalanobis nur doppelt so weit, weil in y -Richtung viel mehr Streuung „normal“ ist.

Wann Mahalanobis ? (Vorteile)	Nachteile
Merkmale mit unterschiedlichen Skalen (eingebaute Standardisierung, skaleninvariant)	Kovarianzmatrix kann singulär sein bei hoher Dimension (mehr Merkmale als Beobachtungen)
Stark korrelierte Merkmale (andere Maße ignorieren Korrelation)	Annahme multivariater Normalverteilung
Anomalieerkennung (große Distanz zum Mittelwert: Ausreißer)	Rechenaufwand der Matrixinversion
Clustering, Klassifikation (Fishers LDA)	Weniger intuitiv interpretierbar

Tabelle 6: Einsatzgebiete und Grenzen der Mahalanobis-Distanz.

2.6 Dynamic Time Warping (DTW)

Intuition: einfach erklärt

Zwei Personen sprechen dasselbe Wort, eine schnell, eine langsam. Die Audiosequenzen sind *unterschiedlich lang* und zeitlich *verschoben*, obwohl der Inhalt gleich ist. Die euklidische Distanz vergleicht stur Zeitpunkt i mit Zeitpunkt i und scheitert. DTW „verbiegt“ (warpt) die Zeitachse: Es sucht die beste Zuordnung der Punkte beider Sequenzen, wie wenn man zwei Gummibänder mit Markierungen so dehnt/staucht, dass sich die Markierungen optimal gegenüberliegen.

Wann reicht die euklidische Distanz auf Zeitreihen nicht ?

- **Offset-Verzerrung:** verschiedene Skalen (Celsius vs. Fahrenheit)
→ vorher normalisieren
- **Verschiebung:** gleiches Muster, zeitlich versetzt
- **Unterschiedliche Längen** $n \neq m \rightarrow$ euklidisch gar nicht definiert

2.6.1 Der DTW-Algorithmus (dynamische Programmierung)

Gegeben zwei Sequenzen $\vec{s} = (s_1, \dots, s_n)$, $\vec{t} = (t_1, \dots, t_m)$ und ein **lokales Maß** $d'(s_i, t_j)$ (z. B. euklidische oder Cityblock-Distanz der Elemente).

1. **Lokale Kostenmatrix** $D' \in \mathbb{R}^{n \times m}$ mit $d'_{ij} = d'(s_i, t_j)$ aufstellen.
2. **Kumulierte Kostenmatrix** per Rekursion (Rand mit ∞ , Start $d_{00} = 0$):

$$d_{ij} = d'_{ij} + \min \{d_{i-1,j}, d_{i,j-1}, d_{i-1,j-1}\}$$

3. **DTW-Distanz** $= d_{nm}$ (rechte obere Ecke);
der zugehörige Pfad ist das optimale Alignment.

Randbedingungen für gültige Pfade ϕ :

- (1) Anfang auf Anfang, Ende auf Ende gematcht;
- (2) **Monotonie:** Indizes steigen monoton, alle Indizes kommen mindestens einmal vor;
- (3) **Schrittweite:** nur $(0, 1)$, $(1, 0)$, $(1, 1)$.

Gesucht: $\hat{\phi} = \arg \min_{\phi} \sum_{(i,j) \in \phi} d'(s_i, t_j)$.

Beispiel : DTW komplett durchgerechnet (Vorlesungsbeispiel)

$\vec{s} = ((0, 0), (1, 0), (1, 4), (2, 5))$, $\vec{t} = ((1, 0), (3, 0), (1, 3))$, lokales Maß: Cityblock.

Schritt 1, lokale Kosten (z. B. $d'(s_1, t_1) = |0 - 1| + |0 - 0| = 1$):

	s_1	s_2	s_3	s_4		s_1	s_2	s_3	s_4
t_3	4	3	1	3	t_3	8	6	4	7
t_2	3	2	6	6	t_2	4	3	7	11
t_1	1	0	4	6	t_1	1	1	5	11

Schritt 2, kumulierte Kosten (rechts; fett = optimaler Pfad),

z. B. $d_{22} = 2 + \min(1, 1, 4) = 3$.

Ergebnis: $d_{DTW}(\vec{s}, \vec{t}) = d_{43} = 7$ mit Alignment $s_1 \rightarrow t_1$, $s_2 \rightarrow t_1$, $s_2 \rightarrow t_2$ (über Diagonale), $s_3 \rightarrow t_3$, $s_4 \rightarrow t_3$.

Vorteile	Nachteile
unterschiedliche Längen vergleichbar	Nicht skaleninvariant (vorher normalisieren!)
Robust gegen Zeitverschiebungen	Rechenintensiv: $O(n \times m)$
Unterstützt unterschiedliche Geschwindigkeiten	Keine Metrik (Dreiecksungleichung verletzt)

Tabelle 7: DTW: Eigenschaften. Anwendungen: Matching/Suche ähnlicher Zeitreihen, Sub-Sequence Matching, Sortieren nach Ähnlichkeit, Clustering von Zeitreihen, Gesten-/Schrift-/Spracherkennung.

2.7 Ähnlichkeitsmaße

Statt Unähnlichkeit (Distanz) kann man auch Ähnlichkeit messen.

Ein **Ähnlichkeitsmaß** s erfüllt:

(1) Abbildung nach $[0, 1]$, (2) Definitheit: $s(\vec{x}, \vec{y}) = 1 \Leftrightarrow \vec{x} = \vec{y}$, (3) Symmetrie.

2.7.1 Binäre Vektoren: SMC und Jaccard

Zähle für zwei binäre Vektoren die Merkmals-Kombinationen:

f_{00} (beide 0), f_{01} , f_{10} , f_{11} (beide 1).

$$\text{SMC}(\vec{x}, \vec{y}) = \frac{f_{11} + f_{00}}{f_{11} + f_{00} + f_{10} + f_{01}} \quad \text{JSC}(\vec{x}, \vec{y}) = \frac{f_{11}}{f_{11} + f_{10} + f_{01}}$$

Intuition: einfach erklärt

Der **Simple Matching Coefficient** zählt gemeinsame Nullen als Übereinstimmung, bei dünn besetzten Daten (Warenkörbe!) wären sich dann *alle* Kunden ähnlich, weil jeder fast alles *nicht* kauft. Der **Jaccard-Koeffizient** ignoriert die f_{00} deshalb komplett: Nur gemeinsame Einsen zählen.

Beispiel : SMC vs. Jaccard

$$\vec{x} = (1, 0, 0, 0, 0, 0, 0, 0, 0, 0)^T, \vec{y} = (0, 0, 0, 0, 0, 0, 0, 1, 0, 1)^T$$

$$\Rightarrow f_{11} = 0, f_{10} = 1, f_{01} = 2, f_{00} = 7.$$

$$\text{SMC} = \frac{0 + 7}{10} = 0,7 \quad (\text{ähnlich!}) \quad \text{JSC} = \frac{0}{0 + 1 + 2} = 0 \quad (\text{korrekt: nichts gemeinsam})$$

2.7.2 Kosinus-Ähnlichkeit (für Dokumente / nicht-binäre Vektoren)

$$d_{\cos}(\vec{x}, \vec{y}) = \frac{\langle \vec{x}, \vec{y} \rangle}{\|\vec{x}\| \|\vec{y}\|} = \cos(\alpha) \in [-1, 1]$$

- $= 1$: gleiche Richtung (Winkel 0°), *unabhängig von der Länge der Vektoren!* (Doppelt so langes Dokument mit gleichen Wortanteilen = maximal ähnlich)
- $= 0$: orthogonal (90°), keine gemeinsamen Terme
- $= -1$: entgegengesetzte Richtung (180°)

Wegen der Längeninvarianz ist die Kosinus-Ähnlichkeit *kein* Ähnlichkeitsmaß im strengen Sinn der Definition (Definitheit verletzt: $s = 1$ auch für $\vec{x} \neq \vec{y}$, wenn $\vec{y} = \alpha \vec{x}$). Standard für BOW-Dokumentvektoren und Embeddings.

2.8 Entscheidungshilfe: Welches Maß für welche Daten?

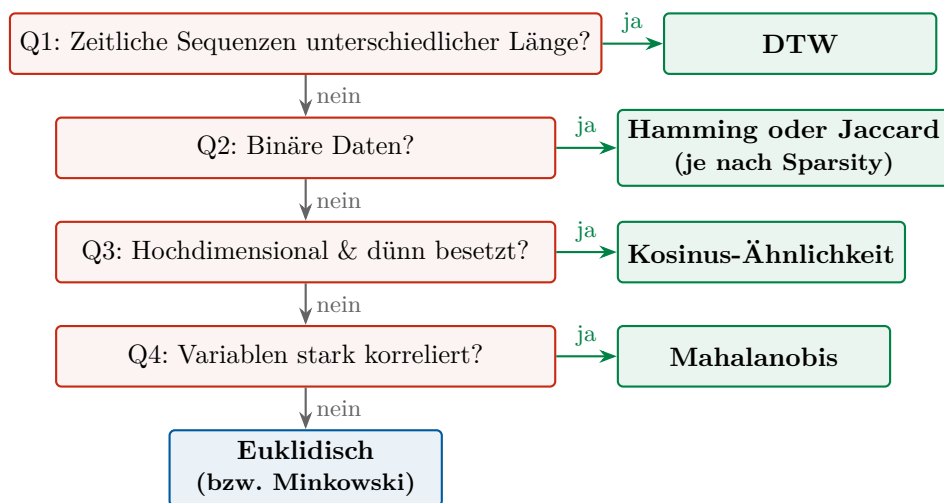


Abbildung 6: Entscheidungsbaum zur Wahl des Distanz-/Ähnlichkeitsmaßes.

Klausur-Fokus

Typische Aufgabenformen zu diesem Kapitel:

- Distanzen ausrechnen (L_1 , L_2 , L_∞ für gegebene Punkte) und bzgl. Einheitskreis einordnen
- Metrik-Eigenschaften benennen / prüfen, warum ein Maß keine Metrik ist (Zeitdifferenz, Mengendifferenz, DTW)
- Kovarianzmatrix für kleinen Datensatz aufstellen (zentrieren, $\frac{1}{N-1}GG^T$), Vorsicht: Nenner $N - 1$!
- Mahalanobis-Distanz mit gegebener (Diagonal-)Kovarianzmatrix berechnen; erklären, wann Mahalanobis statt euklidisch
- DTW: lokale und kumulierte Kostenmatrix füllen, Pfad und Gesamtkosten angeben
- SMC und Jaccard berechnen und erklären, warum Jaccard bei dünnbesetzten Daten besser ist
- Das richtige Maß für gegebene Datensituation wählen und begründen

3 Clustering

3.1 Aufgabenstellung und Charakterisierung

Intuition: einfach erklärt

Clustering ist Sortieren ohne Anleitung: Man bekommt einen Haufen Objekte *ohne Etiketten* und soll sie in Gruppen legen, sodass sich die Objekte innerhalb einer Gruppe ähneln und zwischen den Gruppen unterscheiden. Es gibt keine „richtige“ Antwort vom Lehrer (kein Label!), das macht Clustering zum Paradebeispiel für **unüberwachtes Lernen**.

Formal: Gegeben Datenmenge X (ohne Zielvariable), Parameter k (Anzahl Gruppen), Abstandsmaß $d : X \times X \rightarrow \mathbb{R}$.

Ziel:

1. Abstand **innerhalb** der Gruppen minimieren (Kompaktheit)
2. Abstand **zwischen** den Gruppen maximieren (Trennung)

Ziel 2 passiert meist implizit (kompakte Cluster $\Rightarrow$ großer Inter-Cluster-Abstand);

Ziel 2 allein würde kein gutes Clustering liefern $\Rightarrow$ Trade-off Kompaktheit vs. Trennung.

Anwendungen: Marktsegmentierung (Kundengruppen für Werbung), Bildsegmentierung (MRT: anatomische Bereiche), Musikempfehlung (nutzer- oder inhaltsbasierte Song-Cluster), Keyframe-Extraktion in Videos.

Dimension	Ausprägungen
Verfahren	partitionierend (k-Means, DBSCAN: feste Aufteilung) vs. hierarchisch (Baum von Clustern)
Zugehörigkeit	exklusiv (jeder Punkt in genau 1 Cluster) vs. überlappend vs. fuzzy
Abdeckung	vollständig (jeder Punkt zugeordnet) vs. partiell (DBSCAN: Rauschen bleibt übrig)
Clusteranzahl	vorgegeben (k-Means) vs. dynamisch (DBSCAN, hierarchisch)
Clustertyp	zentrums-, prototyp-, nachbarschafts-, dichte- oder modellbasiert

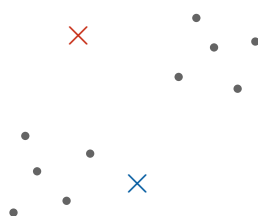
Tabelle 8: Charakterisierung von Clusterverfahren. k-Means ist: partitionierend, exklusiv, vollständig, mit vorgegebenem K , zentrumsbasiert.

3.2 k-Means

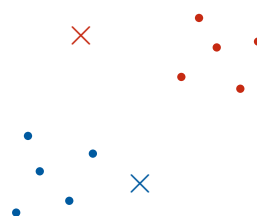
Merke

(Basic) k-Means / Lloyd-Algorithmus:

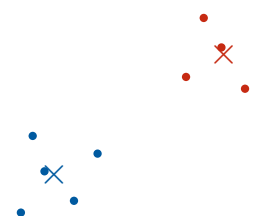
1. Wähle K initiale Punkte als Zentren
2. **repeat**
3. Ordne jeden Punkt dem nächsten Zentrum zu (Assignment)
4. Berechne die Zentren der Cluster neu (Update: Schwerpunkt)
5. **until** Zentren verändern sich nicht mehr



1. Initiale Zentren (zufällig)



2. Punkte dem nächsten Zentrum zuordnen



3. Zentren = Schwerpunkte; wiederholen bis stabil

Abbildung 7: Eine k-Means-Iteration für $K = 2$:

Assignment- und Update-Schritt wechseln sich ab, bis kein Punkt mehr das Cluster wechselt.

3.2.1 Zielfunktion: Sum of Squared Errors (SSE)

Merke

k-Means minimiert (implizit) die Summe der quadrierten Abstände aller Punkte zu ihrem Clusterzentrum:

$$SSE = \sum_{k=1}^K \sum_{\vec{g} \in C_k} \text{dist}(\vec{g}, \vec{z}_k)^2 = \sum_{k=1}^K \sum_{\vec{g} \in C_k} \sum_{i=1}^d (g_i - z_{ik})^2$$

Das entspricht der Minimierung der Varianz innerhalb der Cluster (*Within-Cluster Sum of Squares*). Das exakte Optimierungsproblem ist **NP-hart**, k-Means (Lloyd) ist eine **Heuristik**, die nur ein lokales Minimum findet.

Warum der Schwerpunkt ?

Für SSE mit euklidischer Distanz ist das optimale Zentrum eines Clusters genau sein Mittelwert $\vec{z}_k = \frac{1}{|C_k|} \sum_{\vec{g} \in C_k} \vec{g}$, jede andere Position ergäbe einen höheren SSE.
(Varianten: Median statt Mean, Manhattan- statt euklidische Distanz.)

Beispiel : SSE berechnen (Vorlesungsaufgabe)

Punkte $A(1, 2)$, $B(2, 1)$, $C(1, 1)$, $D(6, 5)$, $E(7, 6)$, $F(6, 7)$;

Zuordnung: Cluster 1 = $\{A, B, C\}$, Cluster 2 = $\{D, E, F\}$.

Zentren: $\vec{z}_1 = \left(\frac{1+2+1}{3}, \frac{2+1+1}{3}\right) = \left(\frac{4}{3}, \frac{4}{3}\right)$, $\vec{z}_2 = \left(\frac{6+7+6}{3}, \frac{5+6+7}{3}\right) = \left(\frac{19}{3}, 6\right)$

Quadrierte Abstände Cluster 1:

$$d^2(A, \vec{z}_1) = \left(1 - \frac{4}{3}\right)^2 + \left(2 - \frac{4}{3}\right)^2 = \frac{1}{9} + \frac{4}{9} = \frac{5}{9}$$

$$d^2(B, \vec{z}_1) = \left(2 - \frac{4}{3}\right)^2 + \left(1 - \frac{4}{3}\right)^2 = \frac{4}{9} + \frac{1}{9} = \frac{5}{9}$$

$$d^2(C, \vec{z}_1) = \left(1 - \frac{4}{3}\right)^2 + \left(1 - \frac{4}{3}\right)^2 = \frac{1}{9} + \frac{1}{9} = \frac{2}{9}$$

Cluster 2: $d^2(D) = \frac{1}{9} + 1 = \frac{10}{9}$, $d^2(E) = \frac{4}{9} + 0 = \frac{4}{9}$, $d^2(F) = \frac{1}{9} + 1 = \frac{10}{9}$

$$SSE = \frac{5+5+2}{9} + \frac{10+4+10}{9} = \frac{12}{9} + \frac{24}{9} = 4$$

3.2.2 Designfreiheiten und Randeffekte

- **Distanzmaß:** euklidisch (Standard), für Dokumente Kosinus-Ähnlichkeit; generell einfache Maße (Rechenaufwand!).
- **Initialisierung:** zufälliges Sampling. Ergebnis hängt davon ab (lokale Minima!)
⇒ *mehrfach* mit verschiedenen Initialisierungen laufen lassen, bestes Ergebnis (kleinster SSE) behalten.
- **Abbruch:** „Zentren ändern sich nicht mehr“ = kein Clusterwechsel; bei großen Daten aufgeweicht („< x % wechseln“).
- **Leere Cluster:** können auftreten. Abhilfe: schlechtestes Cluster splitten oder neues Zentrum beim entferntesten Punkt erzeugen; umgekehrt Cluster verwerfen/vereinigen.

3.2.3 Wahl der Clusteranzahl K : Elbow und Silhouette

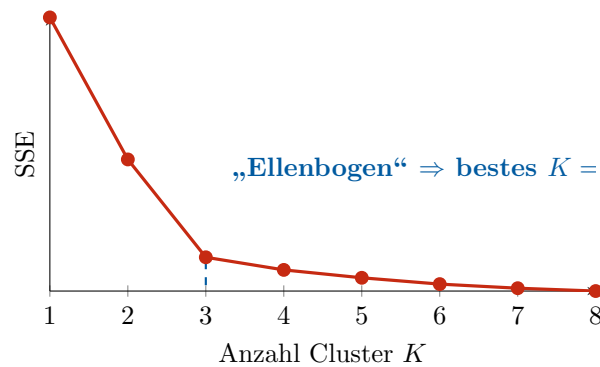


Abbildung 8: **Elbow-Methode**: SSE für verschiedene K berechnen; dort, wo die Abnahme deutlich abflacht (Knick), liegt das beste K .
 K zu klein $\Rightarrow$ viele große Abstände; K zu groß $\Rightarrow$ kaum noch Verbesserung.

Silhouette: Für jedes Objekt $\vec{x}$: $a(\vec{x})$ = Abstand zum *eigenen* Cluster(-schwerpunkt),
 $b(\vec{x})$ = Abstand zum *nächstgelegenen fremden* Cluster.

$$s(\vec{x}) = \frac{b(\vec{x}) - a(\vec{x})}{\max\{a(\vec{x}), b(\vec{x})\}} \in [-1, 1]$$

Der **Silhouettenkoeffizient** ist der Mittelwert über alle Objekte: $\frac{1}{N} \sum \vec{x} s(\vec{x})$.

Wert	Interpretation
nahe +1	Punkt passt gut zum eigenen Cluster, weit von anderen (gute Trennung)
≈ 0	Punkt liegt an der Grenze zwischen zwei Clustern
< 0	Punkt vermutlich dem falschen Cluster zugeordnet

Tabelle 9: Silhouette-Interpretation. Da weitgehend unabhängig von K , gut zum Vergleich verschiedener K geeignet ($K = 2..n$ testen, höchsten Wert wählen).
 Empfehlung: mind. 2 Methoden kombinieren (Elbow + Silhouette).

3.2.4 k -Medoid / PAM: prototypbasierte Variante

Manchmal interessiert weniger das Cluster als sein *Repräsentant* (z. B. repräsentative Keyframes pro Video-Shot). Der **Medoid** ist der *echte Datenpunkt* mit minimaler durchschnittlicher Distanz zu allen anderen Punkten seines Clusters:

$$m_k = \arg \min_{\vec{y} \in C_k} \sum_{\vec{x} \in C_k} \text{dist}(\vec{x}, \vec{y})$$

PAM (Partitioning Around Medoids):

BUILD (initiale Medoids wählen, oft Objekt mit geringster Gesamtdistanz)

$\rightarrow$ **SWAP** (iterativ Medoid gegen non-Medoid tauschen, wenn Distanz sinkt) $\rightarrow$ **Zuweisung**.

Vorteil: robuster gegen Ausreißer als k-Means

(Medoids sind echte Punkte, keine durch Extremwerte verzerrten Mittelwerte).

Nachteile: K vorab nötig, quadratische Laufzeit $\Rightarrow$ nicht für große Datenmengen.

Merke**Grenzen von k-Means & Varianten:**

- Anfällig für **Rauschen/Ausreißer**
(alle Punkte gehen in den Zentroid ein; SSE quadriert Fehler!)
- K schwer zu bestimmen; Ergebnis + Laufzeit hängen stark von der Initialisierung ab
- Cluster müssen **konvex** sein (Voronoi-Zellen, kugelförmig, linear trennbar)
⇒ nichtkonvexe Formen (verschachtelte Ringe, Halbmonde) sind *unmöglich*
→ DBSCAN

3.3 Hierarchisches Clustering

Gruppiert Datenpunkte in **hierarchischen Strukturen** (Baum / Dendrogramm);
benötigt zusätzlich ein Maß für den Abstand *zwischen Clustern*:

- **Agglomerativ (bottom-up)**: Start: jeder Punkt eigenes Cluster;
iterativ die zwei ähnlichsten Cluster verschmelzen, bis alles ein Cluster ist.
- **Divisiv (top-down)**: Start: ein Gesamtcluster;
rekursiv aufteilen, bis jeder Punkt allein steht.

Linkage	Clusterabstand $d(A, B) =$
Single Linkage	minimaler paarweiser Abstand zwischen Punkten aus A und B
Complete Linkage	maximaler paarweiser Abstand (kompakte Cluster)
Average Linkage	Durchschnitt aller paarweisen Abstände
Centroid Linkage	Abstand der Schwerpunkte (virtuelle Punkte)

Tabelle 10: Linkage-Methoden für den Abstand zwischen Clustern.

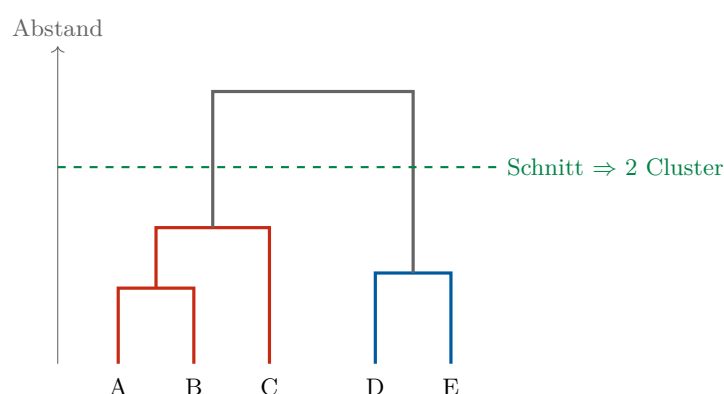


Abbildung 9: **Dendrogramm** (agglomerativ):
A und B verschmelzen zuerst (kleinster Abstand),
dann C dazu; D, E bilden das zweite Cluster.

Ein horizontaler Schnitt auf beliebiger Höhe liefert eine Partition,
die Clusteranzahl muss *nicht* vorab festgelegt werden.

3.4 DBSCAN: Dichtebasiertes Clustering

Intuition: einfach erklärt

k-Means denkt in Kugeln um Zentren, DBSCAN denkt in *Menschenmengen*: Ein Cluster ist überall dort, wo viele Punkte dicht gedrängt stehen, egal welche Form die Menge hat (auch Ringe, Spiralen, Halbmonde). Wer allein in der Pampa steht, gehört zu keiner Gruppe, das ist **Rauschen**. Genau dafür wurde DBSCAN erfunden: Die Idee kommt aus der *Ausreißererkennung*, ein „Nicht-Ausreißer“ hat eine Mindestanzahl Nachbarn in seiner Umgebung.

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) hat zwei Parameter: ε (Nachbarschaftsradius) und *MinPts* (Mindestanzahl Punkte).

3.4.1 Begriffe

- **ε -Umgebung:** $U_\varepsilon(\vec{x}) = \{\vec{y} \in D : \text{dist}(\vec{x}, \vec{y}) < \varepsilon\}$
- **Kernobjekt:** mehr als *MinPts* Objekte in der ε -Umgebung
- **Randobjekt:** kein Kernobjekt, aber von einem Kernobjekt (direkt) erreichbar, bildet den Rand eines Clusters
- **Rauschpunkt:** weder Kern noch erreichbar, gehört zu keinem Cluster (Ausreißer)

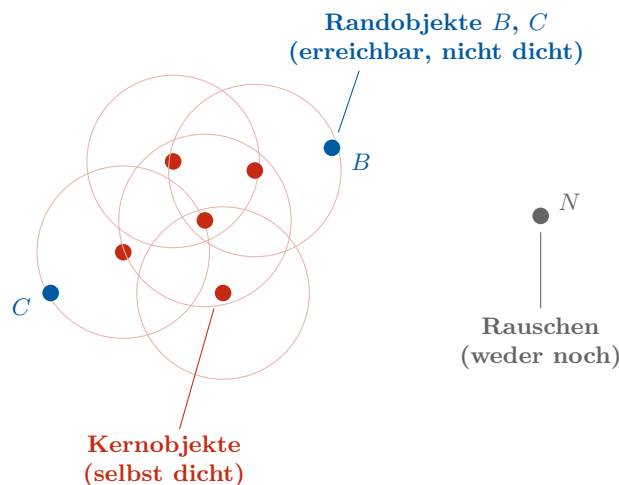


Abbildung 10: Die drei Punkttypen in DBSCAN (Kreise: ε -Umgebungen der Kernobjekte).

3.4.2 Erreichbarkeit und Verbundenheit (Relationen)

- **Direkt erreichbar:** $\vec{x}$ ist direkt erreichbar von $\vec{y}$, wenn $\vec{x} \in U_\varepsilon(\vec{y})$ und $\vec{y}$ Kernobjekt ist. (Nur für Kernobjekte symmetrisch!)
- **Erreichbar:** es existiert eine Kette $\vec{y} = \vec{z}_1, \dots, \vec{z}_n = \vec{x}$, in der $\vec{z}_{i+1}$ jeweils direkt erreichbar von $\vec{z}_i$ ist. Nur das *letzte* Glied darf Randobjekt sein. (Transitiv; symmetrisch nur für Kernobjekte.)
- **Verbunden:** $\vec{x}_1$ und $\vec{x}_2$ sind verbunden, wenn es ein $\vec{y}$ gibt, von dem beide erreichbar sind. (Symmetrisch!)

Merke**Cluster (DBSCAN-Definition):**

Teilmenge C mit mindestens einem Kernobjekt, die erfüllt:

- (1) **Maximalität:** ist $\vec{y} \in C$ und $\vec{x}$ erreichbar von $\vec{y}$, dann $\vec{x} \in C$;
- (2) **Verbundenheit:** alle Punkte in C sind paarweise verbunden.

Ausreißer = Objekte, die zu keinem Cluster gehören.

3.4.3 Algorithmus-Ablauf

1. Parameter ε , *MinPts* wählen
2. Beliebigen unbesuchten Punkt wählen, ε -Nachbarschaft bestimmen
3. $< \text{MinPts}$ Nachbarn $\Rightarrow$ vorläufig als Rauschen markieren;
sonst: Kernpunkt, neues Cluster starten
4. **Expansion:** alle Nachbarn dem Cluster zuordnen;
für jeden Nachbarn, der selbst Kernpunkt ist, rekursiv weiter expandieren;
an Randpunkten endet die Expansion
(frühere „Rauschpunkte“ können noch zu Randpunkten werden)
5. Mit dem nächsten unbesuchten Punkt fortfahren, bis alle verarbeitet sind

3.4.4 Parameterwahl: k -Distanz-Plot

Für jedes Objekt den Abstand zum k -nächsten Nachbarn berechnen (üblich $k = 4$ bei 2D-Daten), aufsteigend sortiert plotten. Erwartung: **steiler Anstieg** (Knick) zwischen Kernpunkten und Rand-/Rauschpunkten. Bei ausgeprägtem Knick: $\text{MinPts} := k$ und ε aus dem Plot ablesen; ohne Knick andere k testen.

Eigenschaft	k-Means	DBSCAN
Clusteranzahl	vorgegeben (K)	ergibt sich automatisch
Clusterform	nur konvex (kugelförmig)	beliebig (dichtebasiert)
Ausreißerbehandlung	alle Punkte zugeordnet	Rauschen explizit erkannt
Parameter	K , Initialisierung	ε , <i>MinPts</i>
Vollständigkeit	vollständig	partiell
Anfälligkeit	Ausreißer, Initialisierung	unterschiedliche Dichten, Parameterwahl

Tabelle 11: k-Means vs. DBSCAN, der wichtigste Vergleich des Kapitels.

Klausur-Fokus

- k-Means-Pseudocode; Charakterisierung (partitionierend, exklusiv, vollständig)
- SSE **berechnen** (Zentren als Mittelwerte, quadrierte Abstände summieren);
- Elbow interpretieren; Silhouette: Formel, Wertebereich, Deutung ($+1/0/<0$)
- DBSCAN: Kern-/Rand-/Rauschpunkte klassifizieren;
- k-Means vs. DBSCAN (nichtkonvex) erklären; Linkage + Dendrogramm

4 Transformationen: PCA und Kernel-PCA

4.1 Motivation: Dimensionsreduktion

Intuition: einfach erklärt

Ein Datensatz mit 100 Merkmalen ist wie ein Foto aus 100 Blickwinkeln, viele Blickwinkel zeigen fast dasselbe. Dimensionsreduktion sucht die *wenigen Blickwinkel, die am meisten zeigen*. Die PCA findet dazu die Richtungen, in denen die Daten am stärksten streuen (*Varianz* = Information), und wirft die Richtungen weg, in denen fast nichts passiert.

Vorteile der Reduktion: weniger Rauschen und irrelevante Merkmale, leichtere Visualisierung, geringere Rechenzeiten, Gegenmittel gegen den Fluch der Dimensionalität (Sparsity, bedeutungslose Abstände). k-NN ist deutlich empfindlicher gegenüber hohen Dimensionen als SVM oder neuronale Netze.

4.1.1 Wege zur Dimensionsreduktion: Merkmalsauswahl

Man unterscheidet **irrelevante** Merkmale (kaum nützliche Information, z. B. Matrikelnummer für Notenvorhersage) und **redundante** Merkmale (verdoppeln vorhandene Information, z. B. Preis und Mehrwertsteuer).

Ansätze zur automatischen Auswahl:

- **Brute-Force** (ideal, unpraktikabel): alle 2^d Teilmengen testen
- **Eingebettete Ansätze:** Auswahl implizit während des Lernens (z. B. Entscheidungsbäume / Random Forests)
- **Filteransätze:** vor dem Lernen, unabhängig vom Verfahren (statistische Tests, Korrelationsanalyse)
- **Wrapper-Ansätze:** wie Brute-Force, aber nur ausgewählte Kombinationen (z. B. Forward Selection)
- **Transformationen:** neue Merkmale als Kombination der alten bilden → **PCA**

4.2 Mathematische Grundlagen: Eigenwerte und Eigenvektoren

Merke

Ein Vektor $\vec{e} \neq \vec{0}$ ist **Eigenvektor** einer Matrix $\mathbf{C}$, wenn es eine Zahl λ (**Eigenwert**) gibt mit

$$\mathbf{C} \vec{e} = \lambda \vec{e}$$

Anschaulich: Die Matrix *dreht* den Eigenvektor nicht, sondern *streckt/staucht* ihn nur um den Faktor λ .

Berechnung in zwei Schritten:

1. Eigenwerte: Nullstellen des charakteristischen Polynoms $\det(\mathbf{C} - \lambda \mathbf{I}) = 0$
2. Eigenvektoren: für jedes λ_i das homogene LGS $(\mathbf{C} - \lambda_i \mathbf{I}) \vec{e}_i = \vec{0}$ lösen

Wichtige Eigenschaften:

- Eigenvektoren sind **nicht eindeutig**:
mit $\vec{e}$ ist auch jedes $\alpha\vec{e}$ Eigenvektor (gleicher Eigenwert).
Der Lösungsraum heißt **Eigenraum**.
Erst durch **Normierung auf Länge 1** werden sie (bis aufs Vorzeichen) eindeutig.
- Konvention: Eigenwerte **absteigend** indizieren: $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_d$.
- Kovarianzmatrizen sind symmetrisch $\Rightarrow$ Eigenvektoren orthogonal,
alle Eigenwerte reell und ≥ 0 .
- Anwendungen: PageRank, PCA, Schwingungsanalyse, Computer Vision (Eigenfaces).

4.3 Der PCA-Algorithmus**Merke**

PCA in 5 Schritten (Trainingsdaten $\vec{g}_1, \dots, \vec{g}_N \in \mathbb{R}^d$):

- Zentrieren:** $\vec{g}_{mean} = \frac{1}{N} \sum_n \vec{g}_n$; $\Delta\vec{g}_n = \vec{g}_n - \vec{g}_{mean}$
- Datenmatrix:** $G = [\Delta\vec{g}_1 | \dots | \Delta\vec{g}_N] \in \mathbb{R}^{d \times N}$; **Kovarianzmatrix:** $C = \frac{1}{N-1} G G^T$
- Eigenwertzerlegung:**
Eigenwerte $\lambda_1 \geq \dots \geq \lambda_d \geq 0$ und normierte Eigenvektoren $\vec{e}_1, \dots, \vec{e}_d$ von C ;
Faktorisierung $C = U \Sigma U^T$ mit $U = [\vec{e}_1 | \dots | \vec{e}_d]$ orthogonal ($U^{-1} = U^T$),
 $\Sigma = \text{diag}(\lambda_1, \dots, \lambda_d)$
- Transformation** (Hintransformation und Rekonstruktion):
$$\vec{g}' = U^T(\vec{g} - \vec{g}_{mean}) \quad \vec{g} = \vec{g}_{mean} + U \vec{g}'$$
- Reduktion:** nur die ersten i_0 Komponenten von $\vec{g}'$ behalten
(Komponenten mit kleinem λ_i weglassen)

Die Eigenvektoren sind die **Hauptkomponenten** (neue Achsen),
die Eigenwerte λ_i sind die **Varianzen entlang dieser Achsen**.

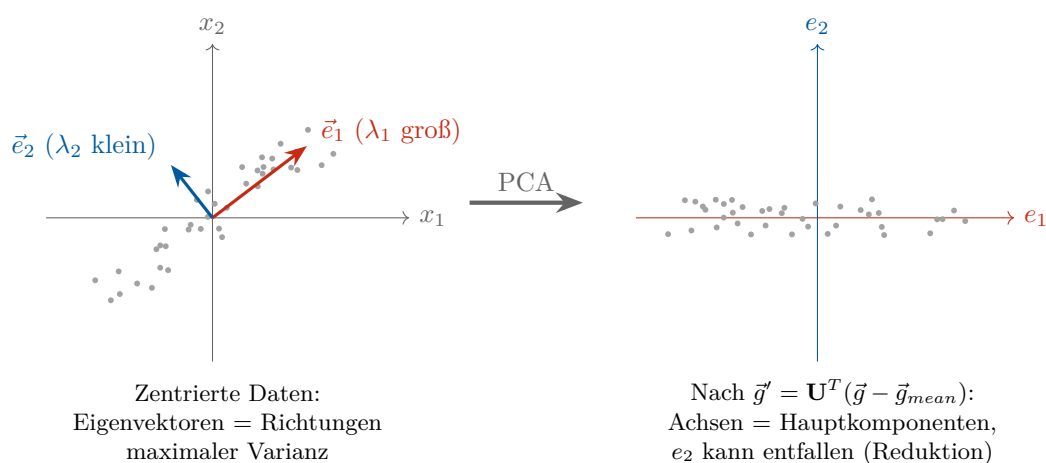


Abbildung 11: PCA = Drehung des Systems in die Richtungen maximaler Streuung.
Nach der Transformation sind die Komponenten **unkorreliert**,
die meiste Information steckt in den ersten Komponenten.

4.3.1 Erklärte Varianz und Wahl von i_0

Die Gesamtvarianz ist die Spur der Kovarianzmatrix ($\text{tr}(\mathbf{C}) = \sum_i \lambda_i$).
Der Anteil der Hauptkomponente i :

$$\text{relVar}_i = \frac{\lambda_i}{\sum_{j=1}^d \lambda_j}$$

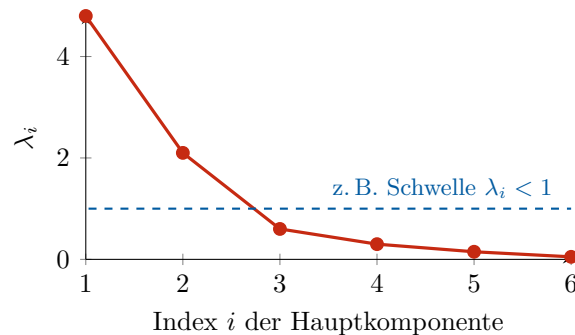


Abbildung 12: **Scree-Plot:**

Bei stark linear abhängigen Merkmalen fallen die Eigenwerte schnell ab,
die ersten 2 Komponenten genügen.

Wahl von i_0 z. B. über Schwellwert oder kumulierte erklärte Varianz (etwa 95 %).

Beispiel : PCA durchgeführt, Teil 1: Hauptachsen bestimmen (Vorlesungsbeispiel)

Merkmalsvektoren: $\vec{g}_1 = \begin{pmatrix} 3 \\ 3 \end{pmatrix}$, $\vec{g}_2 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$, $\vec{g}_3 = \begin{pmatrix} 4 \\ 2 \end{pmatrix}$, $\vec{g}_4 = \begin{pmatrix} 2 \\ 4 \end{pmatrix}$, $\vec{g}_5 = \begin{pmatrix} 5 \\ 5 \end{pmatrix}$

1. Zentrieren: $\vec{g}_{mean} = \frac{1}{5} \begin{pmatrix} 15 \\ 15 \end{pmatrix} = \begin{pmatrix} 3 \\ 3 \end{pmatrix}$

$$\Delta \vec{g}_1 = \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \Delta \vec{g}_2 = \begin{pmatrix} -2 \\ -2 \end{pmatrix}, \Delta \vec{g}_3 = \begin{pmatrix} 1 \\ -1 \end{pmatrix}, \Delta \vec{g}_4 = \begin{pmatrix} -1 \\ 1 \end{pmatrix}, \Delta \vec{g}_5 = \begin{pmatrix} 2 \\ 2 \end{pmatrix}$$

2. Kovarianzmatrix:

$$G = \begin{pmatrix} 0 & -2 & 1 & -1 & 2 \\ 0 & -2 & -1 & 1 & 2 \end{pmatrix}, \quad \mathbf{C} = \frac{1}{4} G G^T = \frac{1}{4} \begin{pmatrix} 10 & 6 \\ 6 & 10 \end{pmatrix}$$

3. Eigenwerte über $\det(\mathbf{C} - \lambda I) = 0$:

$$\left(\frac{10}{4} - \lambda\right)^2 - \left(\frac{6}{4}\right)^2 = 0 \Rightarrow \lambda^2 - 5\lambda + 4 = 0 \Rightarrow \lambda_1 = 4, \quad \lambda_2 = 1$$

Eigenvektoren: $(\mathbf{C} - 4I)\vec{e}_1 = \vec{0}$: $\frac{1}{4} \begin{pmatrix} -6 & 6 \\ 6 & -6 \end{pmatrix} \vec{e}_1 = \vec{0} \Rightarrow e_{1,1} = e_{1,2} \Rightarrow \vec{e}_1 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}$.

Analog $\vec{e}_2 = \frac{1}{\sqrt{2}} \begin{pmatrix} -1 \\ 1 \end{pmatrix}$.

$$\mathbf{U} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix}$$

Beispiel : PCA durchgeführt, Teil 2: Punkt transformieren & reduzieren

4. **Transformation** von $\vec{x} = \begin{pmatrix} 0 \\ 2 \end{pmatrix}$:

$$\vec{x}' = \mathbf{U}^T \left(\begin{pmatrix} 0 \\ 2 \end{pmatrix} - \begin{pmatrix} 3 \\ 3 \end{pmatrix} \right) = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ -1 & 1 \end{pmatrix} \begin{pmatrix} -3 \\ -1 \end{pmatrix} = \frac{1}{\sqrt{2}} \begin{pmatrix} -4 \\ 2 \end{pmatrix} = \begin{pmatrix} -2,83 \\ 1,41 \end{pmatrix}$$

5. **Reduktion** auf $i_0 = 1$: zweite Komponente auf 0 setzen: $\vec{x}'' = \frac{1}{\sqrt{2}} \begin{pmatrix} -4 \\ 0 \end{pmatrix}$.

Rekonstruktions-Test: $\vec{x}_{rek} = \vec{g}_{mean} + \mathbf{U} \vec{x}'' = \begin{pmatrix} 3 \\ 3 \end{pmatrix} + \frac{1}{2} \begin{pmatrix} -4 \\ -4 \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$,
der rekonstruierte Punkt liegt auf der ersten Hauptachse (Projektion).

Erklärte Varianz: $\frac{\lambda_1}{\lambda_1 + \lambda_2} = \frac{4}{5} = 80\%$.

4.3.2 PCA im ML-Workflow (Klassifikation)

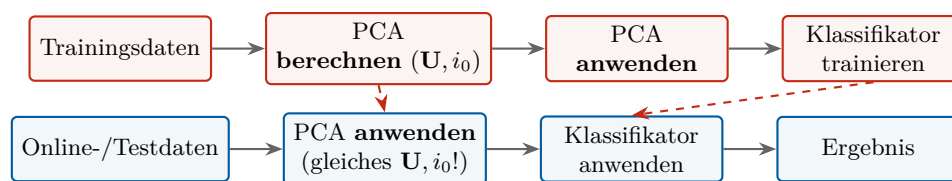


Abbildung 13: Die Transformation wird *nur auf der Trainingsmenge* berechnet und dann unverändert auf neue Daten angewendet (sonst Data Leakage).

4.3.3 Stärken, Fallstricke und Grenzen

Vorteile	Nachteile / Fallen
Dimensions- und Rauschreduktion	Skalierung: unterschiedliche Einheiten und geringe Streuung verzerren ⇒ vorher standardisieren!
Unkorrelierte Features (oft bessere Modelle)	Ausreißer verzerren Varianz und Hauptachsen; Trainingsmenge muss repräsentativ sein
Bessere Visualisierung & Mustererkennung	Komponenten schwer interpretierbar ; nur lineare Zusammenhänge (sonst Kernel-PCA)
Kompression (z. B. Eigenfaces)	Numerik: große Matrizen instabil ⇒ SVD auf G statt Eigenzerlegung von $\mathbf{C}$

Tabelle 12: PCA: Vorteile vs. Fallstricke.

4.3.4 Anwendung: Eigenfaces

Gesichtsbilder ($H \times B$ Pixel) werden zeilenweise zu Vektoren $\vec{g} \in \mathbb{R}^d$, $d = H \cdot B$ (z. B. $112 \times 92 = 10\,304$). Vorgehen: Durchschnittsgesicht abziehen → Kovarianzmatrix → Eigenvektoren = **Eigenfaces** (lassen sich wieder als Bilder visualisieren).

Bei $N = 40 < d$ Bildern gibt es höchstens 40 Eigenwerte $\neq 0$.

Neue Gesichter werden in den Eigenface-Raum projiziert ($\vec{g}' = \mathbf{U}^T(\vec{g} - \vec{g}_{mean})$) und dort klassifiziert / rekonstruiert / komprimiert.

MNIST-Beispiel: Schon wenige Hauptkomponenten rekonstruieren eine „8“ erkennbar; mehr Komponenten = schärfer.

4.4 Kernel-Trick und Kernel-PCA

Intuition: einfach erklärt

Manche Daten sind in ihrer ursprünglichen Form nicht linear trennbar, z.B. eine Klasse „innen“, die andere „außen“ auf einer Linie. Hebt man die Punkte aber in eine höhere Dimension (etwa: neue Koordinate x^2), wird aus dem Ineinander ein sauber trennbares Übereinander. Der **Kernel-Trick** macht genau das, aber *ohne die Transformation je auszurechnen*: Er ersetzt nur jedes Skalarprodukt durch eine Kernelfunktion, die das Skalarprodukt *im hochdimensionalen Raum* direkt liefert.

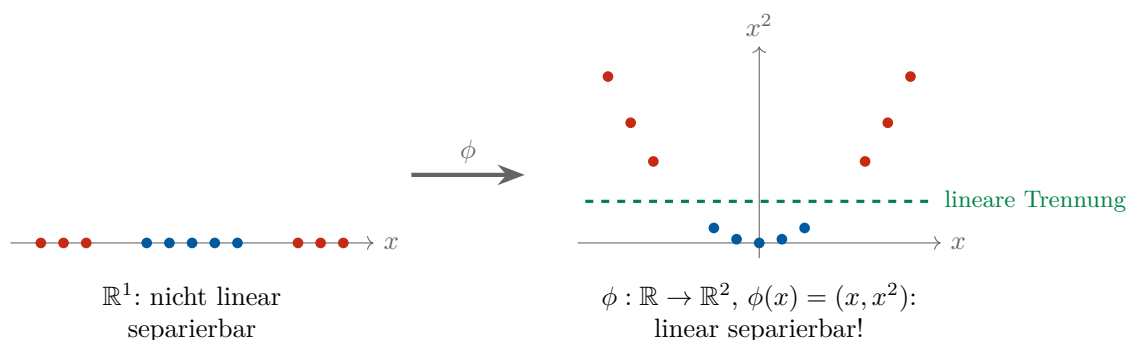


Abbildung 14: Der Kernel-Trick anschaulich:

Transformation in einen höherdimensionalen Raum macht nichtlineare Probleme linear lösbar, der polynomielle Kernel $k(x, x') = \langle x, x' \rangle^2$ leistet das, ohne ϕ explizit zu berechnen.

Merke

Eine **Kernelfunktion** nimmt zwei Vektoren, und liefert das Skalarprodukt im (implizit) transformierten Raum:

$$k(\vec{g}_i, \vec{g}_j) = \langle \phi(\vec{g}_i), \phi(\vec{g}_j) \rangle = \phi(\vec{g}_i)^T \phi(\vec{g}_j)$$

Die Abbildung ϕ wird *nie explizit* bestimmt, nur ihr Skalarprodukt. Das ist deutlich effizienter als transformieren + multiplizieren.

Kernel	Formel	Bemerkung
Polynomiell (Grad r)	$k(\vec{u}, \vec{v}) = (\vec{u}^T \vec{v} + c)^r$	Spezialfall $c = 0, r = 1$: linearer Kernel = Skalarprodukt selbst (keine Änderung)
Sigmoid / tanh	$k(\vec{u}, \vec{v}) = \tanh(\alpha \vec{u}^T \vec{v} + c)$	verwandt mit Aktivierungsfunktionen neuronaler Netze
Gauß / RBF	$k(\vec{u}, \vec{v}) = e^{-\frac{\ \vec{u} - \vec{v}\ ^2}{2\sigma^2}} = e^{-\gamma \ \vec{u} - \vec{v}\ ^2}$	$\gamma = \frac{1}{2\sigma^2}$; entspricht Dichte der Normalverteilung; impliziter Raum unendlichdimensional

Tabelle 13: Typische Kernelfunktionen. Nachteil des Tricks: Kernel und seine Hyperparameter müssen gewählt werden; zu komplexe Kernel $\Rightarrow$ Overfitting.

4.4.1 Kernel-PCA: Ablauf

Die Kovarianzmatrix ist (bis auf Normierung) eine **Skalarprodukt-/Gram-Matrix der zentrierten Daten**. Idee der Kernel-PCA: ersetze die Skalarprodukte durch Kernelauswertungen $\Rightarrow$ implizit nichtlineare Transformation.

1. **Kernelmatrix:** $K = (k(\vec{g}_i, \vec{g}_j))_{ij} \in \mathbb{R}^{N \times N}$ (Achtung: $N \times N$, nicht $d \times d$!)
2. **Zentrieren** der Kernelmatrix (Mittelwertkorrektur im Feature-Space):
 $\tilde{K} = K - \frac{1}{N} \mathbf{1}_N K - \frac{1}{N} K \mathbf{1}_N + \frac{1}{N^2} \mathbf{1}_N K \mathbf{1}_N$ ($\mathbf{1}_N$: Matrix aus Einsen)
3. **Eigenwertzerlegung:** $\tilde{K} = \mathbf{U} \Sigma \mathbf{U}^T$, Eigenvektoren normieren, nach Eigenwert sortieren
4. **Transformation** eines neuen Punkts: $\vec{x}'_{neu} = \mathbf{U}^T \vec{k}_{neu}$ mit $\vec{k}_{neu} = (k(\vec{x}_{neu}, \vec{g}_i))_i$;
 Anzahl verwendeter Eigenvektoren = Ergebnisdimension

Vorteile ggü. linearer PCA	Nachteile
Erfasst nichtlineare Strukturen	Wahl von Kernel & Parametern
Anpassbar über Kernelwahl (poly, RBF, ...)	Overfitting bei zu komplexem Kernel
Bewahrt Merkmale, die linear verloren gingen	Empfindlich gegenüber Skalierung

Tabelle 14: Kernel-PCA vs. PCA.

Klausur-Fokus

- **Komplette PCA von Hand** für 3 bis 5 Punkte in 2D: zentrieren, $\mathbf{C} = \frac{1}{N-1} G G^T$, charakteristisches Polynom, Eigenwerte (absteigend!), Eigenvektoren normieren, Punkt transformieren, reduzieren, rekonstruieren, das ist *die* Standard-Rechenaufgabe im Teil Preisach
- Erklärte Varianz $\lambda_i / \sum_j \lambda_j$ berechnen und interpretieren
- Konzeptfragen: Was sind Hauptkomponenten?
(Eigenvektoren der Kovarianzmatrix, Richtungen maximaler Varianz, unkorreliert, Linearkombinationen der Originalmerkmale)
- Fallstricke nennen: Skalierung, Ausreißer, Linearität, Repräsentativität
- Kernel-Trick erklären (Skalarprodukt ersetzen, ϕ nie explizit);
Kernelmatrix für linearen Kernel aufstellen und zentrieren
- Zusammenhang PCA $\leftrightarrow$ linearer Autoencoder (Kap. 15)

5 Entscheidungsbäume & Ensemble Learning

5.1 Klassifikation, Wiederholung

Merke

Klassifikation ordnet Beispielen $x_i \in X$ diskrete Klassen $Y = \{C_1, \dots, C_k\}$ zu. Gesucht wird ein Modell $m : X \rightarrow Y$, das eine Fehlerfunktion $f : (X \times Y) \times (X \rightarrow Y) \rightarrow \mathbb{R}$ minimiert. Das Lernen eines Modells entspricht dem Lösen eines **Optimierungsproblems**.

5.2 Entscheidungsbäume

Intuition: einfach erklärt

Ein Entscheidungsbaum ist wie ein Fragenkatalog: An jedem Knoten wird eine Frage über ein Merkmal gestellt (z.B. `petal_width < 1.75`?), und je nach Antwort läuft man den linken oder rechten Ast entlang, bis man ein Blatt erreicht, das die Klasse angibt.

Struktur:

- **Wurzel:** oberster Knoten; enthält das erste Splittkriterium.
- **Innere-/Entscheidungsknoten:** enthalten jeweils eine Bedingung über ein Attribut.
- **Blätter:** enthalten die Klassenvorhersage.
- Entscheidungsbäume können manuell von Experten erstellt, *oder* automatisch aus Trainingsdaten gelernt werden.

Vorhersage: Den gelernten Baum mit den Attributwerten des Beispiels von der Wurzel bis zum passenden Blatt durchlaufen, die dort eingetragene Klasse ist die Vorhersage.

*5.2.1 Lernalgorithmus***Merke**

Entscheidungsbäume werden **greedy-rekursiv** (lokal optimierend) gelernt:

1. Alle Trainingsbeispiele gehören zunächst zur Wurzel.
2. Wähle das beste Attribut a und den besten Schwellenwert t (*Splittkriterium*).
3. Teile die Daten in $X_{a \leq t}$ und $X_{a > t}$ auf.
4. Wiederhole rekursiv für jede Partition, bis ein Abbruchkriterium erfüllt ist.

Abbruchbedingungen: keine weiteren Splittattribute, alle Beispiele einer Klasse, maximale Baumtiefe erreicht.

5.2.2 Split-Kriterien: Gini Index

Um das beste Attribut a mit Schwelle t zu finden, werden **alle** Attribute und **alle** möglichen Schwellenwerte ausprobiert. Als Gütemass dient der **Gini Index**:

Merke

Gini Index eines Knotens mit Datenmenge X und k Klassen:

$$G(X) = 1 - \sum_{i=1}^k P(C_i)^2, \quad P(C_i) = \frac{|\{x \in X : x \in C_i\}|}{|X|}$$

Gewichteter Gini Index eines Splits in Partitionen $S_1, \dots, S_m$:

$$G_A(X) = \sum_{j=1}^m \frac{|S_j|}{|X|} \cdot G(S_j)$$

Ziel: **Minimierung** von $G_A \Rightarrow$ möglichst *reine* Kindknoten.

- $G = 0$: vollständig rein (alle Beispiele einer Klasse).
- $G = 0,5$: maximale Unreinheit bei 2 Klassen (50:50).

Beispiel Gini-Berechnung

Iris-Daten, Klassen *versicolor/virginica*, Klassenverhältnis 50:50.

Split $p_width \leq 1.4$: linker Knoten (35|1), rechter Knoten (15|49).

$$G(35, 1) = 1 - \left[\left(\frac{35}{36} \right)^2 + \left(\frac{1}{36} \right)^2 \right] = 0,054, \quad G(15, 49) = 1 - \left[\left(\frac{15}{64} \right)^2 + \left(\frac{49}{64} \right)^2 \right] = 0,358$$

$$G_A = \frac{36}{100} \cdot 0,054 + \frac{64}{100} \cdot 0,358 = \mathbf{0,248}$$

Split $p_width \leq 1.6$: linker Knoten (48|4), rechter Knoten (2|46).

$$G_A = \frac{52}{100} \cdot 0,142 + \frac{48}{100} \cdot 0,079 = \mathbf{0,112}$$

$\Rightarrow$ Split bei 1.6 ist besser (niedrigerer Gini).

Minimaler Gini Index insgesamt bei $p_width \leq 1.7$.

Weitere Splitkriterien: **Entropie** (Informationsgehalt) und **Information Gain**.

5.2.3 Vorteile & Nachteile

Vorteile	Nachteile
Einfach erklärbar, grafisch darstellbar	Oft geringere Vorhersagegenauigkeit
Auch für Laien interpretierbar	Neigen zu Overfitting bei großer Tiefe

Klausur-Fokus

Gini Index berechnen (auch für mehrere Splits vergleichen),
 Aufbau eines Entscheidungsbaums skizzieren, Abbruchkriterien nennen,
 Vorhersage für ein neues Beispiel schrittweise durchführen.

5.3 Ensemble Learning**Intuition: einfach erklärt**

Schwarmintelligenz: Galton (1906), 787 Besucher schätzten das Gewicht eines Ochsen; Durchschnitt 1.197 lbs, tatsächlich 1.198 lbs. Der Schlüssel: *Vielfalt* und *Unabhängigkeit* der Schätzungen. Fehler heben sich auf. Genau dieses Prinzip nutzt Ensemble Learning: viele *schwache Lerner* zusammen ergeben einen *starken Lerner*.

Merke

Idee: Mehrere Modelle kombinieren, um Bias und/oder Varianz zu reduzieren.

- **Varianz-minimierende Verfahren:** Voting, Bagging, Random Forest.
- **Bias-minimierende Verfahren:** Boosting (sequenzielles Training).

5.3.1 Bias-Varianz-Tradeoff

Merke

- **Varianzfehler:** Empfindlichkeit gegenüber kleinen Schwankungen in den Trainingsdaten $\Rightarrow$ Overfitting (Modell lernt Rauschen).
- **Bias-Fehler:** Fehler durch falsche Modellannahmen $\Rightarrow$ Underfitting.
- **Ziel:** Modell findet Balance, erfasst Muster in Trainingsdaten und generalisiert.

Erkennung: *Visuelle Überprüfung* (Prediction vs. Truth) oder *Kreuzvalidierung* (k-fold):
hoher Bias $\rightarrow$ durchweg schlechte Performance über alle Folds;
hohe Varianz $\rightarrow$ stark schwankende Performance.

5.3.2 Voting

Trainiere mehrere Modelle (*mit unterschiedlichen Algorithmen*) auf demselben Trainingssatz; kombiniere Vorhersagen durch Abstimmung.

Merke

- **Hard Voting:** Mehrheitsentscheidung der vorhergesagten Klassen.
- **Soft Voting:** Durchschnitt der Klassenwahrscheinlichkeiten; Klasse mit höchster Wahrscheinlichkeit gewinnt.
- **Variante:** Gewichtetes Voting (Modelle mit höherer Genauigkeit: mehr Gewicht).

Empfehlung: Alle Basismodelle sollen im Allgemeinen gute Leistung zeigen.
Nicht garantiert besser als jedes Einzelmodell.

5.3.3 Bagging (Bootstrap Aggregating)

Erfunden von Leo Breiman (1994). Reduziert **Varianz**, vermeidet Overfitting.

Merke

Bootstrap-Stichproben: Aus Trainingsmenge D mit N Beispielen wird D' erstellt, indem N Beispiele *zufällig mit Zurücklegen* gezogen werden. D' hat gleiche Größe wie D , kann aber Duplikate enthalten. Nicht gezogene Beispiele bilden den **Out-of-Bag-Datensatz** (zum Testen der schwachen Lerner).

Algorithmus:

1. Erstelle k Bootstrap-Stichproben $D_1, \dots, D_k$.
2. Trainiere für jedes D_i **parallel** einen Entscheidungsbaum.
3. Vorhersage: **Mehrheitsbeschluss** (Klassifikation) bzw. **Durchschnitt** (Regression).

Unterschied zu Voting:

Trainingsdaten sind nicht gleich, aber alle Basislerner nutzen *denselben* Algorithmus.

5.3.4 Random Forest

Erweiterung von Bagging; ebenfalls varianzreduzierend.

Merke

Unterschied zu Bagging:

An jedem Knoten wird **nur eine zufällige Teilmenge** der M Merkmale betrachtet (nicht alle). Dadurch sind die Bäume *unabhängiger* $\Rightarrow$ besserer Varianz-Bias-Tradeoff.

Algorithmus:

1. Ziehe k Bootstrap-Stichproben der Größe N .
2. Für jede Stichprobe: trainiere Entscheidungsbaum, wobei an jedem Knoten nur eine *zufällige Merkmalsteilmenge* betrachtet wird.
3. Vorhersage durch Mehrheitsentscheid oder Durchschnitt aller Bäume.

Vorteile: Sehr hohe Vorhersagekraft; robust gegenüber fehlenden Werten und Ausreißern; keine Merkmalsskalierung nötig; berechnet **Feature Importance**.

Nachteile: Hohe Komplexität; längere Trainingsdauer; weniger interpretierbar als einzelne Bäume.

5.3.5 Boosting

Erfunden von Robert Schapire (1990). Reduziert primär **Bias** (und Varianz).

Sequenziell, jeder schwache Lerner gleicht die Schwächen seines Vorgängers aus.

Merke

AdaBoost (Adaptive Boosting):

1. Weise allen Trainingsbeispielen gleiche Gewichte zu.
2. Trainiere einen schwachen Lerner F_s (z. B. *Decision Stump*, Baum mit nur einer Variablen).
3. Bewerte den Fehler err_s des Modells.
4. Berechne Konfidenzkoeffizient: $w_s = \frac{1}{2} \log\left(\frac{1 - err_s}{err_s}\right)$ (niedrige Fehlerrate $\Rightarrow$ hohes Gewicht).
5. Aktualisiere Datengewichte: falsch klassifizierte Beispiele werden stärker gewichtet.
6. Wiederhole ab Schritt 2 bis alle Punkte korrekt klassifiziert oder maximale Iterationen s erreicht.
7. Vorhersage: gewichtete Mehrheitsentscheidung $F_s(x) := \sum_{i=1}^s w_i f_i(x)$.

Merke**XGBoost (eXtreme Gradient Boosting):**

- Für Klassifikation *und* Regression geeignet.
- Gewichtet nicht einzelne Datenpunkte (wie AdaBoost), sondern lernt aus den **Gradienten der Verlustfunktion** (minimiert Residuen durch Gradientenabstieg).
- Nutzt **Regularisierung** (beschränkt Baumtiefe $\Rightarrow$ verhindert Overfitting).
- Verwendet tiefe Bäume statt Decision Stumps.
- Sehr effizient und skalierbar, aber speicherintensiv; viele Hyperparameter.
- **Sehr hohe Vorhersagegenauigkeit** für tabellarische Daten (Gewinner vieler ML-Challenges).

Boosting, Vor- und Nachteile:

- + sehr gute Vorhersageleistung, reduziert Bias;
- empfindlich gegenüber Ausreißern, hoher Rechenaufwand.

5.3.6 Vergleich der Ensemble-Verfahren

Method	Verwendete Daten	Basislerner	Ablauf	Fehlerreduktion
Voting	Alle	Verschiedene Alg.	parallel	Varianz
Bagging	Zufallsauswahl (Boot.)	Gleicher Alg.	parallel	Varianz
Boosting	Gewichtete Daten	Gleicher Alg.	sequenziell	Bias (& Varianz)

Klausur-Fokus

Unterschied Bagging vs. Voting vs. Boosting erklären; Bias vs. Varianz zuordnen; Random Forest als Erweiterung von Bagging beschreiben (Merkmalsteilmenge pro Knoten); AdaBoost-Schritte skizzieren; w_s -Formel; XGBoost-Schlüsselmerkmale nennen; paralleles vs. sequenzielles Training unterscheiden.

6 Zeitreihen & Forecasting**6.1 Zeitreihen und ihre Eigenschaften****Merke**

Eine **Zeitreihe** ist eine Folge von Beobachtungen, die in zeitlicher Reihenfolge aufgezeichnet werden (z. B. Wetter, Stromverbrauch, Aktienkurse, Sensordaten, EKG).

Intuition: einfach erklärt

Bei klassischen ML-Daten nimmt man die Beispiele $(\mathbf{x}_i, y_i)$ als **unabhängig und identisch verteilt (IID)** an. Bei Zeitreihen gilt das *nicht*: Der aktuelle Wert hängt von vergangenen ab ($x_t \rightarrow x_{t+1}$). Konsequenzen: Die **Reihenfolge ist wichtig**, zufälliges Mischen (Shuffling) ist meist unzulässig, und vergangene Information muss explizit als Eingabe bereitgestellt werden. Zeit erzeugt Abhängigkeiten zwischen Beobachtungen.

6.1.1 Komponenten einer Zeitreihe

Merke

$$x_t = \text{Trend} + \text{Saison} + \text{Residuum}$$

- **Trend:** langfristige Entwicklung (steigend, fallend, konstant).
- **Saisonalität:** wiederkehrendes Muster mit *festem* Zeitraum (z. B. jährlich, monatlich).
- **Periodizität:** wiederkehrendes Muster *ohne* festen Zeitraum.
- **Residuum** (Rauschen): nicht erklärbare, zufällige Schwankungen.

Additive Saisonalität: gleichbleibende Amplitude (unabhängig vom Trend).

Multiplikative Saisonalität: Amplitude wächst mit dem Trend.

6.1.2 Autokorrelation

Merke

Autokorrelation: Zusammenhang einer Zeitreihe mit sich selbst (Wert heute vs. Wert in der Vergangenheit, um einen *Lag* verschoben). Berechnet und geplottet mit der **Autocorrelation Function (ACF)** über verschiedene Lags.

- positiv: hohe Werte folgen auf hohe Werte.
- negativ: auf hohe Werte folgen niedrige.
- 0: kein Zusammenhang.

Eine Zeitreihe *ohne* Autokorrelation ist ein reiner Zufallsprozess (**weißes Rauschen**).

6.1.3 Stationarität

Merke

Eine Zeitreihe ist **stationär**, wenn sie *keinen* Trend und *keine* Saisonalität hat und **Mittelwert und Varianz konstant** bleiben. Viele klassische Modelle (z. B. ARIMA) setzen Stationarität voraus.

Differenzierung macht eine Reihe stationär, indem der Trend entfernt wird: man bildet die Differenz aufeinanderfolgender Werte

$$x'_t = x_t - x_{t-1}.$$

Nach der Differenzierung ist oft keine signifikante Autokorrelation mehr vorhanden.

6.2 Datenaufbereitung

- **Datenbereinigung:** fehlende Werte, Ausreißer und ungültige Punkte behandeln (Interpolation, Glättung, Entfernen).
- **Skalierung:** Normalisierung/Standardisierung, um unterschiedliche Einheiten anzugleichen.
- **Glätten:** kurzfristiges Rauschen reduzieren, z.B. mit *gleitendem Durchschnitt* oder exponentiell gewichtetem Durchschnitt.
- **Aggregation:** auf gröbere zeitliche Auflösung zusammenfassen (Stunden $\rightarrow$ Tage $\rightarrow$ Monate).
- **Feature-Engineering:** zusätzliche Merkmale erzeugen (Lage- und Streuungsmaße, Trendindikatoren, saisonale Komponenten).

6.3 Forecasting als überwachtes Lernen

Merke

Wichtige Begriffe:

- **Univariat:** nur eine Messgröße über die Zeit.
Multivariat: mehrere Variablen gleichzeitig, inklusive Wechselwirkungen.
- **Prognosehorizont:** *One-Step Ahead* (ein Schritt) vs. *Multi-Step Ahead* (mehrere Schritte).
- **Lag-Variablen:** wie viele vergangene Werte als Eingabe dienen.
Zusätzlich möglich: **exogene** (externe) Variablen.

Merke

Sliding Window: Um eine Zeitreihe für überwachtes Lernen nutzbar zu machen, schiebt man ein Fenster der Breite w über die Reihe. Jede Beobachtung bekommt so w Eingaben (die letzten w Werte, die Lags), die Zielvariable ist der Wert h Schritte später (x_{t+h}). Allgemein: w Lag-Inputs sagen x_{t+h} vorher.

Beispiel Lag-Trainingsdatensatz bauen

Sonnenstunden pro Monat. Mit **Lag** = 2 und **Prognosehorizont** = 1 ergibt sich für die Monate 10, 11, 12 (Werte: ..., 185, 246, 100, 42, 33):

Monat	Feature 1 (x_{t-2})	Feature 2 (x_{t-1})	Ziel (x_t)
10	185	246	100
11	246	100	42
12	100	42	33

Jede Zeile ist ein klassisches (Features, Label)-Trainingsbeispiel.
Verallgemeinert: w Lag-Inputs, Zielhorizont $t+h$.

6.4 Naive Vorhersagemethoden

Merke

- **Durchschnittsprognose:** alle zukünftigen Werte = Mittelwert aller bisherigen Beobachtungen.
- **Naive Prognose:** alle zukünftigen Werte = letzter beobachteter Wert.
- **Naive saisonale Prognose:** jeder Wert = letzter Wert derselben Saison (z. B. derselbe Monat im Vorjahr).

Beispiel Naive Prognosen

Sonnenstunden 2022 und 2023, Vorhersage für Januar bis Juni 2024:

- **Durchschnittsprognose:** $\approx 157,87$ (Mittelwert aller 24 Monate), für jeden Monat gleich.
- **Naive Prognose:** 33 (letzter Wert, Dezember 2023), für jeden Monat gleich.
- **Naive saisonale Prognose:** die Werte desselben Monats aus 2023, also Januar bis Juni = 35, 90, 100, 150, 244, 305.

6.5 Methoden im Überblick

Merke

Drei Hauptfamilien:

1. **Klassische Zeitreihenanalyse** (univariat): Trend, Saison und Autokorrelation modellieren (Zerlegung, exponentielle Glättung, ARIMA).
2. **Multivariate Analyse / Regression:** nutzt auch andere Merkmale, klassische Regressionsalgorithmen.
3. **Deep Learning:** neuronale Netze (v. a. RNN/LSTM), univariat und multivariat.

6.5.1 Klassische Analyse

- **Zerlegung:** Zeitreihe als Überlagerung von Trend, Saison und Fehler betrachten.
- **Exponentielle Glättung:** gewichteter Durchschnitt vergangener Werte mit *exponentiell abnehmenden* Gewichten (neuere Werte zählen mehr). Glättungsfaktor $0 \leq \alpha \leq 1$: α nahe 1 gewichtet die nahe Vergangenheit stärker, α nahe 0 die ferne. Erweiterte Versionen erfassen auch Trend und Saison.

Merke

ARIMA(p, d, q) (AutoRegressive Integrated Moving Average), geeignet für *univariate* Vorhersage. Idee: erst die Reihe stationär machen, dann Autokorrelation und Fehlerstruktur modellieren.

- **AR**(p) (Auto-Regressive): p vergangene Werte erklären den aktuellen, das „Gedächtnis der Reihe“.
- **I**(d) (Integrated): d -fache Differenzierung stellt Stationarität her.
- **MA**(q) (Moving Average): q vergangene Prognosefehler korrigieren kurzfristige Schocks.

6.5.2 Regression

Regressionsmodelle nutzen **verzögerte Prädiktoren** (Lag-Variablen) und optional exogene Variablen, um y zum Zeitpunkt $t+h$ vorherzusagen. Einsetzbar sind beliebige Regressionsalgorithmen (lineare Regression, entscheidungsbaumbasiert, XGBoost, k-NN).

6.5.3 Deep Learning: RNN und LSTM

Merke

Rekurrente Neuronale Netze (RNN) erlauben *Rückkopplungen*: Eine rekurrente Zelle verarbeitet in jedem Schritt die aktuelle Eingabe $x^{(t)}$ und den Zustand h_{t-1} der vorherigen Zelle, mit *denselben* Parametern (das Netz wird „aufgefaltet“). Dadurch eignen sie sich für **sequentielle Daten** beliebiger Länge.

Gewichte: U (Eingabe zu verborgen), W (verborgen zu verborgen), V (verborgen zu Ausgabe). Training mit **Backpropagation Through Time**.

Merke

Problem langer Reihen: *vanishing gradients* (Gradient < 1 : der Anfang der Reihe wird vergessen) oder *exploding gradients* (Gradient > 1 : Instabilität). Normale RNN haben also **kein Langzeitgedächtnis**. **LSTM** (Long Short-Term Memory) löst das mit

Speicherzellen und drei **Gates**:

- **Forget Gate:** vergisst unwichtige Information.
- **Input Gate:** fügt neue, wichtige Information hinzu.
- **Output Gate:** filtert, welche Information ausgegeben wird.

Zusätzlicher Zustand C_t als **Langzeitgedächtnis**, h_t als Kurzzeitgedächtnis.

Varianten: **GRU** (vereinfachtes LSTM), **Transformer** (z. B. TimeGPT).

6.6 Evaluation von Zeitreihenmodellen

Merke

Klassische Cross-Validation ist problematisch: zufälliges Mischen verletzt die Zeitstruktur, das Modell sähe „Zukunftsdaten“ bereits im Training (**Data Leakage**) und wirkte unrealistisch gut. Modelle dürfen nur Information nutzen, die zum Vorhersagezeitpunkt verfügbar ist.

Merke

Zeitbasierte Aufteilung (Training = Vergangenheit, Test = Zukunft):

- **Hold-Out (zeitbasiert):** ein fester Trennzeitpunkt, davor Training, danach Test.
- **Sliding Window:** ein festes Fenster wandert über die Reihe, ergibt mehrere Train/Test-Sets.
- **Expanding Window:** wie Sliding Window, aber der Trainingsteil wächst stetig.

Gemessen wird mit den üblichen Regressionsmaßen, z. B. dem **RMSE**:

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{t=1}^n (y_t - \hat{y}_t)^2}.$$

Klausur-Fokus

Zeitreihen-Komponenten (Trend/Saison/Residuum) sowie additive vs. multiplikative Saisonalität erklären; Stationarität definieren und Differenzierung anwenden; aus einer Zeitreihe einen **Lag-Trainingsdatensatz** bauen (Sliding Window); naive Methoden (Durchschnitt, naiv, naiv saisonal) berechnen; die ARIMA(p, d, q)-Komponenten erklären; RNN/LSTM beschreiben (Gates, Lang- und Kurzzeitgedächtnis, vanishing gradients); erklären, warum klassische Cross-Validation bei Zeitreihen versagt (Data Leakage), und die korrekten zeitbasierten Splits nennen.

Teil II

Überwachtes Lernen & Neuronale Netze

(Vorlesung Prof. Janka)

7 Überwachtes Lernen: Das Framework

7.1 Lernen als Funktionssuche

Intuition: einfach erklärt

Überwachtes Lernen ist im Kern immer dasselbe Spiel: Es gibt Paare aus *Eingabe* und *gewünschter Ausgabe* (E-Mail → Spam/kein Spam, Verkaufshistorie → nächster Monat). Gesucht ist eine *mathematische Funktion*, die diesen Zusammenhang möglichst gut beschreibt und zwar nicht nur auf den gesehenen Beispielen, sondern auch auf neuen. Alles Weitere (lineare Regression, logistische Regression, neuronale Netze) sind nur verschiedene *Funktionsfamilien* für dieses eine Spiel.

Merke**Universelles Vorgehen im Supervised Learning**

(wurde so 1:1 in der Klausur SS23 gefragt):

1. Ausgangspunkt: Feature-Label-Paare $(\mathbf{x}_i, y_i)$, $i = 1, \dots, N$
2. **Split** in Trainings- und Testdaten (z. B. 80/20)
3. **Training** auf den Trainingsdaten: Funktion f suchen mit $f(\mathbf{x}_i) \approx y_i$, d. h. **Minimum einer Loss-Funktion** finden
4. **Evaluation** auf den (ungesehenen) Testdaten mit einer geeigneten Metrik

7.2 Die Loss-Funktion

Eine **Loss-Funktion für einen Punkt** $\ell(y, f(\mathbf{x}))$ quantifiziert die Kosten, wenn für Eingabe $\mathbf{x}$ die Vorhersage $f(\mathbf{x})$ gemacht wird und der wahre Wert y ist.

Auf den Trainingsdaten wird gemittelt:

$$L(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N \ell(y_i, f(\mathbf{x}_i; \mathbf{w}))$$

wobei $f(\mathbf{x}; \mathbf{w})$ die Abhängigkeit der Vorhersage von den **Parametern** $\mathbf{w}$ explizit macht.

$L: \mathbb{R}^p \rightarrow \mathbb{R}$ ist eine multivariate Funktion des Koeffizientenvektors, die Trainingsdaten sind fest!

Loss	Formel (ein Punkt)	Einsatz
Quadratischer Fehler (MSE)	$(y - f(\mathbf{x}))^2$	lineare Regression, Autoencoder
Binäre Kreuzentropie (Log-Loss)	$-(y \log f(\mathbf{x}) + (1-y) \log(1-f(\mathbf{x})))$	logistische Regression, binäre Klassifikation
Kreuzentropie (K Klassen)	$-\sum_{k=1}^K y_k \log f(\mathbf{x})_k$	Softmax-Regression, NN-Klassifikation

Tabelle 15: Die drei Standard-Loss-Funktionen.

Alle Trainings-Losses sind Mittelwerte der Einzelpunkt-Losses.

7.3 Das eigentliche Ziel: Generalisierungsfehler

Merke

Eigentlich wollen wir nicht den Fehler auf den Trainingsdaten minimieren, sondern den **erwarteten Verlust über alle möglichen Punkte** (Generalisierungsfehler, Risiko):

$$\min_{\mathbf{w}} \mathbb{E}[\ell(y, f(\mathbf{x}; \mathbf{w}))]$$

Diesen Erwartungswert kann man *nicht berechnen* (die wahre Verteilung aller Paare $(\mathbf{x}, y)$ ist unbekannt). Man approximiert ihn durch den Mittelwert der Stichprobe, die Trainingsdaten:

$$L(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N \ell(y_i, f(\mathbf{x}_i; \mathbf{w})) \approx \mathbb{E}[\ell(y, f(\mathbf{x}; \mathbf{w}))]$$

Das Minimieren von $L(\mathbf{w})$ heißt **empirische Risikominimierung**. Der Testdatensatz simuliert „alle unbekannten Punkte“, um Overfitting auf die Trainingsdaten zu erkennen.

Annahme dahinter: die $(\mathbf{x}_i, y_i)$ sind eine Stichprobe aus einer gemeinsamen Wahrscheinlichkeitsverteilung auf $X \times Y$.

Zwei zentrale Fragen strukturieren die folgenden Kapitel:

1. Wie sehen gute **Modelle** (Funktionen f) aus? $\rightarrow$ lineare Modelle (Kap. 8, 10), neuronale Netze (Kap. 12)
2. Wie **trainiert** man sie? (a) Minimum von $L(\mathbf{w})$ finden $\rightarrow$ Optimierung (Kap. 9), SGD/Backpropagation (Kap. 13); (b) sicherstellen, dass $L(\mathbf{w})$ den Generalisierungsfehler gut approximiert $\rightarrow$ Generalisierung, Regularisierung (Kap. 11)

8 Lineare Regression

8.1 Einfache lineare Regression

Intuition: einfach erklärt

Das einfachste denkbare Modell: eine *Gerade durch die Punktwolke* legen. Beispiel Advertising-Datensatz (200 Märkte): Hilft mehr TV-Werbebudget (x) dem Umsatz (y)? Es gibt unendlich viele Geraden, welche ist die beste? Dafür braucht man ein *Maß für die Güte* einer Geraden: den mittleren quadratischen Fehler. Die „beste“ Gerade ist die, die ihn minimiert.

Modellannahme: $y = f(x) + \epsilon$ mit unbekannter Funktion f und zufälligem Fehlerterm ϵ (Erwartungswert 0). Ansatz: f ist **linear**:

$$f(x) = w_0 + w_1x$$

mit **Achsenabschnitt** (Intercept) w_0 und **Steigung** w_1 , den *Modellparametern* (Gewichten).

8.1.1 Residuen und MSE

Das **Residuum** des i -ten Datenpunkts ist die Differenz zwischen echtem Wert und Vorhersage:

$$r_i = y_i - f(x_i) = y_i - (w_0 + w_1x_i)$$

($r_i > 0$: Modell unterschätzt; $r_i < 0$: Modell überschätzt.)

Merke

Mean Squared Error und Loss-Funktion:

$$L(w_0, w_1) = MSE = \frac{1}{N} \sum_{i=1}^N r_i^2 = \frac{1}{N} \sum_{i=1}^N (y_i - (w_0 + w_1x_i))^2$$

Modelltraining = Minimieren der Loss-Funktion: $(w_0, w_1) = \arg \min_{w_0, w_1} L(w_0, w_1)$

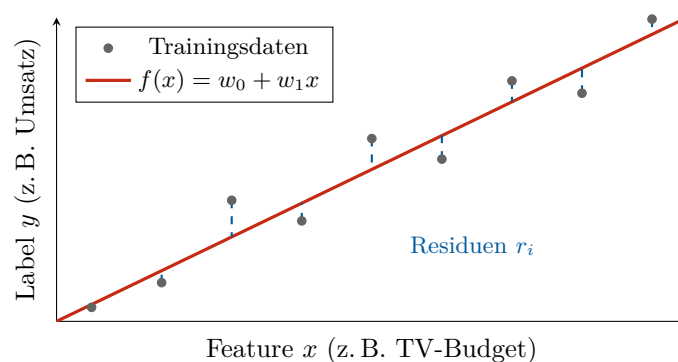


Abbildung 15: Lineare Regression:

Die Loss-Funktion summiert die quadrierten Residuen (gestrichelt). Quadrieren macht negative Abweichungen positiv und bestraft große Fehler überproportional.

Beispiel : MSE berechnen (Vorlesungsbeispiel)

Daten $(x, y) \in \{(0, 3), (1, 6), (2, 4), (3, 11)\}$, Modell $f(x) = 3 + 2x$ (also $w_0 = 3, w_1 = 2$).
Vorhersagen: 3, 5, 7, 9. Residuen: $(3-3), (6-5), (4-7), (11-9) = 0, 1, -3, 2$.

$$MSE = \frac{1}{4} (0^2 + 1^2 + (-3)^2 + 2^2) = \frac{1}{4} (0 + 1 + 9 + 4) = 3,5$$

(Vorlesungsvariante mit anderem Modell: $MSE = \frac{1}{4}(4 + 4 + 9 + 1) = 4,5$, das Prinzip ist identisch: vorhersagen, Differenz, quadrieren, mitteln.)

8.1.2 Train-Test-Split und Gütemaße

Die Modellgüte darf **nicht auf den Trainingsdaten** bewertet werden.

Typisch: 80 % Training / 20 % Test; Testdaten bleiben im Training unsichtbar.

Maß	Formel	Interpretation
RMSE	$\sqrt{MSE}$	gleiche Einheit wie y : „Modell liegt typischerweise um RMSE daneben“
MAE	$\frac{1}{N} \sum_i y_i - f(\mathbf{x}_i) $	robuster gegenüber Ausreißern (keine Quadrierung)
R^2	$1 - \frac{RSS}{TSS}$ mit $RSS = \sum_i (y_i - f(\mathbf{x}_i))^2$ $TSS = \sum_i (y_i - \bar{y})^2$	Anteil der erklärten Varianz , einheitenunabhängig. $R^2 = 1$: perfekt; $R^2 = 0$: nicht besser als der Mittelwert

Tabelle 16: Gütemaße für Regression. Advertising-Beispiel: $R^2 \approx 0,8$, $RMSE \approx 2,2$ M€.

8.2 Multiple lineare Regression

Mit p Features lautet die Modellfunktion

$$f(x_1, \dots, x_p) = w_0 + w_1 x_1 + \dots + w_p x_p = w_0 + \mathbf{w}^T \mathbf{x}$$

Geometrie: bei $p = 2$ eine **Ebene** im $\mathbb{R}^3$, allgemein eine Hyperebene. Interpretation von w_j :
Veränderung der Vorhersage pro Einheit von Feature j *bei gleichbleibenden übrigen Features*
(Advertising: $w_{TV} = 0,054$, $w_{Radio} = 0,102 \Rightarrow$ pro T€ Radio-Budget +102 T€ Umsatz).

8.2.1 Matrixschreibweise

Alle Trainingsdaten in der **Feature-Matrix** $\mathbf{X} \in \mathbb{R}^{N \times (p+1)}$ (Zeile = Datenpunkt, Spalte = Feature). **Intercept-Trick**: eine zusätzliche Spalte aus Einsen vorn anfügen, dann steckt w_0 mit im Vektor $\mathbf{w} = (w_0, w_1, \dots, w_p)^T$:

$$\mathbf{X}\mathbf{w} = \begin{pmatrix} 1 & x_{1,1} & \dots & x_{1,p} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_{N,1} & \dots & x_{N,p} \end{pmatrix} \begin{pmatrix} w_0 \\ w_1 \\ \vdots \\ w_p \end{pmatrix} = \begin{pmatrix} w_0 + w_1 x_{1,1} + \dots + w_p x_{1,p} \\ \vdots \\ w_0 + w_1 x_{N,1} + \dots + w_p x_{N,p} \end{pmatrix}$$

Residuenvektor $\mathbf{r} = \mathbf{y} - \mathbf{X}\mathbf{w} \in \mathbb{R}^N$, damit:

Merke**Lineares Regressionsproblem in Matrixform:**

$$\min_{\mathbf{w}} L(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N r_i^2 = \frac{1}{N} \mathbf{r}^T \mathbf{r} = \frac{1}{N} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2$$

8.3 Nichtlineare Features

Intuition: einfach erklärt

Eine Gerade passt nicht immer (Auto-Datensatz: PS vs. Verbrauch ist gekrümmt).
 Der Trick: Man macht aus dem einen Feature hp einfach *zwei* Features
 $x_1 = hp$ und $x_2 = hp^2$ und schon kann das *lineare* Modell eine *Parabel* lernen.
 Das Modell bleibt linear **in den Parametern $\mathbf{w}$** , nur die Features wurden transformiert.
 Das ist dieselbe Idee wie der Kernel-Trick (Kap. 4.4), nur explizit.

Polynomielle Regression: $mpg = w_0 + w_1 \cdot hp + w_2 \cdot hp^2 + \epsilon$, mit Features (hp, hp^2)
 kompatibel mit dem linearen Framework. Auto-Datensatz: R^2 steigt von 0,605 (linear) auf
 0,688 (quadratisch); Polynom 5. Grades bringt fast nichts mehr (0,697) und wirkt
 „verwackelt“, **Overfitting-Gefahr**.

Allgemein: Feature-Engineering als Transformation $\phi: \mathbb{R}^m \rightarrow \mathbb{R}^p$ (Polynome, sin, exp,
 Domänenwissen); Modell $f(\mathbf{x}) = \mathbf{w}^T \phi(\mathbf{x})$ bleibt linear in $\mathbf{w}$.

8.4 Bias-Variance-Tradeoff

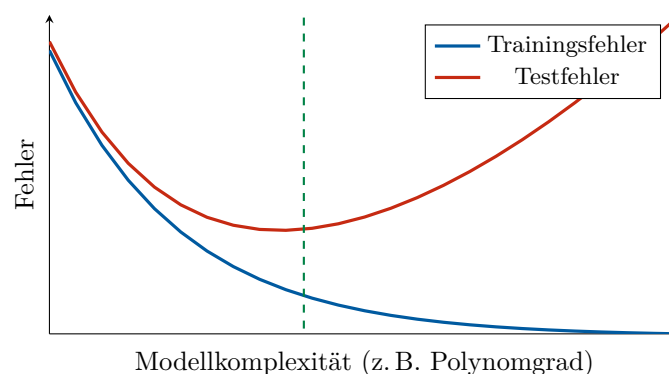


Abbildung 16: Der Trainingsfehler sinkt mit der Komplexität *immer*; der Testfehler sinkt erst
 und steigt dann wieder. Beispiel Polynominterpolation: Grad 0 bis 1 zu einfach,
 Grad 2 bis 3 gut, Grad 9 bis 10 perfekt auf Training aber katastrophal auf Test.

	Underfitting (hoher Bias)	Overfitting (hohe Variance)
Ursache	Modell zu einfach / falsche Annahmen (lineares Modell für nichtlinearen Zusammenhang)	Modell zu komplex, „lernt Trainingsdaten auswendig“
Symptom	Fehler auf Training <i>und</i> Test groß	Trainingsfehler klein, Testfehler groß
Begriff	Bias = Verzerrung durch zu starke Modellannahmen	Variance = Variabilität des gelernten Modells bei anderen Trainingsdaten

Tabelle 17: Bias-Variance-Tradeoff: niedrige Komplexität $\Rightarrow$ hoher Bias, niedrige Varianz;
 hohe Komplexität $\Rightarrow$ niedriger Bias, hohe Varianz. Ziel ist immer **Generalisierung**.

Klausur-Fokus

Standardaufgabe in *jeder* Altklausur (Aufgabe 1):

- **Vorhersage berechnen:** Koeffizienten gegeben, Werte einsetzen ($\hat{y} = w_0 + w_1x_1 + w_2x_2$); kategorische Features als 0/1 kodieren („Hochschulabschluss: ja“ $\rightarrow x_2 = 1$)
- **Least-Squares-Verlustfunktion aufstellen** für 3 gegebene Datenpunkte: $L(w_0, w_1, w_2) = \frac{1}{3} \sum_i (y_i - (w_0 + w_1x_{i1} + w_2x_{i2}))^2$, aufpassen: Faktor $\frac{1}{N}$, alle Klammern, Kodierung kategorischer Features, ggf. „ w_0 weglassen“ beachten (SS23: Features waren Umsätze der Vorjahre!)
- **Optimalität prüfen:** Parameter sind optimal, wenn $\nabla L(\mathbf{w}) = \mathbf{0}$ (partielle Ableitungen verschwinden)
- Bias-Variance-Tradeoff erklären (WS25/26: mit Bezug zur Modellkomplexität)

9 Optimierung

9.1 Grundbegriffe

Modelltraining ist *immer* ein Optimierungsproblem in Standardform:

$$\min_{\mathbf{x} \in \mathbb{R}^n} g(\mathbf{x})$$

Im ML: Optimierungsvariablen = Parameter $\mathbf{w}$, Zielfunktion = Loss-Funktion L .

Annahme: g stetig differenzierbar, keine Nebenbedingungen.

- **Globales Minimum $\mathbf{x}^*$:** $g(\mathbf{x}) \geq g(\mathbf{x}^*)$ für *alle* $\mathbf{x}$
- **Lokales Minimum $\mathbf{x}^*$:** $g(\mathbf{x}) \geq g(\mathbf{x}^*)$ für alle $\mathbf{x}$ in einer *Umgebung* von $\mathbf{x}^*$
- Jedes globale Minimum ist lokal, nicht umgekehrt!
Unbeschränkte Probleme (z. B. $g(x) = x$) haben gar kein Minimum.

9.2 Optimalitätsbedingungen

Merke

Univariat (Kurvendiskussion): $g'(x_0) = 0$ und $g''(x_0) > 0 \Rightarrow$ lokales Minimum ($g'' < 0$: Maximum). Punkte mit $g'(x_0) = 0$ heißen **stationäre Punkte**.

Multivariat: Der **Gradient** ersetzt die erste Ableitung:

$$\nabla g(\mathbf{x}_0) = \left(\frac{\partial g}{\partial x_1}(\mathbf{x}_0), \dots, \frac{\partial g}{\partial x_n}(\mathbf{x}_0) \right)^T \in \mathbb{R}^n$$

Notwendige Bedingung für ein Minimum: $\nabla g(\mathbf{x}_0) = \mathbf{0}$ (Gleichungssystem in n Variablen). Die **Hessematrix** $\nabla^2 g$ (zweite partielle Ableitungen) übernimmt die Rolle von g'' : positiv definit $\Rightarrow$ Minimum, negativ definit $\Rightarrow$ Maximum.

9.3 Analytische Lösung der linearen Regression: Normalengleichung

Für $\min_{\mathbf{w}} L(\mathbf{w}) = \frac{1}{N} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2$ liefert Nullsetzen des Gradienten die **Normalengleichung**:

$$\mathbf{X}^T \mathbf{X} \mathbf{w} = \mathbf{X}^T \mathbf{y} \quad \Longrightarrow \quad \mathbf{w} = \left(\mathbf{X}^T \mathbf{X} \right)^{-1} \mathbf{X}^T \mathbf{y} \quad (\text{falls } \mathbf{X}^T \mathbf{X} \text{ invertierbar})$$

- Lineare Regression ist eines der **wenigen** Modelle mit analytischer Lösung, exakt, in einem Schritt, keine Iteration.
- Nachteil: Matrixinversion kostet $O(p^3)$, bei großen N oder p nutzt man doch Gradientenverfahren.
- Für **neuronale Netze** gibt es **keine analytische Lösung**, dort ist Gradientenabstieg unumgänglich.

9.4 Gradientenabstieg

Intuition: einfach erklärt

Man steht im Nebel auf einem Berg und will ins Tal: Man fühlt mit dem Fuß, in welcher Richtung es am steilsten bergab geht, macht einen kleinen Schritt dorthin und wiederholt das. Der *Gradient* zeigt bergauf, also geht man in Richtung $-\nabla g$. Die *Schrittlänge* ist die Lernrate α : zu kleine Schritte dauern ewig, zu große springen über das Tal hinaus.

Merke

Gradientenabstieg (multivariat):

Gegeben differenzierbares $g : \mathbb{R}^n \rightarrow \mathbb{R}$, Startwert $\mathbf{x}^{[0]}$, Lernrate $\alpha^{[k]} > 0$. Für $k = 0, 1, 2, \dots$:

$$\mathbf{x}^{[k+1]} = \mathbf{x}^{[k]} - \alpha^{[k]} \nabla g(\mathbf{x}^{[k]})$$

Abbruch bei erfüllter Abbruchbedingung (max. Iterationen, wenig Fortschritt, ...).

Geometrie des Gradienten: $\nabla g(\mathbf{x})$ zeigt in Richtung des *steilsten Anstiegs*, $-\nabla g(\mathbf{x})$ des steilsten Abstiegs, $\|\nabla g\|$ gibt die Stärke an. Im Minimum: $\nabla g(\mathbf{x}^*) = \mathbf{0}$.

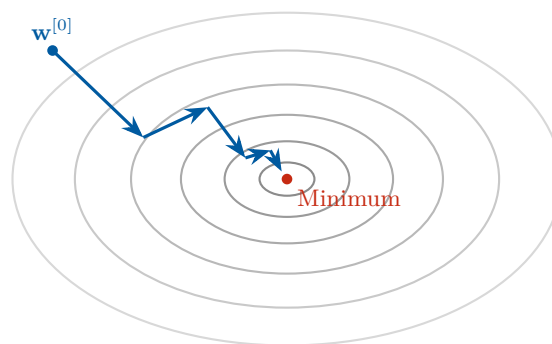


Abbildung 17: Iterierte Punkte im Konturplot der Loss-Funktion:

Jeder Punkt ist ein Parametervektor $\mathbf{w}^{[k]}$;

die Schritte verlaufen senkrecht zu den Konturlinien (Richtung $-\nabla L$) von außen ins Minimum.

9.4.1 Die Lernrate α

Lernrate	Verhalten
zu klein	sehr langsame Konvergenz (viele Iterationen)
gut gewählt	gleichmäßige, zügige Konvergenz
größer	schneller, aber unruhiger (Zickzack)
zu groß	springt über das Minimum hinaus: Oszillation oder Divergenz (Loss steigt!)

Tabelle 18: Einfluss der Lernrate, Klausur-Klassiker (WS23/24 A3e, WS25/26 A4c):
zu groß $\Rightarrow$ Divergenz/Oszillation; zu klein $\Rightarrow$ stabil aber sehr langsam.

9.4.2 Was kann schiefgehen?

- **Divergenz:** α zu groß
- **Langsame Konvergenz:** α zu klein oder schlecht konditioniertes Problem (stark unterschiedliche Krümmung in verschiedene Richtungen, auch ein Grund für Feature-Skalierung!)
- **Lokales Minimum:** Verfahren konvergiert, aber nicht zum globalen Optimum
- **Sattelpunkt:** $\nabla g = \mathbf{0}$, aber kein Minimum

9.4.3 Gradientenabstieg für die lineare Regression

Für $L(\mathbf{w}) = \frac{1}{N} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2$ ist der Gradient $\nabla L(\mathbf{w}) = -\frac{2}{N} (\mathbf{y} - \mathbf{X}\mathbf{w})^T \mathbf{X}$, die Iteration also

$$\mathbf{w}^{[k+1]} = \mathbf{w}^{[k]} + \frac{2\alpha^{[k]}}{N} (\mathbf{y} - \mathbf{X}\mathbf{w}^{[k]})^T \mathbf{X}$$

Anschaulich: jede Iteration = eine Gerade im Streudiagramm, die sich schrittweise in die Punktwolke „dreht“.

Klausur-Fokus

- Optimalitätsbedingung nennen:
Parameter optimal $\Leftrightarrow \nabla L(\mathbf{w}) = \mathbf{0}$ (so gefragt in SS22 A1c)
- Update-Formel des Gradientenabstiegs hinschreiben (mit Vorzeichen und Lernrate!)
- Verhalten bei zu großer/kleiner Lernrate beschreiben
- Normalengleichung kennen + wissen, warum NN keine analytische Lösung haben

10 Klassifizierung und logistische Regression

10.1 Klassifizierung vs. Regression

Regression: numerische Zielvariable.

Klassifizierung: *kategorielle* Zielvariable, Eingabe $\mathbf{x} \in \mathbb{R}^p$,

Ausgabe eine von K diskreten Klassen $C_1, \dots, C_K$.

Beispiele: Spamfilter, Bilderkennung, Betrugserkennung, Spracherkennung.

Im binären Fall ($K = 2$) wird das Label als 0/1 kodiert.

10.2 Das logistische Regressionsmodell

Intuition: einfach erklärt

Warum nicht einfach lineare Regression auf das 0/1-Label? Weil die Gerade Werte wie $-0,4$ oder $1,7$ vorhersagt, als Wahrscheinlichkeit unbrauchbar. Die Lösung: Die lineare Funktion wird durch die **Sigmoid-Funktion** in den Bereich $(0, 1)$ „gestaucht“. Das Ergebnis lässt sich als *Wahrscheinlichkeit für Klasse 1* lesen.

Merke**Sigmoid-Funktion und Modellfunktion:**

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad f(\mathbf{x}) = \sigma(w_0 + w_1 x_1 + \dots + w_p x_p) = \frac{1}{1 + e^{-(w_0 + \mathbf{w}^T \mathbf{x})}}$$

$\lim_{z \rightarrow -\infty} \sigma(z) = 0$, $\lim_{z \rightarrow +\infty} \sigma(z) = 1$, $\sigma(0) = 0,5$. Die Vorhersage $f(\mathbf{x}) \in (0, 1)$ wird als Wahrscheinlichkeit für Klasse 1 interpretiert. Nützliche Ableitung: $\sigma'(z) = \sigma(z)(1 - \sigma(z))$.

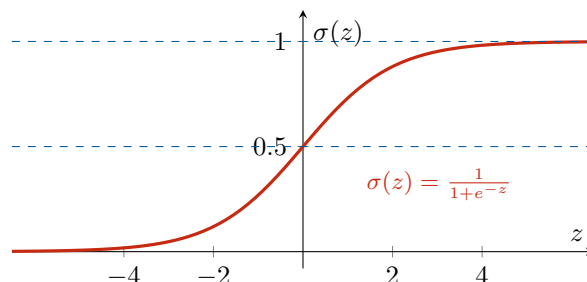


Abbildung 18: Die Sigmoid-Funktion: S-förmig, streng monoton, differenzierbar, beschränkt auf $(0, 1)$, deshalb als Wahrscheinlichkeit interpretierbar und für Gradientenverfahren geeignet (Klausurfrage WS25/26 1c).

10.2.1 Vorhersage und Entscheidungsgrenze

Für eine konkrete Klassenvorhersage legt man einen **Schwellwert** B fest (Default 0,5):

$$\text{Klasse}(\mathbf{x}) = \begin{cases} 1, & f(\mathbf{x}) \geq B \\ 0, & f(\mathbf{x}) < B \end{cases}$$

Die Punkte mit $f(\mathbf{x}_B) = B$ bilden die **Entscheidungsgrenze**; sie teilt den Feature Raum in zwei Regionen. Für $B = 0,5$ gilt: $f(\mathbf{x}) = 0,5 \Leftrightarrow w_0 + \mathbf{w}^T \mathbf{x} = 0$, eine *lineare* Gleichung!

Feature Raum	Entscheidungsgrenze
1D ($p = 1$)	ein Punkt
2D ($p = 2$)	eine Gerade
3D ($p = 3$)	eine Ebene
p D	Hyperebene

Tabelle 19: **Klausur-Klassiker** (SS23 1c, WS23/24 1c):

Warum heißt logistische Regression „lineares Modell“, obwohl σ nichtlinear ist?

$\Rightarrow$ Weil die Entscheidungsgrenze linear ist.

10.2.2 Training: Log-Loss (binäre Kreuzentropie)**Warum nicht MSE?**

Bei $y \in \{0, 1\}$, $\hat{y} \in (0, 1)$ ist der maximale quadratische Fehler $(1 - 0)^2 = 1$, zu schwacher Anreiz, grobe Fehler zu korrigieren.

Besser: ein Loss, der bei schlechten Vorhersagen **beliebig groß** wird:

Merke**Log-Loss / binäre Kreuzentropie** (ein Punkt bzw. Datensatz):

$$\ell(\mathbf{w}) = -(y \log \hat{y} + (1 - y) \log(1 - \hat{y})) \quad L(\mathbf{w}) = -\frac{1}{N} \sum_{i=1}^N (y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i))$$

mit $\hat{y}_i = \sigma(\mathbf{w}^T \mathbf{x}_i)$. Bei $y = 1$: Loss = $-\log \hat{y}$ (groß, wenn $\hat{y}$ klein); bei $y = 0$: Loss = $-\log(1 - \hat{y})$. **Nicht analytisch lösbar** $\Rightarrow$ Training per Gradientenabstieg.

Beispiel : Vorhersage und Loss (Klausur WS25/26, Aufgabe 1)

Unfallvorhersage mit $w_0 = 0$, $w_1 = 4$ (Niederschlag in mm), $w_2 = -1$ (Tag = 1), $w_3 = -7$ (Schienen = 1). Beobachtung: Niederschlag 2 mm, Tag, Schienen.

$$\hat{y} = \frac{1}{1 + e^{-(0+4 \cdot 2 - 1 \cdot 1 - 7 \cdot 1)}} = \frac{1}{1 + e^0} = 0,5$$

Log-Loss für diesen Punkt mit Label $y = 0$ (kein Unfall):

$$\ell = -\log(1 - 0,5) = -\log 0,5 \approx 0,69$$

Beispiel : Schwellwert-Rückrechnung (Klausur WS22/23, Aufgabe 1c)

Fahrradverleih, $w_0 = -15$, $w_1 = 1$ (Temperatur), $w_2 = -0,1$ (Regen).

Bei welcher Temperatur (kein Regen) ist die Wahrscheinlichkeit genau 50 %?

$$\sigma(-15 + x_1) = 0,5 \Leftrightarrow -15 + x_1 = 0 \Leftrightarrow x_1 = 15 \checkmark C$$

Merksatz: $\sigma(z) = 0,5 \Leftrightarrow z = 0$, man setzt einfach das lineare Argument gleich Null.

10.3 Evaluation: Konfusionsmatrix und Metriken

Anders als bei Regression evaluiert man Klassifikation nicht über die Kostenfunktion, sondern über die **Klassenzugehörigkeit**:

	Echtes Label 1	Echtes Label 0
Vorhersage 1	True Positive (TP)	False Positive (FP)
Vorhersage 0	False Negative (FN)	True Negative (TN)

Abbildung 19: Konfusionsmatrix für binäre Klassifikation; $N = N_{TP} + N_{TN} + N_{FP} + N_{FN}$. Bei K Klassen: $K \times K$ -Matrix, Accuracy = Diagonalsumme/ N ; Precision/Recall klassenweise.

Merke

$$\text{Accuracy} = \frac{N_{TP} + N_{TN}}{N} \quad \text{Precision} = \frac{N_{TP}}{N_{TP} + N_{FP}} \quad \text{Recall (TPR)} = \frac{N_{TP}}{N_{TP} + N_{FN}}$$

$$\text{False Positive Rate} = \frac{N_{FP}}{N_{FP} + N_{TN}}$$

Eselsbrücken:

Precision = „Wie viele meiner *positiven Vorhersagen* stimmen?“ (sensibel für FP).

Recall = „Wie viele der *tatsächlich Positiven* finde ich?“ (sensibel für FN).

Szenario	Wichtige Metrik und Begründung
Medizinischer Test (krank/gesund)	Recall maximieren: verpasste Kranke (FN) sind gefährlich
Spam-Erkennung	Precision : wichtige E-Mails fälschlich als Spam (FP) teuer
Chemieanlage, 99,9 % unkritisch	Triviale Klassifikator „nie kritisch“ hat Accuracy $\approx 99,9\%$ $\Rightarrow$ Accuracy allein ungeeignet bei unausgewogenen Klassen

Tabelle 20: Metrik-Wahl ist anwendungsabhängig: Wie teuer sind FP vs. FN?

Hinweis: Precision hängt von der Klassenverteilung ab, Recall nicht.

Der Schwellwert B kann *nach* dem Training angepasst werden, B erhöhen steigert die Precision (weniger, aber sicherere positive Vorhersagen).

10.4 Multi-Klassen-Klassifizierung: Softmax

Für K Klassen wird $f: \mathbb{R}^p \rightarrow \mathbb{R}^K$; das Label wird **one-hot** kodiert $((0, \dots, 1, \dots, 0)^T)$.

K lineare Modelle berechnen je einen Score: $\mathbf{z} = \mathbf{W}^T \mathbf{x} \in \mathbb{R}^K$.

Die **Softmax-Funktion** macht daraus eine Wahrscheinlichkeitsverteilung:

$$\text{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}$$

Eigenschaften: alle Werte in $(0, 1)$, Summe = 1, Reihenfolge der Scores bleibt erhalten.

Vorhersage: $\hat{y} = \arg \max_k \hat{y}_k$.

Training: Kreuzentropie $L(\mathbf{W}) = -\frac{1}{N} \sum_i \sum_k (y_i)_k \log(\hat{y}_i)_k$, minimiert per Gradientenabstieg.

10.5 Nichtlineare Entscheidungsgrenzen und Regularisierung

Logistische Regression kann nur *lineare* Grenzen lernen. Abhilfe wie bei der Regression:

Feature-Engineering. Beispiel kreisförmige Grenze: neues Feature $x_3 = x_1^2 + x_2^2$ (quadratischer Abstand zum Ursprung), im erweiterten Raum trennt eine *Ebene*, zurückprojiziert in den $\mathbb{R}^2$ ergibt das einen *Kreis*. Das Modell bleibt linear in $\phi(\mathbf{x})$. Risiko: Overfitting bei vielen Termen $\Rightarrow$ Regularisierung mit ℓ_2 - oder ℓ_1 -Strafterm auf den Log-Loss (genau wie bei linearer Regression, Kap. 11):

$$L(\mathbf{w}) = -\frac{1}{N} \sum_{i=1}^N (y_i \log \hat{y}_i + (1-y_i) \log(1-\hat{y}_i)) + \lambda \sum_{j=1}^p w_j^2$$

	Lineare Regression	Logistische Regression
Aufgabe	Regression	Klassifizierung
Ausgabe	reelle Zahl	Wahrscheinlichkeit
Modellfunktion	$\mathbf{w}^T \mathbf{x}$	$\sigma(\mathbf{w}^T \mathbf{x})$
Verlustfunktion	MSE	Log-Loss
Training	analytisch / GD	nur Gradientenabstieg
Entscheidungsgrenze	keine	linear

Tabelle 21: Die beiden linearen Modelle im direkten Vergleich, gleicher Ansatz, andere Modell- und Verlustfunktion.

Klausur-Fokus

- **Wahrscheinlichkeit ausrechnen:** Koeffizienten + Featurewerte in $\sigma(w_0 + \mathbf{w}^T \mathbf{x})$ einsetzen (jede Klausur!); Aussage „näher an 0 oder 1?“ über Vorzeichen des Exponenten
- **Log-Loss aufstellen/berechnen** für gegebene Punkte (Probeklausur A1b, WS25/26 1b)
- **50%-Frage:** lineares Argument = 0 setzen und auflösen
- Konfusionsmatrix auswerten: Accuracy, Precision, Recall als Brüche (WS25/26 A2)
- Begründen, welche Metrik wann zählt (Medizin → Recall, Spam → Precision)
- „Wie Precision verbessern?“ → Schwellwert erhöhen (WS22/23 1d)
- Softmax: Formel, Eigenschaften, Einsatz (Ausgabeschicht bei K Klassen)

11 Generalisierung, Overfitting und Regularisierung

11.1 Drei Fehlerbegriffe

Merke

Trainingsfehler: Fehler auf den Trainingsdaten. **Testfehler:** Fehler auf den Testdaten.
Generalisierungsfehler: Fehler auf *unbekannten* Daten, das eigentliche Ziel ist immer ein Modell, das diesen minimiert!

11.2 Overfitting auf den Trainingsdaten

Das Training minimiert den Trainingsfehler. Ein zu flexibles Modell beschreibt die Trainingsdaten perfekt, generalisiert aber schlecht, **Overfitting:** Trainings- und Testfehler liegen weit auseinander. Wichtig: Das passiert *auch bei linearen Modellen* mit vielen Features (Beispiel Kaltmiete Karlsruhe, One-Hot-kodiertes Feature **facilities** ⇒ viele Spalten: R^2 auf Training hoch, auf Test deutlich niedriger).

Drei Gegenmittel:

1. Mehr bzw. diversere Trainingsdaten beschaffen
2. Modellkomplexität reduzieren (weniger Features / nichtlineare Terme)
3. **Regularisierung**

11.3 Overfitting auf den Testdaten (die subtilere Falle)

Intuition: einfach erklärt

Der Algorithmus optimiert den Trainingsfehler → Overfitting auf die Trainingsdaten. Aber: *Der Data Scientist optimiert (implizit) den Testfehler!* Bei jeder Entscheidung, welche Features, welche Vorverarbeitung, welche Hyperparameter, schaut man auf den Testfehler und wählt danach aus. Nach 50 solchen Entscheidungen ist der Testfehler kein neutraler Schiedsrichter mehr, sondern Teil des Trainings: Das Modell overfittet *auf die Testdaten*, und der Testfehler ist ein **zu optimistischer** Schätzer für den Generalisierungsfehler.

Lösung: Kreuzvalidierung und/oder Testdatensatz erst *ganz am Ende* ein einziges Mal verwenden; für Hyperparametersuche ein separates **Validierungsset**.

11.4 Kreuzvalidierung

Merke

k-fache Kreuzvalidierung:

1. Datensatz in *k* gleich große *Folds* aufteilen
2. Für $i = 1, \dots, k$: Fold *i* als Testdaten, übrige *k*−1 Folds als Trainingsdaten → trainieren, MSE_i berechnen
3. Schätzung des Generalisierungsfehlers: $MSE_{CV} = \frac{1}{k} \sum_{i=1}^k MSE_i$

Typisch $k = 5$ oder $k = 10$.

Vorteil: jeder Punkt ist genau einmal Testpunkt → stabilere Schätzung, geringerer Einfluss der zufälligen Aufteilung.

Nachteil: *k*-mal so teuer wie ein einzelnes Training.

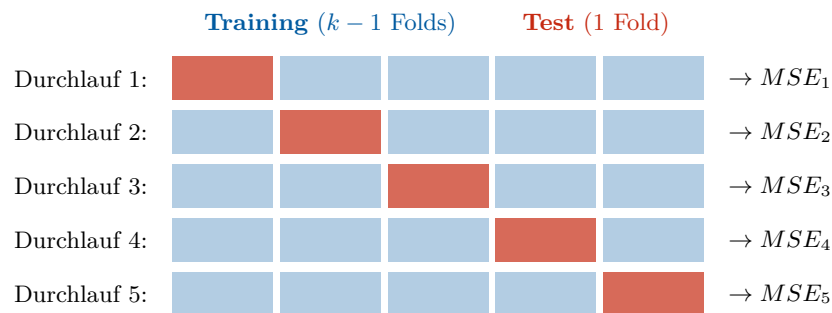


Abbildung 20: 5-fache Kreuzvalidierung: 5 Trainingsläufe, jeder Fold genau einmal Testdaten; Ergebnis ist der Mittelwert der 5 Fehler.

11.4.1 Hyperparametersuche (Gittersuche)

1. Kandidatenmenge festlegen (z. B. 10 Werte für $\lambda \times \{\text{Ridge, Lasso}\}$)
2. Für jeden Kandidaten: Modell trainieren, Güte auf **Validierungsdaten** (oder per CV) messen
3. Finales Modell mit bestem Hyperparameter auf dem *gesamten* Trainingsdatensatz trainieren
4. Erst ganz am Ende: Endgültige Modellleistung einmalig auf den **Testdaten** messen (unverzerrte Schätzung)

Beispiel : Modelle zählen (Klausur-Dauerbrenner !)

10 000 Datenpunkte, 40 % Testset $\Rightarrow$ 6 000 Trainingspunkte.

Gittersuche: 10 Werte für $\lambda \times 2$ Verfahren (Lasso/Ridge), 3-fache Kreuzvalidierung.

Wie viele Modelle mit wie vielen Punkten ?

- Pro Kandidat 3 CV-Läufe: $10 \cdot 2 \cdot 3 = 60$ Modelle.
Trainingsmenge je Lauf: $\frac{2}{3} \cdot 6\,000 = 4\,000$ Punkte.
- Danach **1 finales Modell** mit dem besten Hyperparameter auf allen 6 000 Trainingspunkten.

Rezept:

(Anzahl λ -Werte) $\times$ (Anzahl Verfahren) $\times$ (k) Modelle;

auf je $\frac{k-1}{k}$ der Trainingsdaten + 1 finales Modell auf allen Trainingsdaten.

Bei Validierungsset statt CV: 1 Lauf pro Kandidat auf (Trainingsdaten – Validierungsset).

11.5 Regularisierung: Ridge und Lasso**Intuition: einfach erklärt**

Große Koeffizienten bedeuten: Winzige Änderungen im Input kippen die Vorhersage stark, das Modell ist nervös (hohe Varianz) und overfittet. Regularisierung legt dem Modell eine „Leine“ an: Die Verlustfunktion bekommt einen *Strafterm* für große Koeffizienten. Das Modell muss nun abwägen: Daten gut fitten *und* Koeffizienten klein halten.

Es wird absichtlich etwas unflexibler (mehr Bias), dafür stabiler (weniger Varianz) und auf Testdaten besser.

Merke

$$\text{Ridge } (\ell_2): \min_{\mathbf{w}} \frac{1}{N} \sum_{i=1}^N (y_i - \mathbf{w}^T \mathbf{x}_i)^2 + \lambda \sum_{j=1}^p w_j^2$$

$$\text{Lasso } (\ell_1): \min_{\mathbf{w}} \frac{1}{N} \sum_{i=1}^N (y_i - \mathbf{w}^T \mathbf{x}_i)^2 + \lambda \sum_{j=1}^p |w_j|$$

$\lambda = 0$: normale lineare Regression; $\lambda \rightarrow \infty$: alle Koeffizienten $\rightarrow 0$.

Der Hyperparameter $\lambda > 0$ steuert die Stärke

(Wahl per Kreuzvalidierung; sklearn: `RidgeCV`, `LassoCV`).

	Ridge (ℓ_2)	Lasso (ℓ_1)
Strafterm	$\lambda \sum_j w_j^2$	$\lambda \sum_j w_j $
Koeffizienten	geschrumpft, aber $\neq 0$	viele exakt = 0 (sparsity)
Featureauswahl	nein	ja, automatisch
Differenzierbarkeit	überall	nicht in $w_j = 0$
Algorithmen	effizienter	etwas aufwändiger

Tabelle 22: Ridge vs. Lasso (Klausurfrage WS25/26 3b). Lasso ist nützlich, wenn viele Features vorhanden, aber nur wenige relevant sind, man sieht direkt, welche. In der Praxis werden beide verwendet. Imoscout-Beispiel (61 Features, 383 Punkte): R_{Test}^2 steigt von 0,719 auf 0,759 (Ridge, $\lambda=10$), Trainingsfehler steigt, Testfehler sinkt!

Typische Situationen, die Regularisierung erfordern (SS23 2b): mehr Features als Datenpunkte; polynomielle Features; One-Hot-Encoding mit vielen Ausprägungen.

Nachteile (SS23 2b iii): Modellgüte auf den Trainingsdaten wird schlechter; Parameter verlieren ihre Interpretierbarkeit.

Wahl von λ = Bias-Varianz-Tradeoff: λ zu klein $\rightarrow$ Overfitting möglich;
 λ zu groß $\rightarrow$ Underfitting.

11.5.1 Vor der Regularisierung: Features skalieren!

Bei unskalierten Features (Balkon $\in \{0, 1\}$
 vs. Wohnfläche $\in [30, 300]$ vs. Baujahr $\in [1600, 2021]$):

- **Regularisierung:** Beträge der Koeffizienten spiegeln nicht die Wichtigkeit wider (verschiedene „Einheiten“), aber alle werden mit demselben λ bestraft
 $\Rightarrow$ betragsgroße Koeffizienten werden bevorzugt reduziert, unfair.
- **Gradientenabstieg:** das großskalige Feature dominiert den Gradienten, das Verfahren springt hin und her, konvergiert langsam.

$\Rightarrow$ Standardisierung oder Min-Max-Normalisierung (Formeln in Kap. 1.7).

Wichtig: $\bar{x}$ und σ (bzw. min / max) nur auf den *Trainingsdaten* bestimmen und dann auf Training *und* Test anwenden.

11.6 Data Leakage

Intuition: einfach erklärt

Data Leakage = das Modell schummelt: Es benutzt im Training Informationen, die es zum Vorhersagezeitpunkt gar nicht haben kann. Die Performance wird dadurch *überschätzt*, im Labor glänzt das Modell, in der Realität versagt es.

Leakage-Typ	Beispiel und Abhilfe
Schlechter Split	Dieselben Patienten in Training <i>und</i> Test (mehrere Bilder pro Patient) $\Rightarrow$ nach Patienten splitten, nicht nach Bildern
Stellvertreter-Label	MonthlySalaryGBP als Feature für Label AnnualIncomeUSD, lineare Funktion des Labels $\Rightarrow$ Label-Duplikate entfernen
Zukunftsinformation	LatePaymentReminders bei Kreditvergabe, zum Entscheidungszeitpunkt noch unbekannt, wird erst später befüllt $\Rightarrow$ Spalte löschen
Vorverarbeitung	Skalierung / NaN-Imputation / One-Hot-Encoding <i>vor</i> dem Split auf dem Gesamtdatensatz berechnet $\Rightarrow$ immer nur auf Trainingsdaten fitten

Tabelle 23: Die vier Data-Leakage-Typen aus der Vorlesung.

Klausur-Fokus

- „Was ist das Ziel von Regularisierung und wie wird es erreicht?“, kam in **allen fünf** Altklausuren! Musterantwort: *Ziel ist es, Overfitting (zu starkes Anpassen an die Trainingsdaten) zu vermeiden; erreicht wird das, indem beim Training die Modellflexibilität eingeschränkt wird.*
- **Modelle-Zählen** bei Gittersuche + CV (4 von 5 Klausuren)
- Ridge vs. Lasso: Normen, Wirkung auf Koeffizienten, Sparsity / Featureauswahl
- Warum Validierungsset statt Testdaten für Hyperparameter?
→ kein Overfitting auf die Testdaten
- Kreuzvalidierung: Ablauf, Vorteil (stabilere Schätzung) und Nachteil (Rechenzeit)
- Rollen von Training/Validierung/Test sauber erklären (WS25/26 3c)

12 Neuronale Netze: Grundlagen

12.1 Motivation: Warum reichen lineare Modelle nicht?

Lineare und logistische Regression sind **lineare Modelle**:

Vorhersage bzw. Entscheidungsgrenze beruhen auf $\mathbf{w}^T \mathbf{x}$ bzw. $\sigma(\mathbf{w}^T \mathbf{x})$.

Händisches Feature-Engineering (x^2 , $x_1 x_2$, $\log x$, ...) kann sie nichtlinear erweitern, aber:

- Man muss die Art der Nichtlinearität *vorher* kennen
- Naheliegende Nichtlinearitäten (Polynome) generalisieren oft schlecht
- Bei Bildern (1280×1024 Pixel $\Rightarrow \mathbf{x} \in \mathbb{R}^{1310720}$, stark korrelierte Einzelpixel ohne Aussagekraft) führen handgebaute Features nicht zum Ziel

Gesucht:

generische nichtlineare Modellfunktionen,

die abgeleitete Features *automatisch lernen*, differenzierbar sind (für Gradiententraining)

und aus Bausteinen aufgebaut werden können $\Rightarrow$ **neuronale Netze**.

12.2 Das künstliche Neuron

Intuition: einfach erklärt

Biologisches Vorbild: Eine Nervenzelle sammelt Eingangssignale, gewichtet sie und „feuert“, wenn die Summe eine Schwelle überschreitet. Das künstliche Neuron macht exakt das: gewichtete Summe der Eingänge plus Bias, dann eine Aktivierungsfunktion. *Lernen* heißt: die Gewichte und die Schwelle anpassen. Ein einzelnes Neuron mit Sigmoid ist übrigens genau eine logistische Regression, neuronale Netze sind also „viele logistische Regressionen, geschickt gestapelt“.

Perzeptron (mit Stufenfunktion, nicht differenzierbar):

$z = \text{threshold}(\mathbf{w}^T \mathbf{x} + b) \in \{0, 1\}$. Stattdessen mit differenzierbarer Aktivierung:

$$a = \sigma(\mathbf{w}^T \mathbf{x} + b)$$

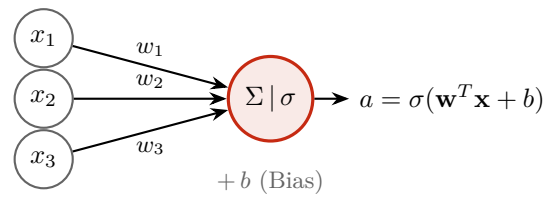


Abbildung 21: Künstliches Neuron: gewichtete Summe + Bias, dann Aktivierungsfunktion.
 Lineare Regression = ein Neuron ohne Aktivierung;
 logistische Regression = ein Neuron mit Sigmoid.

12.3 Aktivierungsfunktionen

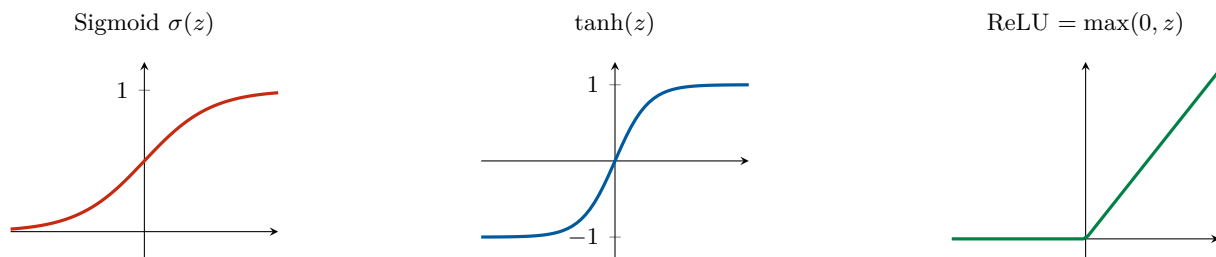


Abbildung 22: Die drei wichtigsten Aktivierungsfunktionen. **ReLU** ist heute am verbreitetsten. Notation allgemein: $g(\cdot)$; auf Vektoren wird g *elementweise* angewendet.

Merke

Die Aktivierungsfunktion **muss nichtlinear** sein, sonst kollabiert das ganze Netz wieder zu einem linearen Modell (Verkettung linearer Funktionen ist linear).

12.4 Netze mit einer verdeckten Schicht

Mehrere Neuronen werden in einer **Schicht** angeordnet;
 der Output ist die gewichtete Summe der Aktivierungen.

Aufbau:

Eingabeschicht (Input Layer)

→ **verdeckte Schicht** (Hidden Layer)

→ **Ausgabeschicht** (Output Layer).

Notation: $w_{i,j}^{[k]}$ = Gewicht in Schicht k vom j -ten Neuron der Vorschicht zum i -ten Neuron.

Merke

Matrixschreibweise (Regression, eine verdeckte Schicht):

$$\hat{y} = \mathbf{W}^{[2]} \sigma(\mathbf{W}^{[1]} \mathbf{x} + \mathbf{b}^{[1]}) + b^{[2]}$$

Dimensionen: $\mathbf{W}^{[1]} \in \mathbb{R}^{m \times p}$ (m Neuronen, p Features), $\mathbf{b}^{[1]} \in \mathbb{R}^m$, $\mathbf{W}^{[2]} \in \mathbb{R}^{1 \times m}$, $b^{[2]} \in \mathbb{R}$.
 Zeilen von $\mathbf{W}^{[k]}$ = Neuronen der Schicht, Spalten = Eingänge.

Beispiel : Forward-Pass von Hand (Vorlesungsbeispiel)

Netz mit 2 Inputs, 3 verdeckten Neuronen (Sigmoid), 1 Output (ohne Aktivierung).

$$\text{Gewichte: } \mathbf{W}^{[1]} = \begin{pmatrix} 1 & -1 \\ 2 & 4 \\ 3 & -2 \end{pmatrix}, \mathbf{b}^{[1]} = \begin{pmatrix} 1 \\ 2 \\ 0 \end{pmatrix}, \mathbf{W}^{[2]} = (1, -2, -3), b^{[2]}$$

= 5. Eingabe $(x_1, x_2) = (1, 2)$.

Verdeckte Schicht:

$$a_1 = \sigma(1 \cdot 1 - 1 \cdot 2 + 1) = \sigma(0) = 0,5$$

$$a_2 = \sigma(2 \cdot 1 + 4 \cdot 2 + 2) = \sigma(12) \approx 0,99$$

$$a_3 = \sigma(3 \cdot 1 - 2 \cdot 2 + 0) = \sigma(-1) \approx 0,27$$

$$\text{Output: } \hat{y} = 1 \cdot 0,5 - 2 \cdot 0,99 - 3 \cdot 0,27 + 5 = 1,312$$

12.4.1 Ausgabeschichten je nach Aufgabe

Aufgabe	Ausgabeschicht	Gesamtformel (1 verdeckte Schicht)
Regression	1 Neuron, <i>keine</i> Aktivierung	$\hat{y} = \mathbf{W}^{[2]} \sigma(\mathbf{W}^{[1]} \mathbf{x} + \mathbf{b}^{[1]}) + b^{[2]}$
Binäre Klassifikation	1 Neuron + Sigmoid (Wahrscheinlichkeit)	$\hat{y} = \sigma(\mathbf{W}^{[2]} \sigma(\mathbf{W}^{[1]} \mathbf{x} + \mathbf{b}^{[1]}) + b^{[2]})$
K-Klassen	K Neuronen + Softmax	$\hat{\mathbf{y}} = \text{softmax}(\mathbf{W}^{[2]} \sigma(\mathbf{W}^{[1]} \mathbf{x} + \mathbf{b}^{[1]}) + \mathbf{b}^{[2]})$

Tabelle 24: Anpassung der Ausgabeschicht, Klausurfrage in SS22 (A3c) und WS23/24 (A3c):
Für K Klassen braucht die Ausgabeschicht K Neuronen und Softmax als Aktivierung.

12.5 Tiefe Netze (Deep Learning)

Beliebig viele Schichten hintereinander;

die Ausgaben der k -ten Schicht sind die Eingaben der $(k+1)$ -ten:

$$\mathbf{a}^{[0]} = \mathbf{x}; \quad \mathbf{a}^{[k]} = g(\mathbf{W}^{[k]} \mathbf{a}^{[k-1]} + \mathbf{b}^{[k]}), \quad k = 1, \dots, m; \quad \hat{\mathbf{y}} = f(\mathbf{W}^{[m+1]} \mathbf{a}^{[m]} + \mathbf{b}^{[m+1]})$$

mit Ausgabefunktion f (= Identität, g oder Softmax).

Deep Learning = Netze mit *mindestens zwei* verdeckten Schichten.

Beispiel : Parameter zählen (MNIST)

MNIST: 28×28 -Graubilder $\Rightarrow \mathbf{x} \in \mathbb{R}^{784}$, 10 Klassen (one-hot).

Architektur: $784 \rightarrow 100 \rightarrow 50 \rightarrow 10$ (Softmax).

$$\underbrace{100 \cdot 784 + 100}_{\mathbf{W}^{[1]}, \mathbf{b}^{[1]}} + \underbrace{50 \cdot 100 + 50}_{\mathbf{W}^{[2]}, \mathbf{b}^{[2]}} + \underbrace{10 \cdot 50 + 10}_{\mathbf{W}^{[3]}, \mathbf{b}^{[3]}} = 84\,060 \text{ Parameter}$$

Regel: Parameter pro Schicht = (Neuronen $\times$ Eingänge) + Neuronen (Bias).

Bemerkenswert: mehr Parameter als Trainingsbeispiele (60 000),
bei linearen Modellen sinnlos, bei NN funktioniert es.

Merke

Universal Approximation Theorem (Cybenko 1988): Schon ein Netz mit *einer* verdeckten Schicht (und genügend Neuronen) kann jede stetige Funktion beliebig genau approximieren, also jedes Klassifikations-/ Regressionsproblem mit stetiger Modellfunktion darstellen. (Sagt nichts darüber, wie man es *trainiert* oder wie viele Neuronen nötig sind!)

Training: wie immer, alle $\mathbf{W}^{[k]}, \mathbf{b}^{[k]}$ in einem großen Vektor $\mathbf{w}$ anordnen und $\min_{\mathbf{w}} L(\mathbf{w}) = \frac{1}{N} \sum_i \ell(y_i, f(\mathbf{x}_i, \mathbf{w}))$ lösen $\rightarrow$ Kap. 13.
Mehr Neuronen/Schichten = flexibler, aber Overfitting-Gefahr.

Klausur-Fokus

Forward-Pass von Hand ist DIE Standardaufgabe (alle 5 Klausuren, Aufgabe 3/4):

- Gegeben Netzdiagramm mit Gewichten an den Kanten, meist **ReLU** in der verdeckten Schicht, keine/Sigmoid-Aktivierung am Output, Bias beachten (mal 0, mal gegeben!)
- Gefragt:
 - (a) Funktionsformel für beliebiges $\mathbf{x}$ hinschreiben, z. B. $-2 \max(0, x_1) + \max(0, -x_1 + 3x_2)$;
 - (b) Wert für konkretes $\mathbf{x}$ ausrechnen;
 - (c) Koeffizientenmatrizen $\mathbf{W}^{[1]}, \mathbf{W}^{[2]}$ ablesen (Zeile = Ziel-Neuron!)
- ReLU sauber auswerten: $\max(0, -3) = 0$!
- Erweiterung auf K Klassen: K Output-Neuronen + Softmax

13 Training von Neuronalen Netzen

13.1 Ausgangslage

Gesucht sind die Matrizen $\mathbf{W}^{[i]}$ und Vektoren $\mathbf{b}^{[i]}$ (gedacht als ein großer Vektor $\mathbf{w}$), die die Verlustfunktion auf dem Trainingsset minimieren:

$$\min_{\mathbf{w}} L(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N \ell(\mathbf{y}_i, f(\mathbf{x}_i; \mathbf{w}))$$

newline Es gibt *keine* analytische Lösung $\Rightarrow$ iterative Verfahren.

Anschauung: Die Loss-Landschaft ist ein hochdimensionales „Gebirge“; jeder Ort ist ein Gewichtsvektor $\mathbf{w}$, die Höhe der Loss.

13.2 Vom Gradientenabstieg zum SGD

Intuition: einfach erklärt

Batch-GD berechnet vor jedem Schritt den Fehler über *alle* N Trainingsbeispiele, präzise, aber bei Millionen Beispielen quälend langsam. **SGD** sagt: ein einzelnes (zufälliges) Beispiel reicht als grobe Schätzung der Richtung, die Schritte zittern, aber im Mittel geht es bergab, und man macht tausendmal mehr Schritte pro Zeit. **Minibatch** ist der Kompromiss: kleine Päckchen (z. B. 32 Beispiele) mitteln das Zittern weg und nutzen die Hardware effizient. Bonus: Das Zittern wirkt wie eingebaute Regularisierung.

	Batch-GD	SGD	Minibatch-SGD
Suchrichtung aus	allen N Beispielen	1 zufälligen Beispiel	m Beispielen ($1 < m < N$)
Update-Formel	$\mathbf{w} - \alpha \nabla_{\mathbf{w}} L(\mathbf{w})$	$\mathbf{w} - \alpha \nabla_{\mathbf{w}} \ell(\mathbf{y}_i, f(\mathbf{x}_i; \mathbf{w}))$	$\mathbf{w} - \alpha \nabla_{\mathbf{w}} \frac{1}{m} \sum_{j \in B} \ell_j$
Verbesserung	in jeder Iteration	nur langfristig / im Erwartungswert (Rauschen)	dazwischen
Kosten/Schritt	hoch ($\propto N$)	minimal	, unabhängig von N
Updates pro Epoche	1	N	N/m

Tabelle 25: Die drei Gradientenverfahren. Die Gradienten haben in allen Fällen dieselbe **Dimension** p (= Anzahl der Parameter $\mathbf{w}$)! Minibatch-SGD ist der Standard; Batchgröße m ist ein Hyperparameter (oft Zweierpotenzen). Der Datensatz muss nie komplett in den Hauptspeicher.

Praktische Umsetzung (Minibatch-SGD):

1. Trainingsbeispiele randomisieren (Zeilen mischen)
2. Start mit Schätzung $\mathbf{w}^{[0]}$
3. Äußere Schleife über $k = 0, 1, 2, \dots$ (**Epochen**); pro Epoche Lernrate $\alpha^{[k]}$ wählen
4. Innere Schleife über die N/m Minibatches:
je ein Parameter-Update mit dem Gradienten der Minibatch
5. Stop bei Abbruchbedingung

Merke

Eine Epoche = ein kompletter Durchlauf durch alle Trainingsdaten.

Anzahl Parameter-Updates:

Batch-GD: 1 pro Epoche; SGD: N pro Epoche; Minibatch: N/m pro Epoche.

Klausurbeispiel (WS23/24): $N = 10\,000$, Minibatchgröße 100, 25 Epochen
 $\Rightarrow \frac{10\,000}{100} \cdot 25 = 2\,500$ Updates (Batch-GD: 25).

Die Verlustfunktion hat $N = 10\,000$ Summanden;
 der Gradient hat immer die Dimension der Parameterzahl (z. B. 1 000),
 egal ob ganze Verlustfunktion oder Minibatch!

13.2.1 Wahl der Lernrate: Losskurve und Learning Schedule

Werkzeug: **Losskurve** (x: Epochen, y: Trainingsloss)

für verschiedene Lernraten auf kleiner Teilmenge beobachten. Faustregel:

1. Größte Lernrate finden, bei der SGD nicht divergiert (Faktor-10-Raster)
2. Mit konstanter Lernrate trainieren, bis die Losskurve ein Plateau erreicht
3. Lernrate um konstanten Faktor (z. B. 10) reduzieren
4. Schritte 2 und 3 bis zur Konvergenz wiederholen

13.3 Backpropagation: Gradienten automatisch berechnen

Intuition: einfach erklärt

Für das Training braucht man ∇L , die Ableitung des Netz-Outputs nach *jedem einzelnen* der vielleicht 84 000 Gewichte. Von Hand mit der Kettenregel: hoffnungslos. Der Trick: Man zeichnet die Funktion als **Berechnungsgraph** (jeder Knoten eine elementare Operation), rechnet einmal *vorwärts* (Werte an allen Knoten speichern) und dann einmal *rückwärts* (Ableitungen von hinten nach vorn aufsammeln). Dank dynamischer Programmierung wird jeder Knoten nur zweimal angefasst, statt exponentiell vieler Pfade. Im NN-Kontext heißt dieser Rückwärtsmodus der automatischen Differentiation **Backpropagation**.

13.3.1 Berechnungsgraphen

Jede implementierte Funktion ist eine Abfolge elementarer Operationen.

Beispiel $f(\mathbf{x}) = x_1 x_2 \sin x_3 + e^{x_1 x_2} / x_3$: Zwischenvariablen $x_4, \dots, x_9$ repräsentieren die Rechenschritte (Tools wie PyTorch bauen den Graph automatisch). Berechnungsgraphen beschreiben **Verkettungen von Funktionen**; abgeleitet wird mit der **Kettenregel**.

Merke

Multivariate Kettenregel auf Graphen (Pfadregel):

Die Ableitung des Output-Knotens nach einem Eingangsknoten, ist die *Summe über alle Pfade* zwischen ihnen, wobei entlang jedes Pfades die lokalen Ableitungen *multipliziert* werden.

$$\frac{\partial f(g_1(x), \dots, g_k(x))}{\partial x} = \sum_{i=1}^k \frac{\partial f}{\partial g_i(x)} \cdot \frac{\partial g_i(x)}{\partial x}$$

Direkt auszuwerten wäre exponentiell (3 Schichten à 100 Knoten: 10^6 Pfade!) $\Rightarrow$ dynamische Programmierung: einmal berechnete Teilableitungen werden wiederverwendet.

13.3.2 Der Algorithmus

Gegeben: gerichteter azyklischer Berechnungsgraph mit Knoten x_i , Auswertungsstelle $\mathbf{x}$, Output-Funktion f .

- Vorwärtsphase:** f auswerten, dabei an jedem Knoten den Wert x_i *speichern*.
- Rückwärtsphase:** Für jeden Knoten eine **adjungierte Variable** $\bar{x}_i$ einführen; am Ende gilt $\bar{x}_i = \frac{\partial f}{\partial x_i}$.

1. Initialisiere am Output: $\bar{x}_N = \frac{\partial f}{\partial f} = 1$
2. Wähle einen unbearbeiteten Knoten i , dessen *Kinder* alle bekannt sind, und setze:

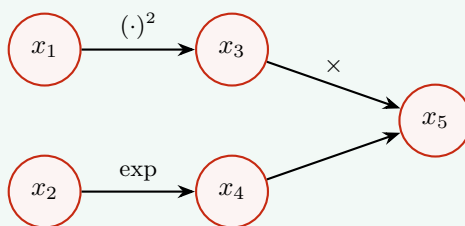
$$\bar{x}_i = \sum_{j \text{ Kind von } i} \bar{x}_j \cdot \frac{\partial x_j}{\partial x_i}$$

3. Wiederholen, bis alle Knoten bearbeitet sind

Beispiel : Backpropagation komplett (Stil aller Altklausuren)

$f(x, y) = x^2 \cdot e^y$ an der Stelle $(2, 0)$.

Berechnungsgraph: $x_1 = x$, $x_2 = y$, $x_3 = x_1^2$, $x_4 = e^{x_2}$, $x_5 = x_3 \cdot x_4$ (Output)



Vorwärtsphase bei $(x, y) = (2, 0)$:

$$x_1 = 2, \quad x_2 = 0, \quad x_3 = x_1^2 = 4, \quad x_4 = e^0 = 1, \quad x_5 = 4 \cdot 1 = 4$$

Rückwärtsphase (Adjungierte):

$$\begin{aligned} \bar{x}_5 &= 1 \\ \bar{x}_4 &= \bar{x}_5 \cdot \frac{\partial x_5}{\partial x_4} = \bar{x}_5 \cdot x_3 = 1 \cdot 4 = 4 \\ \bar{x}_3 &= \bar{x}_5 \cdot \frac{\partial x_5}{\partial x_3} = \bar{x}_5 \cdot x_4 = 1 \cdot 1 = 1 \\ \bar{x}_2 &= \bar{x}_4 \cdot \frac{\partial x_4}{\partial x_2} = \bar{x}_4 \cdot e^{x_2} = 4 \cdot 1 = 4 \\ \bar{x}_1 &= \bar{x}_3 \cdot \frac{\partial x_3}{\partial x_1} = \bar{x}_3 \cdot 2x_1 = 1 \cdot 4 = 4 \end{aligned}$$

$$\Rightarrow \nabla f(2, 0) = (\bar{x}_1, \bar{x}_2) = (4, 4)$$

Probe analytisch: $\partial f / \partial x = 2xe^y = 4$; $\partial f / \partial y = x^2 e^y = 4$. ✓

Achtung bei verzweigten Graphen (z. B. $f = \sin(x^2) + \cos(x^2)$ oder $(x_1 + x_2)e^{x_1 + x_2}$):
Hat ein Knoten *mehrere Kinder*, werden die Beiträge **summiert**: $\bar{x}_i = \sum_j \bar{x}_j \partial x_j / \partial x_i$.

13.3.3 Anwendung auf neuronale Netze

Zwei Besonderheiten gegenüber gewöhnlichen Berechnungsgraphen:

(1) Neuron = zwei Knoten: gewichtete Summe $z^{[i]} = W^{[i]} a^{[i-1]}$
und Aktivierung $a^{[i]} = g(z^{[i]})$, mit lokalen Ableitungen

$$\frac{\partial z^{[i]}}{\partial a^{[i-1]}} = W^{[i]} \quad \frac{\partial a^{[i]}}{\partial z^{[i]}} = g'(z^{[i]})$$

(2) Ableitung nach den Kanten (W), nicht nur den Knoten:

Erst normal rückwärts bis zum Knoten unmittelbar nach $W^{[i]}$, dann „nachdifferenzieren“:

$$\frac{\partial y}{\partial W^{[i]}} = \frac{\partial y}{\partial z^{[i]}} \cdot \frac{\partial z^{[i]}}{\partial W^{[i]}} = \bar{z}^{[i]} \cdot a^{[i-1]}$$

Beispiel : Backprop durch ein Mini-Netz (Vorlesungsbeispiel)

Kette $x \rightarrow z^{[1]} \rightarrow a^{[1]} \rightarrow z^{[2]} \rightarrow a^{[2]} = \hat{y}$ mit Sigmoid g ,
 skalaren Gewichten $W^{[1]} = 2$, $W^{[2]} = -1$, Input $x = 1$. (Nützlich: $\sigma'(z) = \sigma(z)(1 - \sigma(z))$.)

Vorwärts: $z^{[1]} = 2$, $a^{[1]} = \sigma(2) = 0,88$, $z^{[2]} = -0,88$, $\hat{y} = \sigma(-0,88) = 0,29$

Rückwärts: $\bar{a}^{[2]} = 1$; $\bar{z}^{[2]} = g'(-0,88) = 0,29 \cdot 0,71 \approx 0,21$;
 $\bar{a}^{[1]} = 0,21 \cdot W^{[2]} = -0,21$; $\bar{z}^{[1]} = -0,21 \cdot g'(2) \approx -0,02$

Nachdifferenzieren: $\bar{W}^{[2]} = \bar{z}^{[2]} \cdot a^{[1]} = 0,21 \cdot 0,88 = 0,18$;
 $\bar{W}^{[1]} = \bar{z}^{[1]} \cdot a^{[0]} = -0,02 \cdot 1 = -0,02$

Im allgemeinen Fall (Matrizen) wird die Kettenregel mit Jacobimatrizen ausgewertet;
 $\frac{\partial a^{[i]}}{\partial z^{[i]}} = \text{diag}(g'(z^{[i]}))$ und $\frac{\partial \mathbf{y}}{\partial \mathbf{W}^{[i]}} = \left(\frac{\partial \mathbf{y}}{\partial \mathbf{z}^{[i]}}\right)^T \cdot (\mathbf{a}^{[i-1]})^T$ (äußeres Produkt). Die Verlustfunktion
 wird einfach als zusätzlicher Knoten an den Graph angehängt und mitdifferenziert.
 Software: **PyTorch** (`torch.tensor` + `autograd`).

13.4 Probleme beim Training und Lösungsansätze**13.4.1 Vanishing Gradients****Merke**

Problem: Die Ableitung nach Gewichten *vorderer* Schichten enthält das *Produkt* der Aktivierungs-Ableitungen aller nachfolgenden Schichten:

$$\frac{\partial y}{\partial W^{[1]}} = \sigma'(\cdot) W^{[3]} \sigma'(\cdot) W^{[2]} \sigma'(\cdot) x$$

Da σ' fast überall nahe Null ist (Maximum 0,25!), wird das Produkt mit jeder Schicht kleiner $\Rightarrow$ vordere Schichten lernen praktisch nicht („Lerngeschwindigkeit“ sinkt, je mehr nachfolgende Schichten). Fundamentales Problem tiefer Netze.

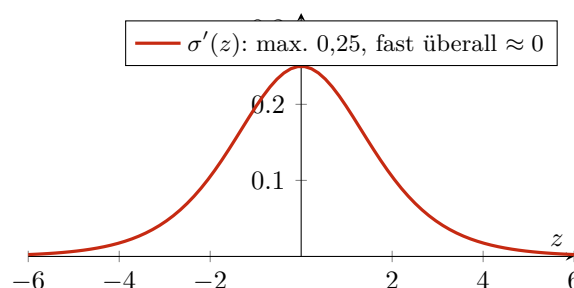


Abbildung 23: Ursache des Vanishing-Gradient-Problems: Die Sigmoid-Ableitung ist klein; viele kleine Faktoren multipliziert ergeben fast Null.

Lösungsansätze:

- **Andere Aktivierungsfunktionen:** ReLU (Ableitung = 1 für $x > 0$, aber = 0 für $x < 0$, Vanishing noch möglich); **Leaky ReLU** = $\max(\varepsilon x, x)$ (Ableitung nie 0). Kehrseite: *Exploding Gradients* möglich $\rightarrow$ Regularisierung.

- **Adaptive Lernraten** (Adagrad, RMSProp, Adam):
automatisch eine *eigene Lernrate pro Parameter*. Adagrad:

$$w_i^{[k+1]} = w_i^{[k]} - \frac{\alpha^{[k]}}{\epsilon + \sqrt{s_i^{[k]}}} \nabla L(\mathbf{w}^{[k]})_i, \quad s_i^{[k]} = \sum_{j=1}^k \left(\frac{\partial L(\mathbf{w}^{[j]})}{\partial w_i} \right)^2$$

Lernraten einzelner Parameter sind umgekehrt proportional zum Betrag ihrer bisherigen Ableitungen ($\epsilon \approx 10^{-8}$ gegen Division durch 0).

- **Vortrainieren** von Schichten, z. B. mit Autoencodern (Kap. 15).

13.4.2 Regularisierung für neuronale Netze

Technik	Idee
Strafterm / Weight Decay	ℓ_1 - oder ℓ_2 -Term auf die Verlustfunktion addieren, wie bei linearen Modellen: $\min_{\mathbf{w}} L(\mathbf{w}) + \lambda \sum_i w_i^2$
SGD selbst	In jeder Iteration wird eine leicht andere Funktion ausgewertet, verhindert Konvergenz in ein Minimum, das nur Eigenheiten der Trainingsdaten abbildet
Early Stopping	Fehler auf Validierungsdaten überwachen; stoppen (und alte Gewichte laden), wenn er z. B. M Epochen lang nicht mehr sinkt, obwohl der Trainingsfehler weiter fällt = Beginn von Overfitting
Dropout	In jeder SGD-Iteration eine zufällige Teilmenge der Neuronen auf Null setzen ($p \approx 0,2$ bis $0,5$) $\Rightarrow$ robustere interne Darstellungen, kein Neuron wird übergewichtet. Nur im <i>Training</i> ; Aktivierungen werden mit $\frac{1}{1-p}$ skaliert, damit die Outputsumme im Mittel gleich bleibt. Evaluierung mit vollem Netz.

Tabelle 26: Regularisierungstechniken für NN, „Nennen Sie zwei Möglichkeiten“ ist eine Standardfrage (SS22 A2b).

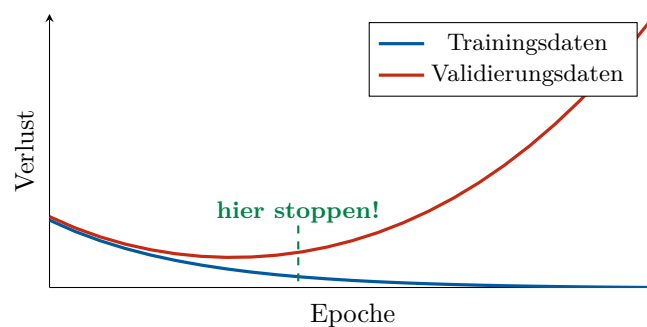


Abbildung 24: **Early Stopping**: Der Validierungsfehler beginnt zu steigen, während der Trainingsfehler weiter sinkt, ab hier overfittet das Netz.

13.4.3 Hyperparameter-Übersicht

Training: Lernrate (+ Schedule), Weight-Decay-Parameter λ , Batchgröße, Dropout-Wahrscheinlichkeit. Architektur: Anzahl Schichten, Neuronen pro Schicht, Aktivierungsfunktion, Schichttypen/Verbindungen $\Rightarrow$ enorme Anzahl an Stellschrauben.

Klausur-Fokus

Backpropagation ist in jeder Klausur eine eigene Aufgabe (8 bis 10 Punkte):

1. Berechnungsgraph zeichnen (Knoten x_i mit *Funktionsdefinition* beschriften!)
2. Vorwärtsphase: alle Knotenwerte an gegebener Stelle
3. Rückwärtsphase: alle Adjungierten $\bar{x}_i$, beginnend mit $\bar{x}_N = 1$; bei Verzweigung **summieren**
4. Bisher geprüfte Funktionen: $x^2 e^y$; $(e^x \cos x)^2$; $x_1^2 \cos(x_2)$; $\sin(x_1 x_2) + \cos(x_1 x_2)$; $x^2 + y^2$; $(x_1 + x_2)e^{x_1 + x_2}$
5. Erweiterung WS22/23: Ableitung nach Kanten-Gewicht W , erst $\bar{z}$ am Knoten nach W , dann $\bar{z} \cdot \partial z / \partial W$

Außerdem:

SGD-Dimensionsfragen $\rightarrow$ Summanden $= N$; Gradientendimension $=$ Parameterzahl;
 Updates $= \frac{N}{m} \cdot$ Epochen; Lernratenverhalten; Vanishing Gradient erklären +
 Gegenmaßnahmen (WS22/23 A3c); Early Stopping beschreiben (mehrfach gefragt!).

14 Faltungsnetze (CNNs)

14.1 Motivation

Bilder als Tensoren: CIFAR100-Bild $= 32 \times 32$ Pixel $\times$ 3 Kanäle (RGB)

$\Rightarrow$ Element des $\mathbb{R}^{32 \times 32 \times 3}$ (*Feature-Map*). Vollvernetzte Schichten hätten Millionen Parameter und ignorieren die räumliche Struktur.

Intuition: einfach erklärt

Idee der Faltungsnetze: Lerne zu erkennen, *ob* bestimmte Muster *irgendwo* im Bild vorkommen, erst einfache (Kanten, Farbflecken), daraus zusammengesetzt komplexere (Augen, Ohren), bis zur Klasse („Katze“). Ein **Filter** ist wie eine kleine Schablone, die über das Bild geschoben wird: Wo das Bild der Schablone ähnelt, leuchtet die *Featuremap* auf. Der Clou: Welche Schablonen nützlich sind, wird *mitgelernt*.

14.2 Die Faltungsoperation

Ein Faltungskern $\mathbf{W}$ (Filter, Filterkern, Faltungsmatrix) der Größe $k \times k$ wird auf alle $k \times k$ -Submatrizen des Bildes angewendet: elementweises Produkt
 + Aufsummieren ergibt je *eine Zahl* der Featuremap:

$$I_{out}(i, j) = \sum_{m=1}^k \sum_{n=1}^k I_{in}(i+m-1, j+n-1) \cdot W(m, n)$$

(Technisch ist das eine *Kreuzkorrelation*;
 eine echte Faltung würde die Indizes gegenläufig durchlaufen,
 im ML egal, da die Filter gelernt werden.)

Beispiel : Faltung von Hand (Klausur WS22/23, Aufgabe 5a)

Eingabe und 2×2 -Kern, kein Padding, Stride 1:

$$I = \begin{pmatrix} 2 & 0 & 1 \\ 0 & 1 & 3 \\ 4 & 1 & 2 \end{pmatrix}, \quad W = \begin{pmatrix} -1 & -1 \\ 2 & 2 \end{pmatrix}$$

Vier Positionen ($3 - 2 + 1 = 2$ pro Richtung):

$$\begin{aligned} (1, 1) : -2 - 0 + 0 + 2 &= 0 & (1, 2) : -0 - 1 + 2 + 6 &= 7 \\ (2, 1) : -0 - 1 + 8 + 2 &= 9 & (2, 2) : -1 - 3 + 2 + 4 &= 2 \end{aligned}$$

$$\Rightarrow \text{Featuremap} = \begin{pmatrix} 0 & 7 \\ 9 & 2 \end{pmatrix}$$

Tipp: Kern gedanklich über die linke obere Ecke legen, elementweise multiplizieren, summieren; dann um Stride 1 weiterschieben.

Warum erkennen Faltungen Features ? Ist eine Bildregion dem Kern ähnlich, entsteht dort ein großer Wert, die Featuremap hebt die Regionen hervor, die dem Muster des Kerns ähneln (z. B. Sobel-Filter für vertikale/horizontale Kanten).

14.3 Faltungsschichten im neuronalen Netz

- Die Filterkerne werden **gelernt** (+ Bias + Aktivierungsfunktion, meist ReLU); Ableitungen wie gewohnt per Backpropagation.
- **Weight Sharing:** Jedes Neuron der Featuremap nutzt *dieselben* Kerngewichte $\Rightarrow$ drastisch weniger Parameter als fully connected. (Beispiel: 5×5 -Kern: jedes Output-Neuron mit 25 Input-Neuronen verbunden, aber alle teilen sich dieselben 25+1 Gewichte.)
- **Mehrere Kanäle:** Bei RGB hat der Filter auch 3 Kanäle ($3 \times 3 \times 3 = 27$ Gewichte); summiert wird über *alle* Dimensionen $\Rightarrow$ Ergebnis ist eine *2D*-Featuremap. Die Tiefe von Eingabe und Filter muss übereinstimmen!
- **Mehrere Filter:** K Filter $\Rightarrow K$ Featuremaps; Faltungsschichten sind Abbildungen $3D\text{-Tensor} \rightarrow 3D\text{-Tensor}$ und damit stapelbar. Auch 1×1 -Faltungen sind üblich (Aggregation über die Tiefe).

14.3.1 Hyperparameter: Padding und Stride

- **Padding P :** Auffüllen des Rands mit Nullen. Ohne Padding schrumpft das Bild um (Filtergröße $- 1$); *Half-Padding* erhält bei ungerader Kerngröße die Dimension; verhindert auch, dass Randpixel unterrepräsentiert sind.
- **Stride S :** Verschiebungsweite des Filters. $S > 1 \Rightarrow$ kleinere Output-Dimension, Informationsverlust; üblich ist $S = 1$. *Stride ändert die Parameteranzahl nicht!* (Klausurfrage SS23 5c)

Merke

Die zwei Klausur-Formeln für eine Faltungsschicht mit Eingabe $W_{in} \times H_{in} \times D_{in}$, K Filtern der Größe $W_K \times H_K \times D_{in}$, Stride S , Padding P :

$$W_{out} = \left\lfloor \frac{W_{in} - W_K + 2P}{S} \right\rfloor + 1 \quad H_{out} \text{ analog} \quad D_{out} = K$$

$$\#Parameter = K \cdot (W_K \cdot H_K \cdot D_{in} + 1) \quad (+1 = \text{Bias pro Filter})$$

Beispiel : Dimensionen und Parameter (Klausur SS23, Aufgabe 5b)

28×28 -RGB-Bilder ($D_{in} = 3$), 4 Filter à 3×3 , kein Padding, Stride 1.

Parameter: je Filter $3 \cdot 3 \cdot 3 + 1 = 28$; gesamt $4 \cdot 28 = 112$.

Output: $W_{out} = \frac{28-3+0}{1} + 1 = 26 \Rightarrow \text{Tensor } 26 \times 26 \times 4$.

Variante WS25/26 (mit Padding): $28 \times 28 \times 1$, 8 Filter 3×3 , $S=1$, Zero-Padding $P=1$:
 $W_{out} = \frac{28-3+2}{1} + 1 = 28 \Rightarrow 28 \times 28 \times 8$; Parameter = $8 \cdot (3 \cdot 3 \cdot 1 + 1) = 80$.

14.4 Pooling**Merke**

Max-Pooling gibt blockweise (z.B. 2×2 , Stride 2) das Maximum der Featuremap aus: erkennt, ob ein Feature in einer Region vorhanden ist, verwirft die exakte Position.
 Ergebnis: kleinere Featuremaps $\Rightarrow$ weniger Parameter in späteren Schichten.

$$W_{out} = \left\lfloor \frac{W_{in} - F}{S} \right\rfloor + 1, \quad D_{out} = D_{in}, \quad \mathbf{0 \text{ lernbare Parameter!}}$$

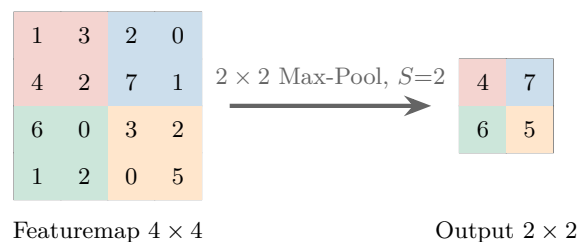


Abbildung 25: Max-Pooling: pro Block bleibt nur das Maximum übrig.

14.5 Typische Architekturen**Merke****Architekturmuster:**

INPUT $\rightarrow$ $[[\text{CONV} \rightarrow \text{RELU}] * N \rightarrow \text{POOL ?}] * M \rightarrow [\text{FC} \rightarrow \text{RELU}] * K \rightarrow \text{FC}$

Erst Faltungs-/Pooling-Blöcke (**Feature Learning**: wo sind Kanten/Formen/Muster), dann Fully-Connected-Schichten (**Klassifizierung**: Features gewichten); am Ende immer eine FC-Schicht auf die Outputklassen (Softmax).

- **LeNet** (1998, Handschriftenerkennung):
[[CONV->RELU]->POOL]*2 -> FC -> FC -> FC ($N=1, M=2, K=2$)
- **VGG16** (2014, Bildklassifikation): [[CONV->RELU]*2->POOL]*2 -> [[CONV->RELU]*3->POOL]*3 -> [FC->RELU]*3 -> SOFTMAX
- ILSVRC-Wettbewerb: Fehlerrate von 28 % (flache Modelle, 2010) auf 2 bis 3 % (152 Schichten, 2015 bis 2017), der Durchbruch des Deep Learning.
- Visualisierung (Zeiler/Fergus): frühe Schichten lernen Kanten (low-level), mittlere geometrische Muster (mid-level), späte komplexe Objekte (high-level).
- Analog existieren **Conv1D** (Zeitreihen, Audio) und **Conv3D** (Video, Volumendaten).

Klausur-Fokus

- **Faltung von Hand** ausrechnen (3×3 -Input, 2×2 -Kern), WS22/23, WS23/24
- **Output-Dimension und Parameterzahl** einer Faltungsschicht (Formeln oben!), SS23, Probeklausur, WS25/26; RGB nicht vergessen: $D_{in} = 3$ steckt in der Parameterformel
- Faltung als Feedforward-Schicht interpretieren: Anzahl Neuronen (= Größe der Featuremap), Verbindungen (= Neuronen $\times$ Kerngröße), aber nur (Kerngröße) *Gewichte* (Weight Sharing!), WS22/23 5b, WS23/24 5b
- Max-Pooling beschreiben; **Pooling hat 0 Parameter** (WS25/26 6c); Stride ändert Parameterzahl nicht, nur Output-Dimension (SS23 5c)
- Konzept: Wie erkennen Faltungen Muster? (hohe Werte = Region ähnelt dem Kern), SS23 5a

15 Autoencoder

15.1 Idee und Aufbau

Intuition: einfach erklärt

Ein Autoencoder bekommt eine Aufgabe, die absurd einfach klingt: *Gib einfach deine Eingabe wieder aus.* Der Haken: In der Mitte des Netzes gibt es einen **Flaschenhals**, eine Schicht mit viel weniger Neuronen als die Eingabe. Das Netz muss die Daten also erst *komprimieren* (Encoder) und dann wieder *rekonstruieren* (Decoder). Was im Flaschenhals übrig bleibt, ist zwangsläufig das Wesentliche der Daten, eine gelernte, kompakte Repräsentation. Unüberwachtes Lernen:
Es braucht keine Labels, das „Label“ ist die Eingabe selbst.

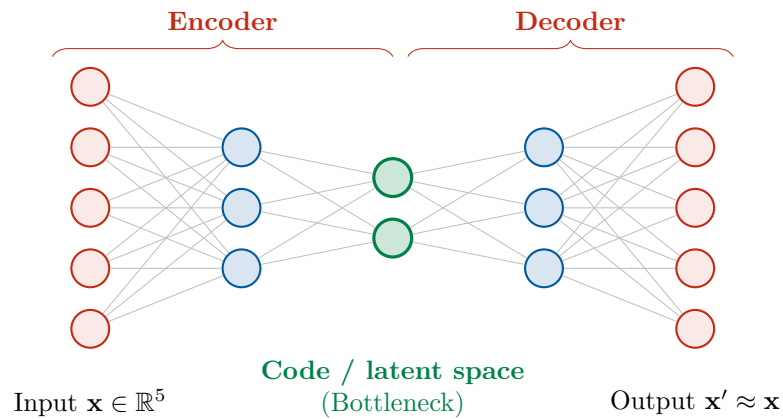


Abbildung 26: Autoencoder: Ein- und Ausgabeschicht haben dieselbe Dimension; mittlere Schicht ist typischerweise kleiner, verlustbehaftete Kompression. Meist ungerade Schichtanzahl und symmetrische Architektur.

Merke

Training: Rekonstruktionsfehler (MSE) zwischen Original und Rekonstruktion minimieren:

$$\min_{\mathbf{w}} \frac{1}{N} \sum_{i,j} (x_{ij} - x'_{ij})^2 \quad (+ \text{Regularisierung}) \quad \text{bzw.} \quad L(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N \|\mathbf{x}^{(i)} - \mathbf{x}'^{(i)}\|^2$$

Nach dem Training liefert der Output der mittleren Schicht den **Code** (reduzierte Darstellung).

Für Encoder/Decoder steht der gesamte NN-Baukasten zur Verfügung: beliebig viele Schichten (*Deep Autoencoder*), Faltungsschichten (*Convolutional Autoencoder*), diverse Aktivierungen und Regularisierungen (mit Regularisierung darf die Mittelschicht sogar *größer* sein als der Input $\rightarrow$ *Sparse Autoencoder*). Operationen wie Max-Pooling sind nicht exakt umkehrbar $\Rightarrow$ völlige Symmetrie wird i. d. R. nicht erreicht.

15.2 Lineare Autoencoder und PCA

Einfachster Fall: 1 verdeckte Schicht, *keine* Aktivierungsfunktion. Vorhersage $\mathbf{x}' = VU\mathbf{x}$ (Encoder-Matrix $U \in \mathbb{R}^{k \times p}$, Decoder-Matrix $V \in \mathbb{R}^{p \times k}$); für die ganze Featurematrix: $X' = XU^TV^T$.

Trainingsproblem:

$$\min_{U,V} \|X - XU^TV^T\|_F^2$$

(quadrierte Frobeniusnorm = Summe der quadrierten Matrixeinträge = Least-Squares-Verlust).

Merke

Zusammenhang mit der PCA (Klausurfrage SS22 A5c, SS23 A5d!): Die PCA-Projektionsmatrix U_k (Eigenvektoren zu den k größten Eigenwerten von $X^T X$) minimiert genau den Rekonstruktionsfehler $\|X - XU_k^T U_k\|_F^2$. Sie ist damit *eines der Minima* des linearen Autoencoders. Erzwingt man $V = U^T$ (geteilte Gewichte, halbiert die Parameterzahl), „simuliert“ der lineare Autoencoder eine PCA: gleicher Unterraum (die Zeilen von U sind nur nicht notwendigerweise orthonormal, Ergebnis bis auf Drehung/Spiegelung äquivalent).

Rezept (Klausur SS22): Autoencoder, der eine 2D-Darstellung von 3D-Daten wie die PCA lernt: eine 2×3 -Matrix U , die für Encoder *und* Decoder ($V = U^T$) verwendet wird $\Rightarrow$ **6 Parameter**. Nach dem Training Ausgabeschicht entfernen
 $\rightarrow$ verdeckte Schicht liefert die PCA-Projektion.

Einordnung: Lineare Autoencoder drücken spektrale Verfahren (PCA, SVD, Matrixfaktorisierung) in der Sprache neuronaler Netze aus. In der Praxis berechnet man eine SVD nicht per Autoencoder (es gibt effizientere Numerik), aber der Autoencoder lässt sich mit minimalen Änderungen *nichtlinear* machen (Aktivierungen, mehr Layer), ohne am Framework (NN + SGD + Backprop) etwas zu ändern.

15.3 Anwendungen

15.3.1 Nichtlineare Dimensionsreduktion und Vortrainieren

Deep Autoencoder lernen niedrigdimensionale Darstellungen komplexer Klassenverteilungen, auf denen Klassen oft *linear* separierbar werden (Feature-Extraktion). Wichtige zweite Anwendung: **Vortrainieren** tiefer Netze (gute Initialisierung gegen Vanishing Gradients):

1. Flachen Autoencoder auf den Trainingsdaten trainieren
 $\rightarrow$ Initialisierung der Gewichte der *ersten* Schicht
2. Zweiten Autoencoder auf der *reduzierten Darstellung* aus Schritt 1 trainieren
 $\rightarrow$ Gewichte der zweiten Schicht
3. Klassifizierer mit diesen initialen Gewichten normal trainieren
 (Ausgabeschicht lernen + Fine-Tuning der übrigen)

15.3.2 Convolutional Autoencoder

Encoder mit Faltungs-/Pooling-Schichten; der Decoder braucht Umkehrungen:

- **Unpooling** (Upsampling): Position des Maximums beim Pooling speichern (*switch variables*); Unpooling setzt den Wert an die richtige Stelle zurück und füllt mit Nullen auf (Informationsverlust bleibt).
- **Deconvolution / transponierte Faltung:** *nicht* die mathematische Inverse, eine Faltung, die Ein-/Ausgabedimension der zugehörigen Faltung vertauscht (per Padding und eingefügten Nullen); ihre Kerngewichte werden ebenfalls gelernt.

Oft liegt in der Mitte zusätzlich ein Flatten-/Dense-Layer für eine vektorielle Code-Darstellung.

15.3.3 Anomalieerkennung (rekonstruktionsbasiert)

Intuition: einfach erklärt

Der Autoencoder wird nur mit *normalen* Daten trainiert, er lernt, Normales gut zu komprimieren und zu rekonstruieren. Kommt etwas Ungewöhnliches, passt es nicht in die gelernte Kompression: Der **Rekonstruktionsfehler schießt hoch** $\Rightarrow$ Anomalie. Wie ein Karikaturist, der nur Gesichter zeichnen gelernt hat, bei einem Auto wird die Zeichnung unbrauchbar.

Vorgehen (Beispiel Prozessindustrie / Hefeproduktion):

1. Autoencoder auf normalen Daten trainieren;
Statistik über den typischen Rekonstruktionsfehler erheben
2. Im Betrieb: Daten auf rollierendem Fenster komprimieren + rekonstruieren,
Rekonstruktionsfehler überwachen
3. **Alarm, falls Fehler > Schwellwert**

Warum hier unüberwacht? Viele Daten, aber kaum gelabelte Anomalien, und Labeling braucht teures Fachwissen. (Gleiches Prinzip im Preisach-Teil: Waldschadens-Erkennung mit Autoencoder auf gesunden Bäumen.)

15.3.4 Zusatz: Matrixfaktorisierung / Recommender als Autoencoder

Rating-Matrix $R \approx PQ^T$ (Rang k); R ist unvollständig (nicht jeder User bewertet jeden Film) $\Rightarrow$ SVD nicht direkt anwendbar. Als Netz: Eingabe one-hot über User, verdeckte Schicht = k Faktoren, Ausgabe = Filme; Verlust nur über *beobachtete* Ratings ($e_i P Q^T = \vec{p}_i^T \vec{q}_j \approx r_{ij}$), identisch zur unvollständigen Matrixfaktorisierung (Brücke zum Vorlesungsteil Preisach).

Stichwort	Kernaussage
Architektur	Input-Dim = Output-Dim; Bottleneck in der Mitte (Code / latent space)
Training	Eingabe rekonstruieren, MSE-Loss, unüberwacht
Linear	= PCA (bei geteilten Gewichten $V = U^T$)
Anwendungen	Dimensionsreduktion / Feature Learning, Vortrainieren, Anomalieerkennung, Denoising, Empfehlungsdienste

Tabelle 27: Autoencoder in vier Zeilen, das Minimalgerüst für die Klausur.

Klausur-Fokus

- „Was ist ein Autoencoder, wofür wird er eingesetzt?“, Musterantwort: *NN, dessen Eingabeschicht dieselbe Dimension wie die Ausgabeschicht hat; Ziel ist eine reduzierte Darstellung des Datensatzes; trainiert wird auf Rekonstruktion der Eingabe.* (SS22 A5a, WS22/23 A5c)
- Verlustfunktion hinschreiben: $L(\mathbf{w}) = \frac{1}{N} \sum_i \|\mathbf{x}^{(i)} - \mathbf{x}'^{(i)}\|^2$ (SS22 A5b)
- PCA-Autoencoder skizzieren **inkl. Parameterzahl**
(2×3 -Matrix geteilt $\Rightarrow$ 6 Parameter) (SS22 A5c)
- Zusammenhang Autoencoder $\leftrightarrow$ PCA in einem Satz:
linearer Autoencoder lernt die gleiche Transformation wie eine PCA (SS23 A5d)
- Anwendung nennen: Vortrainieren tiefer Netze, Anomalieerkennung

16 Sequenzielle Entscheidungsprobleme & Reinforcement Learning

Intuition: einfach erklärt

Bisher ging es im überwachten Lernen immer um *Vorhersagen*: ein Modell f schätzt y aus $\mathbf{x}$. Sequenzielle Entscheidungsprobleme (SDPs) drehen das Ziel um: Gesucht sind nicht gute Vorhersagen, sondern gute *Entscheidungen*. Man trifft nacheinander Aktionen, beobachtet neue Information, trifft die nächste Aktion, und so weiter, mit dem Ziel, einen aufsummierten Gewinn (Reward) zu maximieren.

Die Schwierigkeit: Entscheidungen werden *unter Unsicherheit* getroffen, ihre Auswirkung auf zukünftige Rewards ist nicht genau bekannt.

Merke

Ein **sequenzielles Entscheidungsproblem (SDP)** hat die Form

Entscheidung $\rightarrow$ Information $\rightarrow$ Entscheidung $\rightarrow$ Information $\rightarrow \dots$

Über einen endlichen Zeithorizont $t = 0, 1, \dots, T$ werden **Aktionen** so gewählt, dass der kumulierte **Reward** maximiert wird. *Reinforcement Learning* ist der bekannteste, aber nicht der einzige Ansatz, solche Probleme zu lösen.

16.1 Grundbegriffe: Zeitdiskrete dynamische Systeme

Merke

Ein **zeitdiskretes dynamisches System** besteht aus:

- **Zustand $\mathbf{x}_t$** : alle Informationen, die man braucht, um eine Entscheidung zu treffen und den Reward auszuwerten.
- **Entscheidung / Aktion $\mathbf{u}_t$** : die zum Zeitpunkt t getroffene Wahl.
- **Exogene Information $\mathbf{W}_{t+1}$** : Information, die zum Zeitpunkt t *noch nicht* bekannt ist, zum Zeitpunkt $t + 1$ aber schon (mathematisch eine **Zufallsvariable**).

Die zeitliche Entwicklung beschreibt die **Übergangsfunktion $\mathbf{F}$** :

$$\mathbf{x}_{t+1} = \mathbf{F}(\mathbf{x}_t, \mathbf{u}_t, \mathbf{W}_{t+1}).$$

Intuition: einfach erklärt

Batterie-Arbitrage (durchgehendes Beispiel): Ein Großspeicher kauft Strom, wenn er billig ist, und verkauft, wenn er teuer ist, um Gewinn zu machen und das Netz zu stabilisieren. Aktion u_t : laden ($u_t > 0$, Strom kaufen) oder entladen ($u_t < 0$, Strom verkaufen). Zustand: Ladestand S_t und aktueller Preis p_t . Die exogene Information ist der *nächste Strompreis*, den man heute noch nicht kennt.

Merke

Die **Reward-Funktion** $C(\mathbf{x}_t, \mathbf{u}_t, \mathbf{W}_{t+1})$ ist der unmittelbare Gewinn in Periode t . Für das Batteriebeispiel: $C(S_t, p_t, u_t) = -p_t u_t$ (man zahlt p_t beim Laden, erhält p_t beim Entladen).

16.2 Policy und Problemformulierung

Das SDP besteht darin, Entscheidungen zu finden, die den *kumulierten* Reward maximieren. Da die Rewards von zufälligen Größen abhängen (z. B. den Preisen), ist die Summe nicht-deterministisch. Man maximiert deshalb ihren **Erwartungswert**.

Merke

Eine **Policy** π ist eine Funktion, die einen Zustand auf eine Aktion abbildet:

$$\pi : \mathbb{R}^{N_s} \rightarrow \mathbb{R}^{N_u}, \quad \mathbf{u}_t = \pi(\mathbf{x}_t).$$

Das **sequenzielle Entscheidungsproblem** ist dann eine Suche über Policies:

$$\max_{\pi} \mathbb{E} \left[\sum_{t=1}^T C(\mathbf{x}_t, \pi(\mathbf{x}_t), \mathbf{W}_t) \right] \quad \text{s.t.} \quad \mathbf{x}_{t+1} = \mathbf{F}(\mathbf{x}_t, \pi(\mathbf{x}_t), \mathbf{W}_{t+1}), \quad \mathbf{x}_0 \text{ gegeben.}$$

Intuition: einfach erklärt

Hier schließt sich der Kreis zum maschinellen Lernen: Überwachtes Lernen sucht eine Funktion f , die für unbekannte Situationen gute *Vorhersagen* macht; ein SDP sucht eine Policy π , die für unbekannte Situationen den *kumulierten Reward* maximiert. In beiden Fällen optimiert man einen Erwartungswert über eine unbekannte Zukunft.

Merke

Was gehört in den Zustand?

Der Zustandsvektor $\mathbf{x}_t$ enthält genau die Information, die man braucht, um die Aktion $\mathbf{u}_t = \pi(\mathbf{x}_t)$ zu bestimmen und den Reward auszuwerten. Welche Information das ist, kann von der gewählten Policy abhängen.

16.3 Lösungsstrategien

16.3.1 Parametrische Approximation (Policy Gradient)

Man parametrisiert die Policy direkt als $\pi_{\mathbf{w}}$ (z. B. ein neuronales Netz mit dem Zustand als Eingabe und der Aktion als Ausgabe) und lernt die Parameter $\mathbf{w}$. Wie im überwachten Lernen ersetzt man den erwarteten Reward durch einen **Mittelwert über Stichproben** (z. B. mehrere historische oder simulierte Preisverläufe):

$$\frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T C(\mathbf{x}_t^i, \pi_{\mathbf{w}}(\mathbf{x}_t^i), \mathbf{W}_t^i).$$

Das ist eine deterministische Funktion, die per **Backpropagation** abgeleitet und mit (stochastischem) Gradientenabstieg optimiert werden kann. Dieselben Probleme wie im überwachten Lernen drohen (nicht repräsentative Daten, Overfitting der Policy).

Merke

Simulator nötig?

Wenn die Aktion den Zufallsprozess *nicht* beeinflusst (Strompreise hängen nicht davon ab, ob wir kaufen), reicht ein fester Trainingsdatensatz. Beeinflusst die Aktion den Prozess (z. B. der Gegnerzug beim Schach), braucht man einen **Simulator**, der neue Stichproben erzeugt. In der Literatur heißt dieser Ansatz oft *policy gradient*.

16.3.2 Model-predictive Control (MPC)

Stammt aus der Regelungstechnik. Pro Zeitperiode:

1. **Vorhersage** der exogenen Information über einen Horizont $\hat{\mathbf{W}}_{t+1}, \dots, \hat{\mathbf{W}}_{t+n}$ machen und so tun, als wäre sie korrekt.
2. Das resultierende **deterministische** Optimierungsproblem lösen (z. B. ein lineares Programm mit einem Solver).
3. Nur die Aktion $\mathbf{u}_t$ der *aktuellen* Periode umsetzen, den Rest der Lösung verwerfen.

Im nächsten Zeitschritt wird mit aktualisierter Vorhersage neu gerechnet (**rollierender Horizont**). Das Ersatzproblem skizziert eine mögliche Zukunft (*lookahead problem*).

MPC gehört in der Praxis zu den erfolgreichsten Methoden für SDPs.

16.3.3 Value-Function-Approximation („Reinforcement Learning“)

Dieser Ansatz schätzt eine **Wertfunktion** (value function), die angibt, wie gut ein Zustand langfristig ist, und leitet daraus die Policy ab. Grundlage sind die **Bellman-Gleichung** und (approximative) **dynamische Programmierung**. So wurde z. B. AlphaZero trainiert.

Merke

Achtung, der Begriff „Reinforcement Learning“ wird uneinheitlich verwendet,

- (a) das Suchen von Policies per trial-and-error,
- (b) als Synonym für „sequenzielles Entscheidungsproblem“,
- (c) speziell für den value-function-Ansatz.

Die Aussage „Ich löse das mit RL“ ist deshalb mehrdeutig.

Klausur-Fokus

Theorieverständnis (keine Rechenaufgaben):

Das SDP-Schema (Entscheidung, Information, Entscheidung, ...) erklären; die drei Bestandteile eines dynamischen Systems (Zustand, Aktion, exogene Information) plus Übergangsfunktion benennen; Reward und Policy definieren; das SDP als Suche über Policies formulieren und erklären, warum man den *Erwartungswert* maximiert; den Bezug Vorhersage (Supervised) vs. Entscheidung (SDP) erläutern; die drei Lösungsansätze (parametrische Approximation/Policy Gradient, MPC, Value-Function/RL) unterscheiden und je in einem Satz erklären; sagen können, wann ein Simulator nötig ist.

Teil III

Anhang

17 Formelsammlung (Schnellreferenz)

Distanzen und Ähnlichkeiten

Maß	Formel
Minkowski	$d_r(\vec{x}, \vec{y}) = (\sum_k x_k - y_k ^r)^{1/r}$
Euklidisch ($r=2$)	$d_2 = \sqrt{\sum_k (x_k - y_k)^2}$
Cityblock ($r=1$)	$d_1 = \sum_k x_k - y_k $
Chebyshev ($r=\infty$)	$d_\infty = \max_k x_k - y_k $
Mahalanobis	$d_C(\vec{x}, \vec{y}) = \sqrt{(\vec{x} - \vec{y})^T C^{-1} (\vec{x} - \vec{y})}$
DTW (kumuliert)	$d_{ij} = d'_{ij} + \min\{d_{i-1,j}, d_{i,j-1}, d_{i-1,j-1}\}$
SMC	$(f_{11} + f_{00}) / (f_{11} + f_{00} + f_{10} + f_{01})$
Jaccard	$f_{11} / (f_{11} + f_{10} + f_{01})$
Kosinus	$\langle \vec{x}, \vec{y} \rangle / (\ \vec{x}\ \ \vec{y}\) \in [-1, 1]$

Statistik / PCA

Größe	Formel
Varianz	$Var(X) = \frac{1}{N-1} \sum_k (x_k - \bar{X})^2$
Kovarianz	$Cov(X, Y) = \frac{1}{N-1} \sum_k (x_k - \bar{X})(y_k - \bar{Y})$
Korrelation	$r_{xy} = Cov(X, Y) / \sqrt{Var(X)Var(Y)}$
Kovarianzmatrix	$C = \frac{1}{N-1} GG^T$, G = zentrierte Datenmatrix ($d \times N$)
Eigenwerte	$\det(C - \lambda I) = 0$; Eigenvektoren: $(C - \lambda_i I)\vec{e}_i = \vec{0}$
PCA hin	$\vec{g}' = U^T(\vec{g} - \vec{g}_{mean})$, $U = [\vec{e}_1 \dots \vec{e}_d]$
PCA zurück	$\vec{g} = \vec{g}_{mean} + U\vec{g}'$
Erklärte Varianz	$\lambda_i / \sum_j \lambda_j$; Gesamtvarianz = $tr(C) = \sum_i \lambda_i$
Kernel	$k(\vec{u}, \vec{v}) = \phi(\vec{u})^T \phi(\vec{v})$; poly: $(\vec{u}^T \vec{v} + c)^r$; RBF: $e^{-\gamma \ \vec{u} - \vec{v}\ ^2}$

Clustering

Größe	Formel
SSE	$\sum_{k=1}^K \sum_{\vec{g} \in C_k} dist(\vec{g}, \vec{z}_k)^2$
Zentrum (Update)	$\vec{z}_k = \frac{1}{ C_k } \sum_{\vec{g} \in C_k} \vec{g}$
Silhouette	$s(\vec{x}) = \frac{b(\vec{x}) - a(\vec{x})}{\max\{a(\vec{x}), b(\vec{x})\}} \in [-1, 1]$
Medoid	$m_k = \arg \min_{\vec{y} \in C_k} \sum_{\vec{x} \in C_k} dist(\vec{x}, \vec{y})$
DBSCAN Kernobjekt	$ U_\epsilon(\vec{x}) > MinPts$

Lineare Modelle und Metriken

Größe	Formel
Lineare Regression	$f(\mathbf{x}) = w_0 + \mathbf{w}^T \mathbf{x}; \quad L(\mathbf{w}) = \frac{1}{N} \ \mathbf{y} - \mathbf{X}\mathbf{w}\ ^2$
Normalengleichung	$\mathbf{w} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$
R^2	$1 - RSS/TSS; \quad RSS = \sum (y_i - \hat{y}_i)^2, \quad TSS = \sum (y_i - \bar{y})^2$
Sigmoid	$\sigma(z) = \frac{1}{1+e^{-z}}; \quad \sigma'(z) = \sigma(z)(1 - \sigma(z))$
Logistische Regr.	$f(\mathbf{x}) = \sigma(w_0 + \mathbf{w}^T \mathbf{x}); \quad \text{Grenze: } w_0 + \mathbf{w}^T \mathbf{x} = 0$
Log-Loss	$-\frac{1}{N} \sum_i (y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i))$
Softmax	$\text{softmax}(\mathbf{z})_k = e^{z_k} / \sum_j e^{z_j}$
Accuracy	$(N_{TP} + N_{TN})/N$
Precision	$N_{TP}/(N_{TP} + N_{FP})$
Recall (TPR)	$N_{TP}/(N_{TP} + N_{FN})$
FPR	$N_{FP}/(N_{FP} + N_{TN})$
Ridge / Lasso	$L + \lambda \sum_j w_j^2 \quad / \quad L + \lambda \sum_j w_j $
Standardisierung	$x' = (x - \bar{x})/\sigma; \quad \text{Min-Max: } x' = \frac{x - \min}{\max - \min}$

Neuronale Netze

Größe	Formel
Schicht	$\mathbf{a}^{[k]} = g(\mathbf{W}^{[k]} \mathbf{a}^{[k-1]} + \mathbf{b}^{[k]})$
Parameterzahl (FC)	pro Schicht: Neuronen $\times$ Eingänge + Neuronen
ReLU	$\max(0, z); \quad \text{Leaky ReLU: } \max(\varepsilon z, z)$
GD-Update	$\mathbf{w}^{[k+1]} = \mathbf{w}^{[k]} - \alpha^{[k]} \nabla L(\mathbf{w}^{[k]})$
SGD-Updates/Epoche	SGD: N ; Minibatch: N/m ; Batch: 1
Backprop	$\bar{x}_i = \sum_{j \text{ Kind}} \bar{x}_j \cdot \partial x_j / \partial x_i; \quad \text{Start } \bar{x}_N = 1$
Ableitung nach W	$\bar{W}^{[i]} = \bar{z}^{[i]} \cdot a^{[i-1]}$
Conv-Output	$W_{out} = \lfloor (W_{in} - W_K + 2P)/S \rfloor + 1; \quad D_{out} = K$
Conv-Parameter	$K \cdot (W_K \cdot H_K \cdot D_{in} + 1)$
Pooling-Output	$W_{out} = \lfloor (W_{in} - F)/S \rfloor + 1; \quad D_{out} = D_{in}; \quad \mathbf{0} \text{ Parameter}$
Autoencoder-Loss	$\frac{1}{N} \sum_i \ \mathbf{x}^{(i)} - \mathbf{x}'^{(i)}\ ^2$

Klausur-Checkliste: Die zehn Standardaufgaben

1. Lineare Regression: Vorhersage einsetzen + Loss aufstellen
2. Logistische Regression: σ -Wahrscheinlichkeit, Log-Loss, 50 %-Schwelle ($z = 0$)
3. Konfusionsmatrix: Accuracy / Precision / Recall + Anwendungsbegründung
4. Regularisierung: Ziel (Overfitting vermeiden, Flexibilität einschränken), Ridge vs. Lasso
5. Modelle zählen bei Gittersuche + Kreuzvalidierung
6. Forward-Pass durch kleines Netz mit ReLU (Funktion + Zahlenwert + Matrizen)
7. SGD-Dimensionen: Summanden, Gradientendimension, Update-Anzahl
8. Backpropagation: Graph, Vorwärts-, Rückwärtsphase (Verzweigung $\Rightarrow$ summieren!)
9. CNN: Faltung von Hand, Output-Dimension, Parameterzahl, Pooling
10. Autoencoder: Definition, Loss, PCA-Zusammenhang
Teil Preisach analog: Distanzen rechnen, Kovarianzmatrix, Mahalanobis, DTW-Matrix, SSE/k-Means, Silhouette, DBSCAN-Begriffe, PCA komplett.